

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 3 — Aplicacao, UI, Ferramentas e Build

Logica Aplicacao, UI Qt/PySide6, Design Atlas, Ingestion Pipeline, Ferramentas
Diagnosticas, Suite de Testes, Remediacao

Autor: Renan Augusto Macena

Agosto 2026

Ultimate CS2 Coach — Parte 3: Programa, UI, Ferramentas e Build

Topicos: Schema completo do banco de dados (three-tier storage, SQLite WAL, SQLAlchemy ORM), Regime de treinamento e limites de maturidade (CALIBRATING/LEARNING/MATURE), Catalogo das funcoes de perda, Logica Completa do Programa (do lancamento ao conselho): Session Engine, Digester, Teacher, Hunter, Pulse; Interface desktop Qt/PySide6 (15 telas, ViewModels, Qt Signals); Pipeline de ingestao (demo e pro); Console de controle unificada; Onboarding de novo usuario; Arquitetura de storage; Motor de playback e viewer tatico; Dados espaciais e mapas; Observabilidade e logging; Reporting; Quotas e limites; Tolerancia a falhas; Jornada completa do usuario (4 fluxos); Suite de ferramentas (validacao e diagnostico); Test suite; Pre-commit hooks; Build, packaging, deployment; Migracoes Alembic; HLTV sync service; RASP Guard; MatchVisualizer; Arquivos de configuracao runtime; Entry point root-level; Glossario.

Autor: Renan Augusto Macena

Revisao base: em sincronia com [Book-Coach-3.md](#) (canonico italiano) em 2026-08-15.

Este documento e a continuacao da **Parte 2 — Servicos, Analise e Banco de Dados**, que cobre os servicos de coaching, os motores de analise, o conhecimento e o Banco de Dados. A Parte 3 desce ao nivel de programa: como o sistema inteiro se inicializa, como os daemons se orquestram, como a interface desktop torna visivel o coaching, como o codigo e testado, validado e implantado.

Indice

Parte 3 — Programa, UI, Ferramentas e Build (este documento)

9. [Schema do banco de dados e ciclo de vida dos dados](#)
10. [Regime de treinamento e limites de maturidade](#)
11. [Catalogo das funcoes de perda](#)
12. [Logica Completa do Programa — Do Lancamento ao Conselho](#)
 - 12.1 Inicializacao da aplicacao e bootstrap
 - 12.2 Session Engine e Quad-Daemon
 - 12.3 Daemon Digester e pipeline de ingestao
 - 12.4 Daemon Teacher e training orchestrator
 - 12.5 Interface Desktop (Qt/PySide6, 15 telas, ViewModels)
 - 12.6 Pipeline de Ingestao ([ingestion/](#))
 - 12.7 Console de Controle Unificada ([backend/control/](#))
 - 12.8 Onboarding e Fluxo de Novo Usuario
 - 12.9 Arquitetura de Storage ([backend/storage/](#))
 - 12.10 Motor de Playback e Viewer Tatico
 - 12.11 Dados Espaciais e Gestao de Mapas
 - 12.12 Observabilidade e Logging

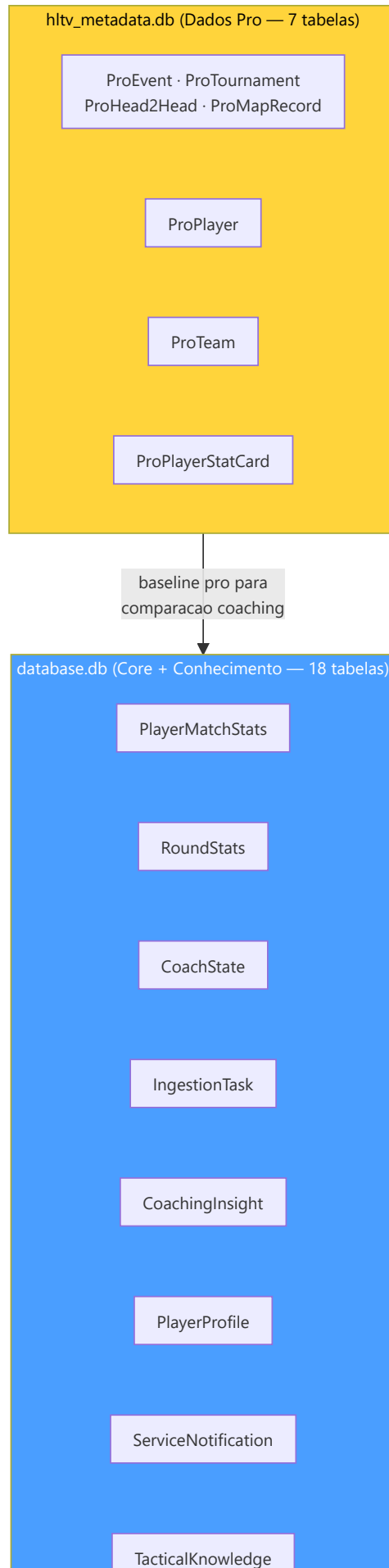
- 12.13 Reporting e Visualizacao
 - 12.14 Gestao de Quotas e Limites
 - 12.15 Tolerancia a Falhas e Recuperacao
 - 12.16 Jornada Completa do Usuario — 4 Fluxos Principais
 - 12.17 Suite de Ferramentas — Validacao e Diagnostico (`tools/`)
 - 12.18 Arquitetura da Test Suite (`tests/`)
 - 12.19 As Fases de Remediacao Sistemica
 - 12.19.1 A campanha de auditoria integral (agosto de 2026)
 - 12.20 Pre-commit Hooks e Quality Gates
 - 12.21 Build, Packaging e Deployment
 - 12.22 Sistema de Migracoes Alembic
 - 12.23 Orquestrador de Ingestao Principal (`run_ingestion.py`)
 - 12.24 HLTV Sync Service e Background Daemon
 - 12.25 RASP Guard — Integridade Runtime doCodigo
 - 12.26 MatchVisualizer — Rendering Avancado
 - 12.27 Arquivos de Dados e Configuracao Runtime
 - 12.28 Entry Point Root-Level
- [Resumo Arquitetural](#)
 - [Mapa das Interconexoes entre as 3 Partes](#)
 - [Glossario Tecnico](#)

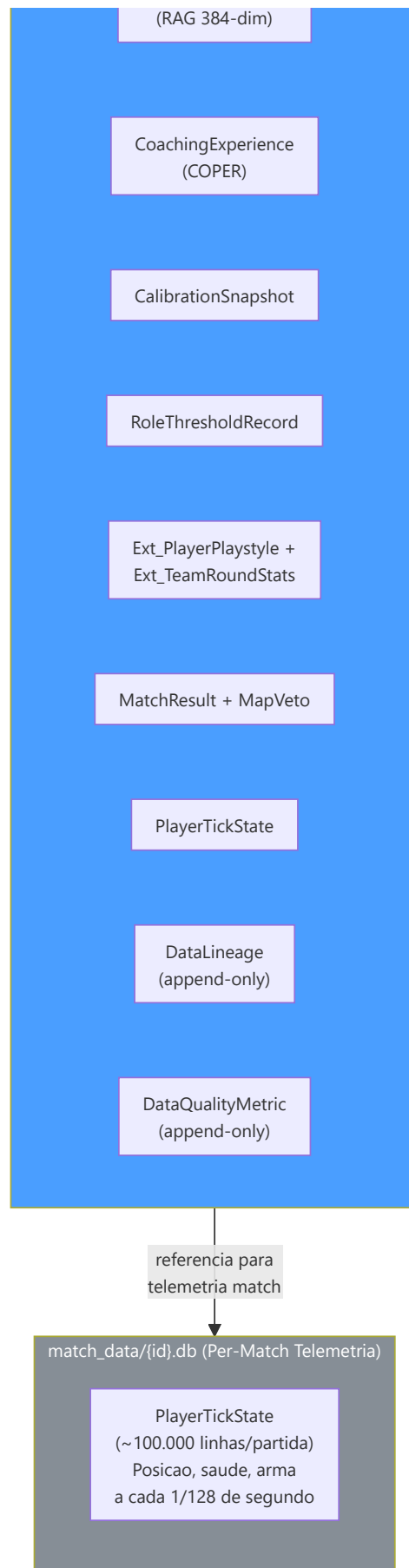
9. Schema do banco de dados e ciclo de vida dos dados

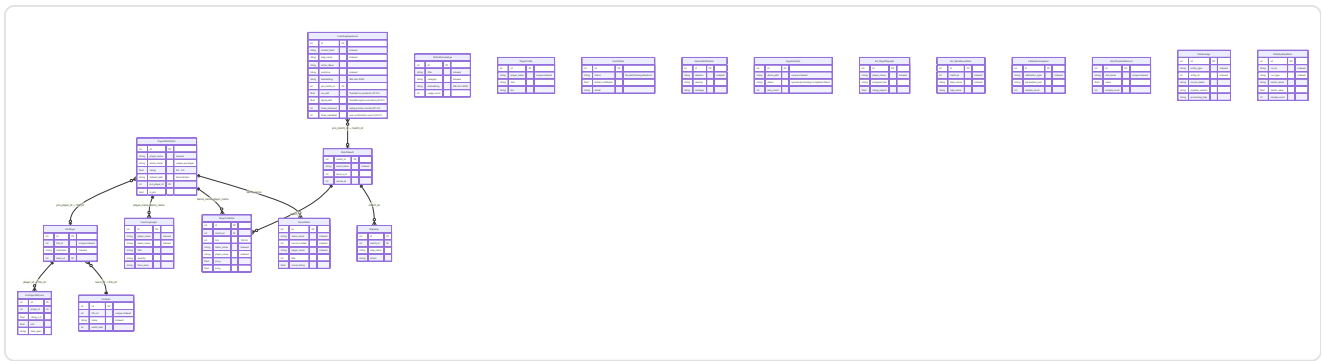
O projeto usa **SQLModel** (Pydantic + SQLAlchemy) com SQLite (modo WAL) e uma **arquitetura three-tier storage** especializada. No total **28 tabelas SQLModel** (`backend/storage/db_models.py` : 25 tabelas monolite+HLTV; `backend/storage/match_data_manager.py` : 3 tabelas por-match) distribuidas em 3 tiers de storage:

1. **database.db** — Banco de dados monolite principal da aplicacao (**18 tabelas**, listadas explicitamente em `database.py:_MONOLITH_TABLES`). Contem todas as tabelas core: estatisticas de jogadores (`PlayerMatchStats`), estado do coach (`CoachState`), tasks de ingestao (`IngestionTask`), insights de coaching (`CoachingInsight`), perfis de usuario (`PlayerProfile`), notificacoes de sistema (`ServiceNotification`), base RAG (`TacticalKnowledge`), banco de experiencias COPER (`CoachingExperience`), resultados de partidas (`MatchResult` , `MapVeto`), calibracoes (`CalibrationSnapshot`), limites de papel (`RoleThresholdRecord`), rastreamento de proveniencia (`DataLineage` , `DataQualityMetric`), e tabelas estendidas para round team (`Ext_TeamRoundStats`) e estilo de jogo (`Ext_PlayerPlaystyle`), alem de `RoundStats` e `PlayerTickState` arquivais.
2. **hlvtv_metadata.db** — Banco de dados dos metadados profissionais (**7 tabelas**, `database.py:_HLTV_TABLES`): perfis dos jogadores pro (`ProPlayer` , `ProTeam`), cartoes estatisticos (`ProPlayerStatCard`) mais as tabelas estendidas `ProEvent` , `ProTournament` , `ProHead2Head` , `ProMapRecord` . Separado do monolite porque e escrito por um processo separado (HLTV sync service) para eliminar a contencao WAL com os daemons do session engine.
3. **match_data/{id}.db** — Banco de dados por-match de telemetria (**3 tabelas**, definidas em `match_data_manager.py` : `MatchTickState` , `MatchEventState` , `MatchMetadata`). Cada partida tem seu proprio arquivo SQLite dedicado contendo os dados tick-a-tick (~100.000 linhas por partida). Gerenciado por `MatchDataManager` . Essa separacao resolve o problema do "Telemetry Cliff" — evita que o banco de dados monolite cresca indefinidamente com dados de alta frequencia.

Essa separacao em tres niveis garante que as operacoes de escrita intensivas do session engine (ingestao demo, treinamento ML → `database.db`) nao disputem locks WAL com o scraping HLTV em processo separado (`hltv_metadata.db`), e que a telemetria de alta frequencia por-match nao sobrecarregue o monolite (`match_data/{id}.db`).







Descricao das tabelas: **PlayerMatchStats** — estatísticas agregadas por jogador por partida (kills, deaths, ADR, rating HLTV 2.0, 25 features normalizadas). **PlayerTickState** — telemetria tick-by-tick a 64/128 Hz: posicao (x,y,z), saude, armadura, angulo de visao, arma atual, economia, estado para cada jogador em cada tick. **RoundStats** — estatísticas isoladas por round: kills, deaths, dano, noscope kills, flash assists, round rating, uteis para coaching granular. **CoachingExperience** — registro COPER: contexto do momento de coaching, conselho dado, resultado (positivo/negativo/neutro), eficacia numerica, embedding 384-dim para busca semantica. **CoachingInsight** — insights de coaching gerados pelo sistema e mostrados ao usuario na UI. **TacticalKnowledge** — base de conhecimento RAG: estrategias, posicionamentos, uso de utilidades, indexados com embeddings Sentence-BERT 384-dim, versao v3, 14 categorias. **RoleThresholdRecord** — limites aprendidos para a classificacao dos 10 papeis taticos, persistidos entre reinicios. **CalibrationSnapshot** — timestamp e contagem de amostras de cada auto-calibracao do estimador Bayesiano de morte. **Ext_PlayerPlaystyle** — metricas de estilo de jogo de CSVs externos, usadas para treinamento do NeuralRoleHead. **ServiceNotification** — mensagens de erro/evento dos daemons em background, mostradas na UI via polling. **DataLineage** — audit trail append-only: rastreia qual demo originou cada entidade e atraves de qual step do pipeline foi processada. **DataQualityMetric** — metricas quantitativas de qualidade por execucao do pipeline (percentual de amostras descartadas, taxa de fallback zero-tensor).

Ciclo de vida dos dados:

Fase	Tabelas escritas	Volume
Insercao demo	PlayerMatchStats , PlayerTickState , MatchMetadata	~100.000 tick/partida
Enriquecimento round	RoundStats (isolamento por round, por jogador)	~30 linhas/partida (round x jogadores)
Scan HLTV	ProPlayer , ProTeam , ProPlayerStatCard	~500 jogadores
Importacao CSV	Tabelas externas via csv_migrator.py	~10.000 linhas
Dados de estilo de jogo	Ext_PlayerPlaystyle (de CSV para NeuralRoleHead)	~300+ jogadores
Engenharia de features	PlayerMatchStats.dataset_split atualizado	In-place
Populacao RAG	TacticalKnowledge	~200 artigos
Extracao de experiencia	CoachingExperience	~1.000 por partida
Output de coaching	CoachingInsight	~5-20 por partida
Aprendizado de limites de papel	RoleThresholdRecord	9 limites
Calibracao de crenças	CalibrationSnapshot (apos retreinamento)	1 por cada calibracao executada
Telemetria de sistema	CoachState , ServiceNotification , IngestionTask	Continuo
Rastreamento de proveniencia	DataLineage (append-only por cada entidade processada)	~N linhas por demo ingerida
Metricas de qualidade pipeline	DataQualityMetric (append-only por cada execucao pipeline)	~5-10 metricas por run
Backup	Automatizado via BackupManager (7 rotacoes diarias + 4 semanais)	Copia completa do banco

Indices e otimizacao de query:

As tabelas mais consultadas tem indices estrategicos para garantir queries rapidas:

Tabela	Indice	Colunas	Tipo de query otimizada
PlayerMatchStats	idx_pms_player	player_name	Busca por jogador
PlayerMatchStats	idx_pms_demo	demo_name	Busca por partida
PlayerMatchStats	idx_pms_processed	processed_at	Ordenacao cronologica
RoundStats	idx_rs_demo_round	demo_name, round_number	Busca por round especifico
IngestionTask	idx_it_status	status	Fila de trabalho (status=queued)
CoachingInsight	idx_ci_player	player_name	Insight por jogador
TacticalKnowledge	idx_tk_category	category	Busca RAG por categoria
ProPlayer	idx_pp_name	nickname	Busca pro por nome
DataLineage	idx_dl_entity	entity_type, entity_id	Rastreamento proveniencia por entidade
DataQualityMetric	idx_dqm_run	run_id, run_type	Metricas qualidade por execucao

Restricoes de integridade:

Restricao	Tabelas	Enforcement
<code>player_name</code> NOT NULL	PlayerMatchStats, RoundStats	Nivel de schema
<code>demo_name</code> UNIQUE por player	PlayerMatchStats	Previne duplicatas
<code>status</code> CHECK IN ('queued', 'processing', 'completed', 'failed')	IngestionTask	Enum enforcement
<code>dataset_split</code> CHECK IN ('train', 'val', 'test')	PlayerMatchStats	Split validity
Foreign key demo_name	RoundStats → PlayerMatchStats	Relacao round-partida

Detalhe de tabelas-chave:

PlayerMatchStats (32 campos) — a tabela mais consultada do sistema:

A tabela PlayerMatchStats contem todas as estatisticas agregadas por jogador por partida. E o "boletim" de cada partida analisada:

Campo	Tipo	Descricao
<code>id</code>	Integer PK	Identificador unico
<code>player_name</code>	String	Nome do jogador (indexed)
<code>demo_name</code>	String	Nome do arquivo demo (unique por player)
<code>kills, deaths, assists</code>	Integer	KDA base
<code>adr</code>	Float	Average Damage per Round
<code>kast</code>	Float	Kill/Assist/Survive/Trade %
<code>headshot_percentage</code>	Float	HS%
<code>hltv_rating</code>	Float	Rating HLTV 2.0 calculado
<code>dataset_split</code>	String	"train" / "val" / "test"
<code>is_pro</code>	Boolean	Flag jogador profissional
<code>processed_at</code>	DateTime	Timestamp de processamento
<code>map_name</code>	String	Mapa jogado
...	...	20+ campos adicionais para features avancadas

TacticalKnowledge — a base RAG:

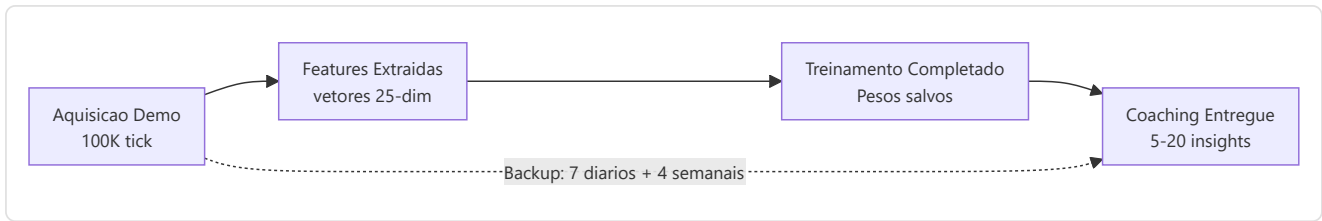
Campo	Tipo	Descricao
<code>id</code>	Integer PK	Identificador
<code>title</code>	String	Titulo do documento (indexed)
<code>description</code>	String	Descricao do conteudo tatico
<code>category</code>	String	"positioning" / "economy" / "utility" / "aim" (indexed)
<code>map_name</code>	String	Mapa especifico (indexed, opcional)
<code>situation</code>	String	Contexto situacional: "T-side pistol round", "CT retake A site"
<code>pro_example</code>	String	Referencia a demo pro (opcional)
<code>embedding</code>	String	Vetor 384-dim JSON (sentence-transformers)
<code>created_at</code>	DateTime	Timestamp de criacao
<code>usage_count</code>	Integer	Contador de uso

A busca semantica RAG funciona calculando a **cosine similarity** entre o embedding da query do usuario e os embeddings precomputados de cada documento. Os top-3 resultados com similarity > 0.5 sao usados para enriquecer o contexto de coaching. O campo `situation` permite o filtro contextual (ex. "T-side pistol round") antes da busca semantica.

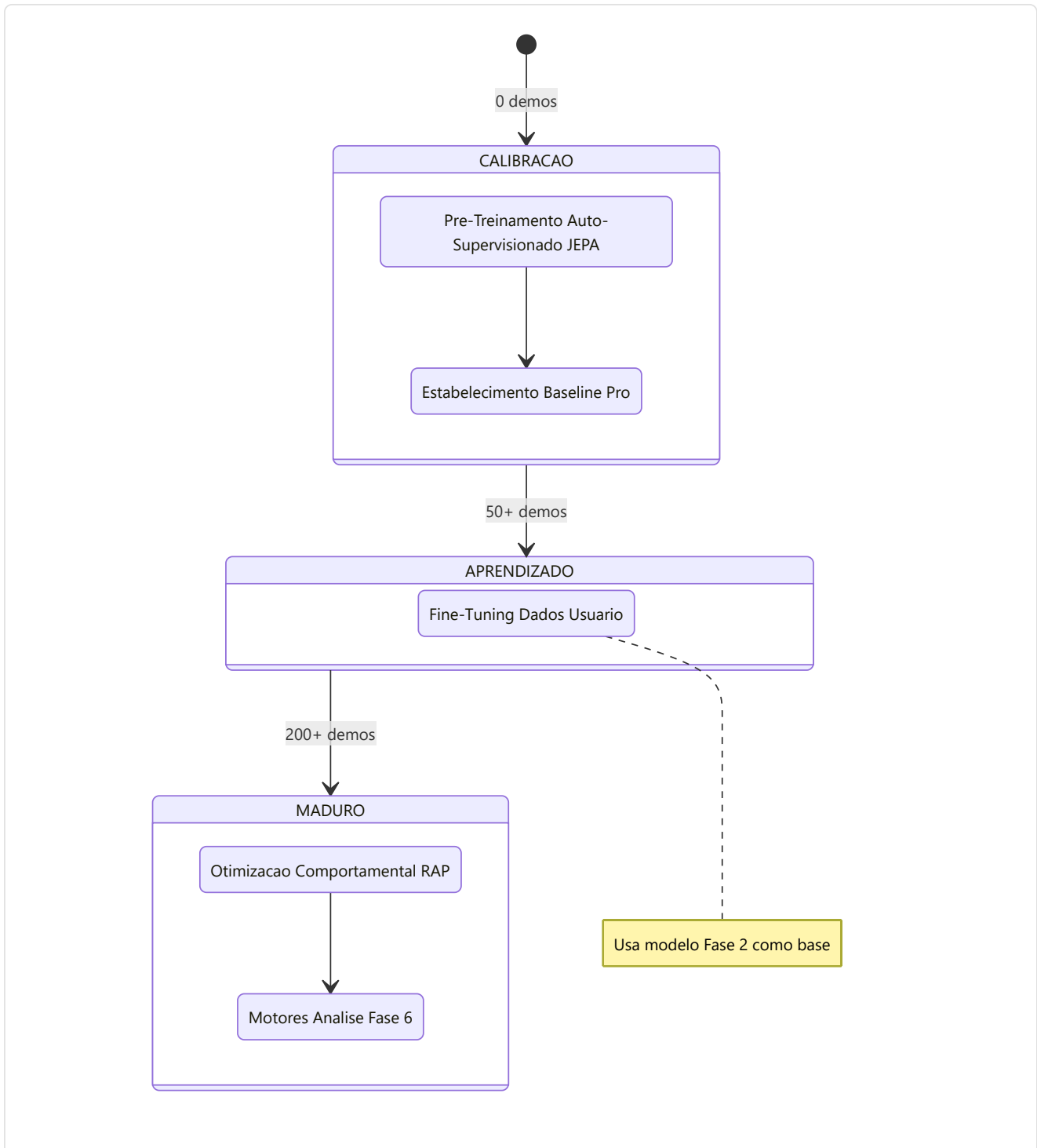
CoachingExperience — o banco COPER (22+ campos):

Campo	Tipo	Descricao
<code>id</code>	Integer PK	Identificador
<code>context_hash</code>	String	Hash do estado de jogo para lookup rapido (indexed)
<code>map_name</code>	String	Mapa (indexed)
<code>round_phase</code>	String	"pistol" / "eco" / "full_buy" / "force"
<code>side</code>	String	"T" / "CT"
<code>position_area</code>	String	"A-site" / "Mid" / etc. (indexed, opcional)
<code>game_state_json</code>	String	Snapshot completo do tick (max 16KB, validado com <code>field_validator</code>)
<code>action_taken</code>	String	"pushed" / "held_angle" / "rotated" / "used_utility" / etc.
<code>outcome</code>	String	"kill" / "death" / "trade" / "objective" / "survived" (indexed)
<code>delta_win_prob</code>	Float	Variacao da probabilidade de vitoria desta acao
<code>confidence</code>	Float	Confiabilidade/generalizabilidade 0.0-1.0
<code>usage_count</code>	Integer	Quantas vezes recuperada para coaching
<code>pro_match_id</code>	Integer FK	Referencia a MatchResult (ON DELETE SET NULL)
<code>pro_player_name</code>	String	Nome jogador pro de referencia (indexed, opcional)
<code>embedding</code>	String	Vetor 384-dim JSON para busca semantica (opcional)
<code>source_demo</code>	String	Demo de origem (opcional)
<code>created_at</code>	DateTime	Timestamp de criacao
<code>outcome_validated</code>	Boolean	Se o resultado foi validado
<code>effectiveness_score</code>	Float	Score -1.0 a 1.0
<code>follow_up_match_id</code>	Integer	Partida de follow-up para tracking
<code>times_advice_given</code>	Integer	Quantas vezes o conselho foi dado
<code>times_advice_followed</code>	Integer	Quantas vezes o conselho foi seguido
<code>last_feedback_at</code>	DateTime	Ultimo feedback recebido
<code>mu_skill</code>	Float	TrueSkill mu posterior — estimativa da qualidade da experiencia (KT-01)
<code>sigma_skill</code>	Float	TrueSkill sigma uncertainty — incerteza sobre a qualidade (KT-01)
<code>times_retrieved</code>	Integer	Contador replay priority — quantas vezes recuperada (KT-01)
<code>times_validated</code>	Integer	Contador de confirmacoes do usuario — quantas vezes validada (KT-01)

O sistema COPER usa essas experiencias para **aprender com seus proprios conselhos**: o campo `context_hash` permite o lookup rapido de situacoes similares, `outcome` e `delta_win_prob` medem a eficacia da acao, e o ciclo de feedback (`outcome_validated` , `times_advice_given/followed` , `effectiveness_score`) permite ao sistema priorizar conselhos que historicamente produziram resultados positivos. O campo `game_state_json` e limitado a 16KB para prevenir crescimento descontrolado do banco. Os campos TrueSkill `mu_skill` / `sigma_skill` (KT-01) fornecem uma estimativa bayesiana da qualidade da experiencia, usada para a priorizacao do replay: $confidence_score = mu_skill - kappa \times sigma_skill$. O contador `times_retrieved` implementa a replay priority, enquanto `times_validated` rastreia as confirmacoes do usuario para o CRUD semantico.

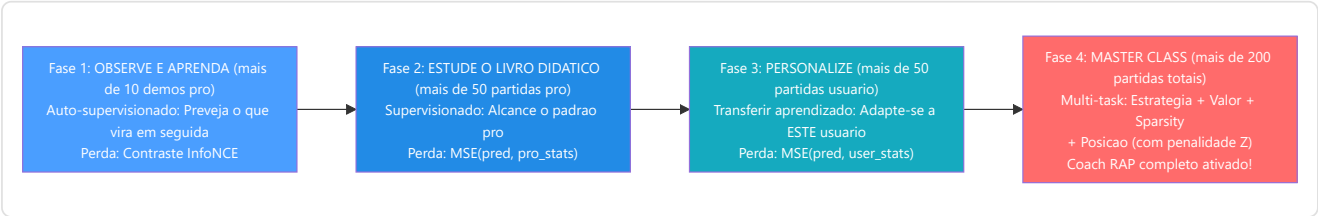


10. Regime de treinamento e limites de maturidade



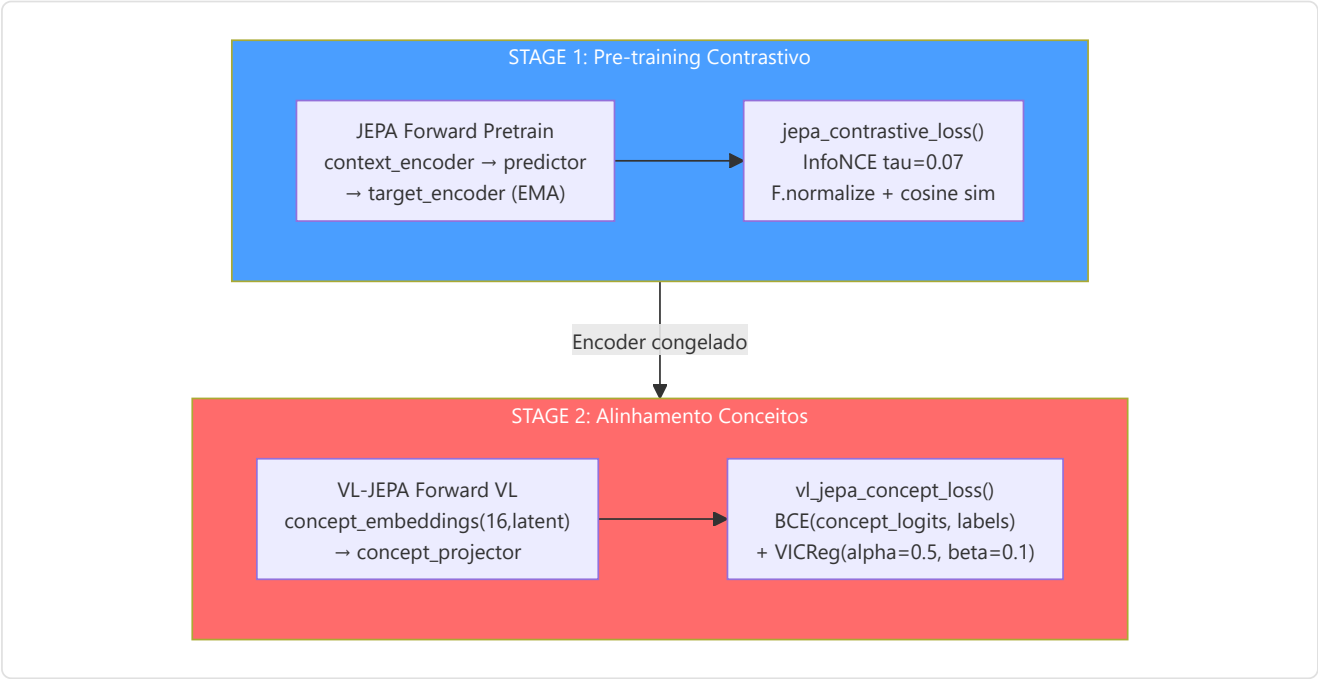
Requisitos de dados por fase:

Fase	Dados minimos	Tipo de treinamento	Perda primaria
1. Pre-treinamento JEPA	10 demos pro	Auto-supervisionado (InfoNCE)	Contrastivo com negativos em batch
2. Baseline pro	50 partidas pro	Supervisionado	MSE(pred, pro_stats)
3. Otimizacao usuario	50 partidas usuario	Supervisionado (transferencia)	MSE(pred, user_stats)
4. Otimizacao RAP	200 partidas totais	Multi-task	Estrategia + Valor + Sparsidade + Posicao



Protocolo VL-JEPA Two-Stage (Alinhamento de Conceitos):

Quando o VL-JEPA esta ativo, as Fases 1-2 sao estendidas com um **protocolo two-stage** que alinha as representacoes latentes aos 16 coaching concepts:



Stage	O que se treina	O que esta congelado	Loss	Proposito
1	Context encoder, predictor	Target encoder (EMA)	InfoNCE (tau=0.07)	Representacoes latentes gerais
2	Concept embeddings, concept projector, concept temperature	Encoder (opcional fine-tuning)	BCE + VICReg diversity	Alinhamento aos 16 coaching concepts

Os 16 Coaching Concepts (taxonomia — de `COACHING_CONCEPTS` em `jepa_model.py`):

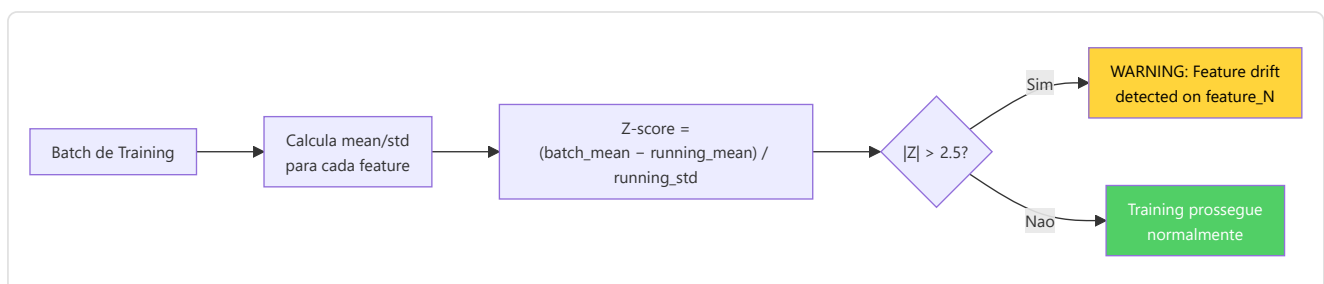
Indice	Conceito	Dimensao	Descricao
0	positioning_aggressive	Posicionamento	Combates de curta distancia, push de angulos
1	positioning_passive	Posicionamento	Angulos longos, evita o contato
2	positioning_exposed	Posicionamento	Posicao vulneravel, alta probabilidade de morte
3	utility_effective	Utility	Utilidade com impacto significativo
4	utility_wasteful	Utility	Utilidade nao usada ou de baixo impacto
5	economy_efficient	Decisao	Equipamento alinhado ao tipo de round
6	economy_wasteful	Decisao	Force-buy desfavoraveis ou morte com gear caro
7	engagement_favorable	Engajamento	Combates com vantagem de HP / posicao / numeros
8	engagement_unfavorable	Engajamento	Combates em desvantagem numerica ou de HP
9	trade_responsive	Engajamento	Trade kills rapidos, boa coordenacao
10	trade_isolated	Engajamento	Morte sem trade, jogando isolado demais
11	rotation_fast	Decisao	Rotacao posicional rapida apos informacao
12	information_gathered	Decisao	Boa coleta de intel, inimigos localizados
13	momentum_leveraged	Psicologia	Capitaliza hot streak com jogadas confiantes
14	clutch_composed	Psicologia	Decisoes calmas em situacoes 1vN
15	aggression_calibrated	Psicologia	Agressividade apropriada a situacao

AdamW + CosineAnnealing (JEPA Trainer):

Hiperparametro	Valor	Proposito
Optimizer	AdamW	Weight decay separado dos gradientes
Learning rate	1e-4 (default; camadas concept a lr x0.05)	Taxa de aprendizado inicial
Weight decay	0.01	Regularizacao L2
Scheduler	SequentialLR: warmup linear (5%) → CosineAnnealingLR	Warmup + decaimento cosseno do LR
EMA decay	0.996 base, agendamento cosseno → 1.0	Target encoder momentum update
AMP + acumulo	GradScaler (CUDA) + 4 passos de acumulo	Eficiencia em GPUs com pouca VRAM
Gradient clip	1.0	Prevencao de gradient explosion

DriftMonitor (z_threshold=2.5):

O **DriftMonitor** integrado no JEPA Trainer monitora o **drift das features** durante o training. Se o Z-score de uma feature superar 2.5, emite um warning que indica possivel distribuicao shifting:



Trigger de retreinamento: O daemon Teacher monitora o crescimento do numero de demos profissionais; ativa o retreinamento quando `count ≥ last_count x 1,10`.

JEPAPretrainDataset (`jepa_train.py`):

O dataset de pre-training JEPA usa **janelas temporais** para criar pares contexto-target:

Parametro	Valor	Proposito
<code>context_len</code>	10 ticks	Tamanho janela de contexto (input)
<code>target_len</code>	10 ticks	Tamanho janela target (a prever)
<code>seed</code>	42	RNG dedicado para janelas reprodutíveis
Batch size (pretrain standalone)	16	Numero de pares por batch
Restricoes de sequencia	min 20 / max 500 ticks	<code>_MIN_TICKS_FOR_SEQUENCE</code> / <code>_MAX_TICKS_PER_SEQUENCE</code>

11. Catalogo das funcoes de perda

Modelo	Nome da perda	Formula	Proposito
JEPA	InfoNCE Contrastive	<code>-log(exp(sim(pred, target)/tau) / Sigma exp(sim(pred, neg_i)/tau))</code> , tau=0.07, <code>F.normalize</code> antes da similaridade cosseno	Alinhamento das previsoes de contexto com os embeddings do target
JEPA	Otimizacao	<code>MSE(coaching_head(Z_ctx), y_true)</code>	Pontuacao de coaching supervisionado
AdvancedCoachNN	Supervisionado	<code>MSELoss(MoE_output, y_true)</code>	Treinamento a nivel de partida
RAP	Estrategia	<code>MSELoss(advice_probs, target_strat)</code>	Recomendacao tatica correta
RAP	Valor	<code>0,5 x MSE(V(s), true_advantage)</code>	Estimativa precisa da vantagem
RAP	Sparsidade	<code>Entropia(gate_probs) x context_gate_l1_weight (1e-4)</code>	Especializacao de especialistas
RAP	Posicao	<code>MSE(xy) + 2x MSE(z)</code>	Posicionamento otimo com penalidade no eixo Z
WinProb	Previsao	<code>BCELoss(pred, resultado)</code> (a saida ja passou por Sigmoid)	Previsao do resultado do round
NeuralRoleHead	KL-Divergence	<code>KLDivLoss(log_softmax(pred), target)</code> com smoothing de labels epsilon=0,02	Correspondencia da distribuicao de probabilidade do papel
VL-JEPA	Alinhamento de conceitos	<code>BCE(concept_logits, concept_labels) + VICReg(concept_diversity)</code>	Fundamentos do conceito de linguagem visual

Detalhe: InfoNCE Contrastive Loss (JEPA)

O InfoNCE e a loss principal do pre-training JEPA. Seu proposito e alinhar as predicoes do contexto com os embeddings do target, rejeitando simultaneamente os negativos (outras amostras no batch):

```
L_InfoNCE = -log( exp(sim(pred, target+)/ tau) / Sigma_i exp(sim(pred, target_i) / tau) )
```


Componente	Valor	Papel
<code>sim()</code>	Cosine similarity apos <code>F.normalize</code>	Medida de similaridade [-1, +1]
<code>tau</code> (temperature)	0.07	Sharpness da distribuicao — valores baixos = mais seletivos
<code>target+</code>	O embedding target correto para este contexto	O "positivo" — a resposta correta
<code>target_i</code>	Todos os embeddings no batch	Negativos in-batch — as respostas erradas
Batch size	32	Numero de negativos = batch_size - 1 = 31

Detalhe: VL-JEPA Concept Loss (VICReg Components)

A loss de alinhamento de conceitos do VL-JEPA combina dois componentes:

```
L_concept = BCE(concept_logits, concept_labels) + alpha*VICReg_diversity
```

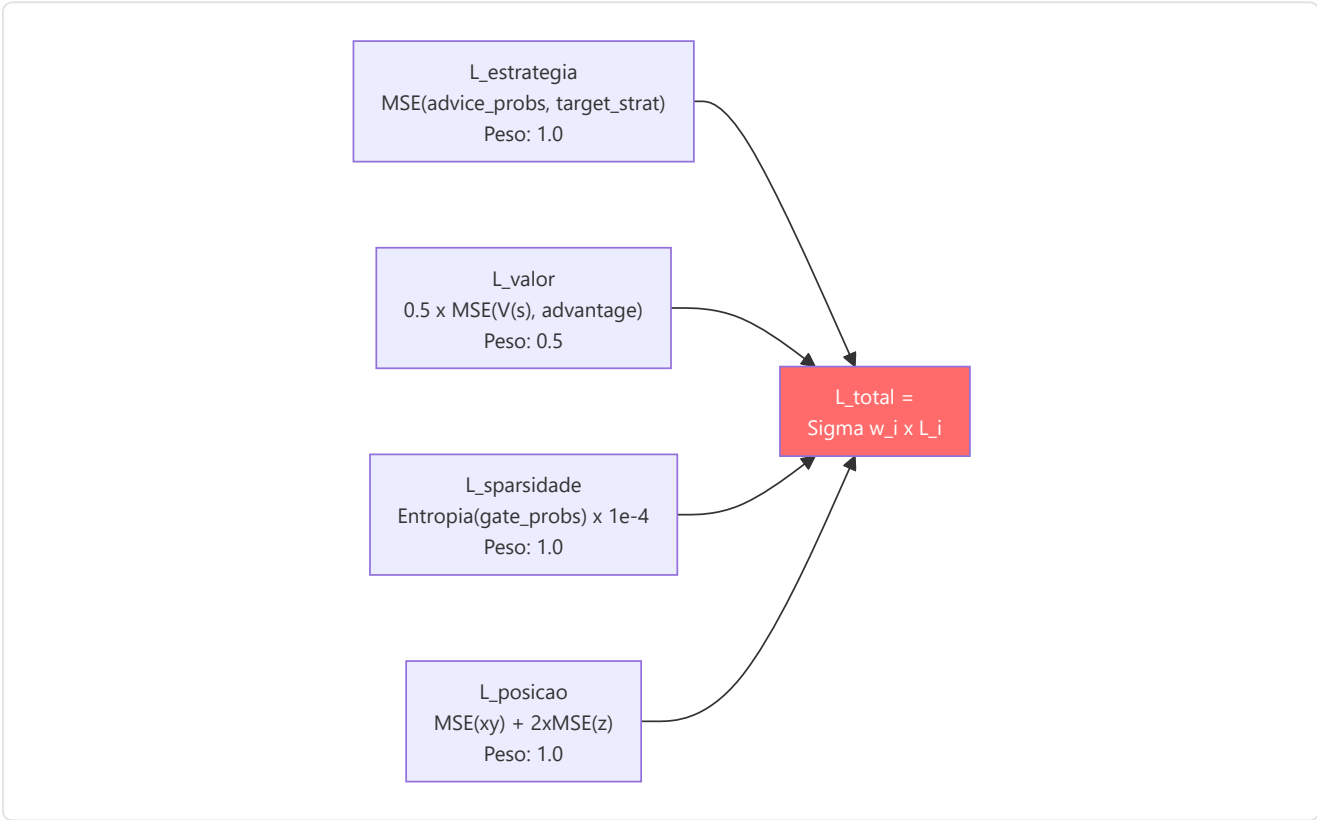
No detalhe de implementacao:

Termo	Formula	Peso	Proposito
Variance	<code>max(0, gamma - std(z))</code> para cada dimensao	alpha=0.5	Previne o colapso das representacoes — cada dimensao deve variar
Diversity	<code>-std(L2_norm(concept_embeddings), dim=0).mean()</code>	beta=0.1	Previne o colapso — os embeddings de conceito devem permanecer distintos

Uma regularizacao VICReg completa (`vicreg_regularization`, `lambda_var=25.0`, `lambda_cov=1.0`) esta disponivel separadamente e e adicionada com peso 0.01 a loss InfoNCE no trainer JEPA.

Detalhe: RAP Multi-Task Loss

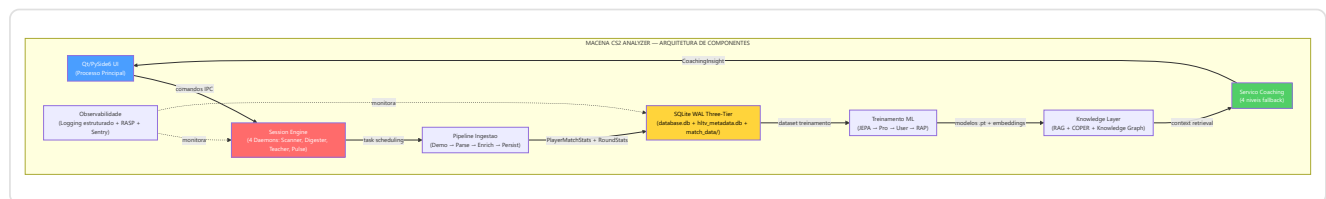
O RAP Coach combina 4 losses em uma loss total ponderada:



A penalidade Z 2x na loss de posicao reflete o fato de que em CS2 errar o **andar** (em cima vs embaixo em Nuke/Vertigo) e muito mais grave do que errar a posicao horizontal. Um erro de 100 unidades no eixo Z (andar errado) e estrategicamente catastrofico, enquanto o mesmo erro em X/Y pode ser irrelevante.

12. Logica Completa do Programa — Do Lancamento ao Conselho

Este capitulo documenta a **logica completa** do Macena CS2 Analyzer, desde o momento em que o usuario lanca a aplicacao ate quando recebe os conselhos de coaching. Ao contrario dos capitulos anteriores que se concentram nos subsistemas AI, aqui e explicado como **cada componente do programa** trabalha em conjunto: a interface desktop, a arquitetura quad-daemon, a pipeline de ingestao, o sistema de storage, o playback tatico, a observabilidade e o ciclo de vida da aplicacao.



12.1 Ponto de Entrada e Sequencia de Inicializacao

O sistema dispoe de **um entry point principal** (Qt) e tres entry points de utilidade:

#	Entry Point	Comando	Papel
1	Qt (primario)	<code>python -m Programma_CS2_RENAN.apps.qt_app.app</code>	UI desktop PySide6
2	Session Engine	<code>python -m Programma_CS2_RENAN.backend.console</code>	Backend headless
3	Headless Validator	<code>python -m Programma_CS2_RENAN.tools.headless_validator</code>	Validacao CI/CD
4	HLTV Sync	<code>python -m Programma_CS2_RENAN.tools.hltv_sync</code>	Scraping dados pro

12.1.1 Entry Point Qt (Primario) — `apps/qt_app/app.py`

Arquivo: `Programma_CS2_RENAN/apps/qt_app/app.py`

A sequencia de inicializacao Qt e baseada em **QApplication** e **signal/slot**:

- High-DPI setup** — Habilita scaling automatico para displays de alta densidade
- QApplication** — Cria a instancia aplicacao Qt com gestao de args
- Tema e font** — Registra os 3 temas (CS2, CSGO, CS1.6) via QSS e as fontes personalizadas
- MainWindow** — Constroi `QMainWindow` com sidebar + `QStackedWidget` (15 telas)
- Signal wiring** — Conecta os sinais Qt entre sidebar, telas e backend
- First-run gate** — Se `SETUP_COMPLETED=False`, mostra o wizard; caso contrario a home
- Backend console boot** — Lanca o Session Engine como subprocess
- Window show** — Mostra a janela e inicia o loop de eventos Qt
- CoachState polling** — Timer Qt para atualizacao periodica do estado

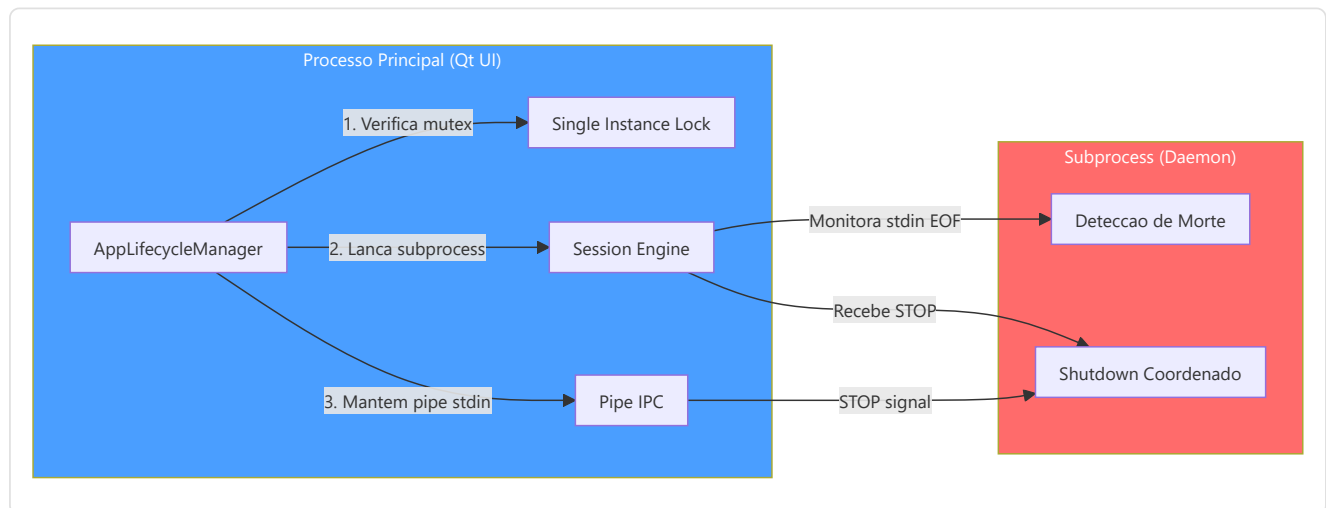
12.2 Gestao do Ciclo de Vida (`lifecycle.py`)

Arquivo: `Programma_CS2_RENAN/core/lifecycle.py`

O `AppLifecycleManager` é um **Singleton** que gerencia o ciclo de vida da aplicacao inteira: desde a garantia de que exista uma unica instancia ativa, ao lancamento do subprocess daemon, ate o shutdown coordenado.

Mecanismos principais:

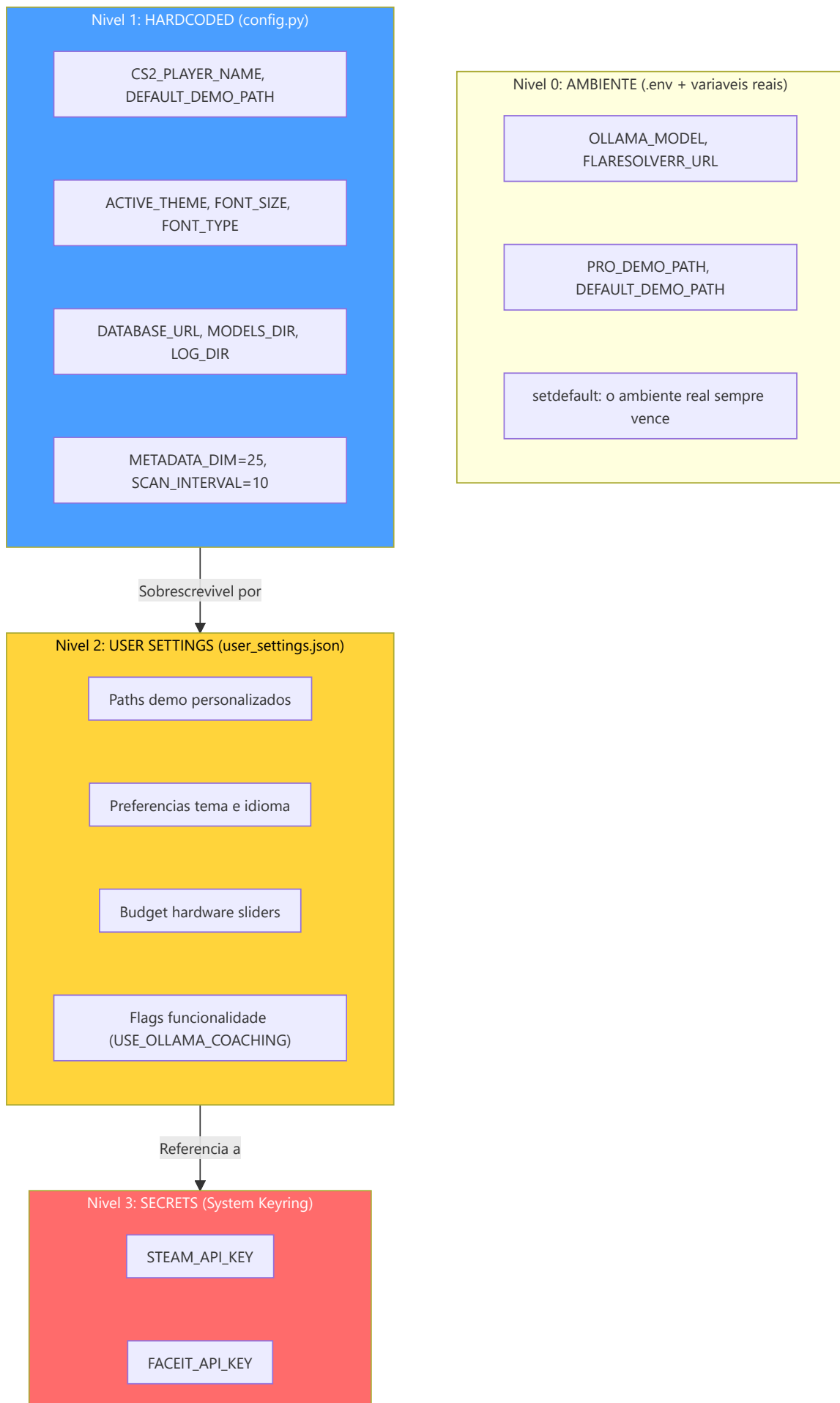
Mecanismo	Implementacao	Proposito
Single Instance Lock	Windows Named Mutex / file lock em Linux	Impede instancias multiplas (corrupcao DB)
Lancamento Daemon	<code>subprocess.Popen(session_engine.py)</code> com <code>PYTHONPATH</code>	Processo separado para trabalho pesado
Deteccao Morte do Pai	O daemon monitora EOF em <code>stdin</code>	Se o processo principal morre, o daemon para
Shutdown Graceful	Envio "STOP" via stdin → daemon termina em 5s	Nenhuma perda de dados ou tasks zumbis
Status Polling	UI consulta <code>CoachState</code> a cada 10s	Atualizacao status daemon sem IPC direto
Error Recovery	Se daemon morre, erro registrado + ServiceNotification	Usuario informado, nenhum crash silencioso
Keep-Alive	UI verifica heartbeat a cada 15s	Se heartbeat > 30s stale → warning "Daemon nao responde"



12.3 Sistema de Configuracao (`config.py`)

Arquivo: `Programma_CS2_RENAN/core/config.py`

O sistema utiliza **quatro níveis de configuracao**, cada um com um diferente nível de persistencia e seguranca:



Nunca em JSON, nunca em log

O nível 0 e a licao que ele carrega. Até agosto de 2026 o arquivo `.env` era uma superfície de configuracao documentada mas nunca lida: nenhuma dependencia o interpretava e nenhum script o carregava, portanto todo override descrito na documentacao era uma instrucao que nao fazia nada — falhava em silencio, que e o pior modo de falhar. Hoje `_load_dotenv_file()` em `core/config.py` o analisa usando apenas a biblioteca padrao, no momento do import do modulo, e o faz com tres cautelas deliberadas:

- escreve com `os.environ.setdefault`, portanto **uma variavel de ambiente real sempre vence** o arquivo — quem executa o programa dentro de um container ou de um job de CI nao e sobrescrito por um arquivo em disco;
- descarta as chaves que nao sejam alfanumericas mais underscore, de modo que uma linha malformada nao injete nada estranho no ambiente do processo;
- **nunca registra os valores em log**, porque `.env` pode conter chaves de API.

Se o arquivo nao existe, nada acontece; se nao for legivel obtem-se um aviso, nao um erro fatal. Acima deste nivel, quatro chaves de caminho (`PRO_DEMO_PATH`, `DEFAULT_DEMO_PATH`, `BRAIN_DATA_ROOT`, `CUSTOM_STORAGE_PATH`) podem ser reescritas pelo ambiente mesmo quando `user_settings.json` ja contem um valor, e tres segredos (`STEAM_API_KEY`, `FACEIT_API_KEY`, `STORAGE_API_KEY`) vem do chaveiro do sistema. `set_secret()` **retorna False em vez de levantar excecao** quando o chaveiro nao esta disponivel: numa maquina Linux sem backend instalado, salvar as configuracoes nao pode derrubar a aplicacao.

Constantes criticas do sistema:

Constante	Valor	Arquivo	Proposito
<code>METADATA_DIM</code>	25	<code>config.py</code>	Dimensao do feature vector — contrato unificado
<code>SCAN_INTERVAL</code>	10	<code>config.py</code>	Intervalo scan Scanner (segundos)
<code>MAX_DEMOS_PER_MONTH</code>	10	<code>config.py</code>	Quota mensal upload demo
<code>MAX_TOTAL_DEMOS</code>	100	<code>config.py</code>	Limite total de demos por vida
<code>MIN_DEMOS_FOR_COACHING</code>	10	<code>config.py</code>	Limite para coaching personalizado completo
<code>TRADE_WINDOW_S</code>	3.0	<code>trade_kill_detector.py</code>	Janela temporal trade kill em segundos (ciente do tick-rate: 192 ticks a 64/s, 384 a 128/s)
<code>BASLINE_KPR</code>	0.679	<code>feature_engineering/rating.py</code>	Baseline HLTV 2.0 para KPR
<code>BASLINE_DPR_COMPLEMENT</code>	0.317	<code>feature_engineering/rating.py</code>	Baseline HLTV 2.0 para survival
<code>FOV_DEGREES</code>	90	<code>core/constants.py</code>	Campo de visao simulado do jogador
<code>MEMORY_DECAY_TAU_S</code>	2.5s	<code>core/constants.py</code>	Meia-vida da memoria de inimigos (x tick_rate)
<code>CONFIDENCE_ROUNDS_CEILING</code>	300	<code>correction_engine.py</code>	Teto de rounds para confianca maxima
<code>SILENCE_THRESHOLD</code>	0.2	<code>explainability.py</code>	Limite abaixo do qual o silencio e acao valida
<code>MIN_SAMPLES_FOR_VALIDITY</code>	30	<code>role_thresholds.py</code>	Amostras minimas para limite papel valido
<code>HALF_LIFE_DAYS</code>	90	<code>pro_baseline.py</code>	Decaimento temporal dados pro
<code>Z_LEVEL_THRESHOLD</code>	200	<code>connect_map_context.py</code>	Limite Z para classificacao de andar

Arquitetura de paths:

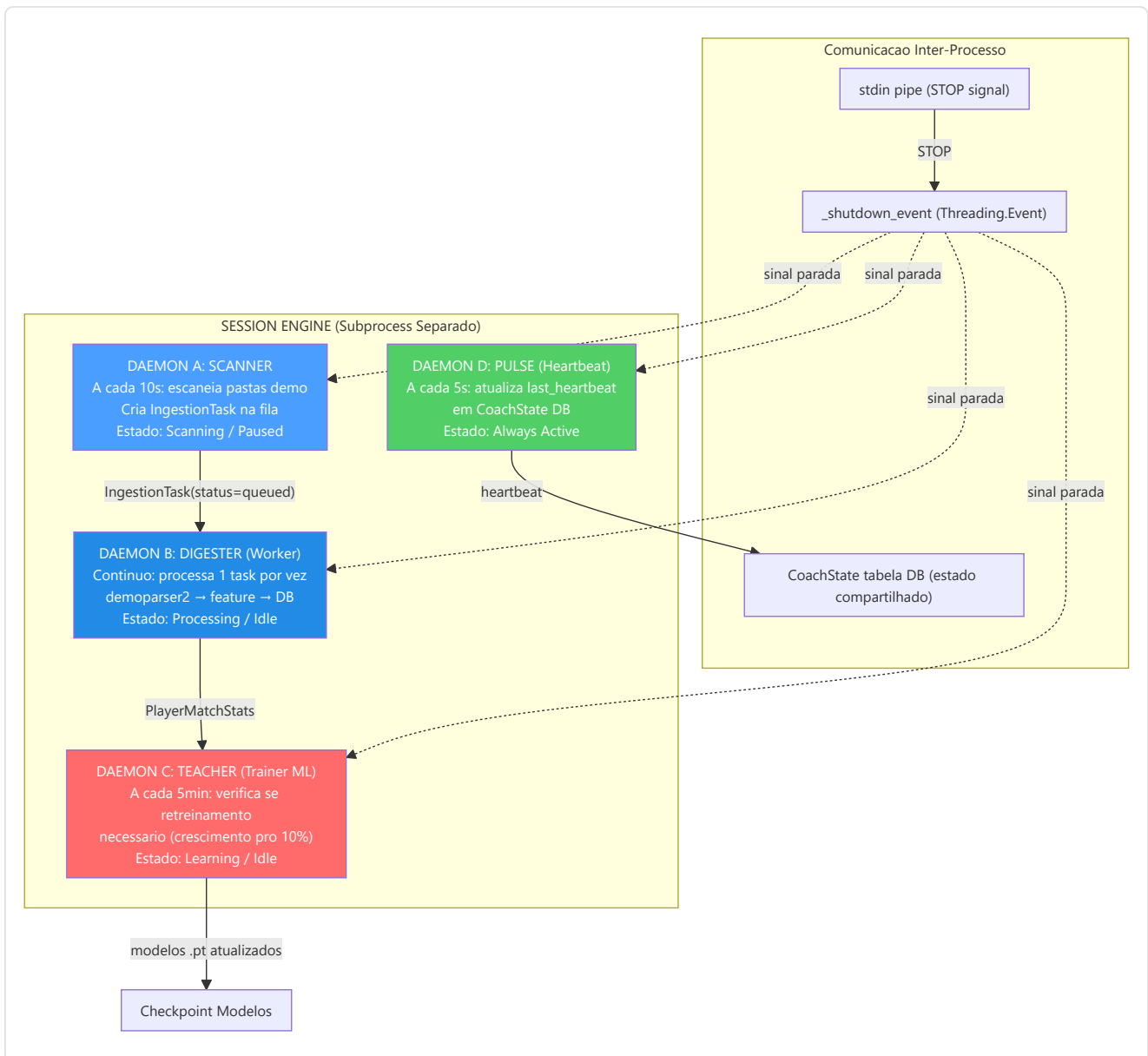
O sistema gerencia paths com uma atencao especial a **portabilidade Windows/Linux**. O coracao e `BRAIN_DATA_ROOT` : um diretorio configuravel pelo usuario que contem modelos, logs e dados derivados. Se nao existe, o sistema recai na pasta do projeto.

Path	Conteudo	Configuravel
<code>DATABASE_URL</code>	Banco monolite principal (<code>database.db</code>)	Nao — sempre na pasta do projeto
<code>BRAIN_DATA_ROOT</code>	Raiz para dados derivados (modelos, logs)	Sim — via <code>user_settings.json</code>
<code>MODELS_DIR</code>	Checkpoints dos modelos <code>.pt</code>	Derivado de <code>BRAIN_DATA_ROOT</code>
<code>LOG_DIR</code>	Arquivos de log da aplicacao	Derivado de <code>BRAIN_DATA_ROOT</code>
<code>MATCH_DATA_PATH</code>	Bancos por-match (<code>match_XXXX.db</code>)	Derivado de <code>BRAIN_DATA_ROOT</code>
<code>DEFAULT_DEMO_PATH</code>	Pasta demo do usuario	Sim — via UI Settings
<code>PRO_DEMO_PATH</code>	Pasta demos profissionais	Sim — via UI Settings

12.4 Motor de Sessao — Arquitetura Quad-Daemon (`session_engine.py`)

Arquivo: `Programma_CS2_RENAN/core/session_engine.py`

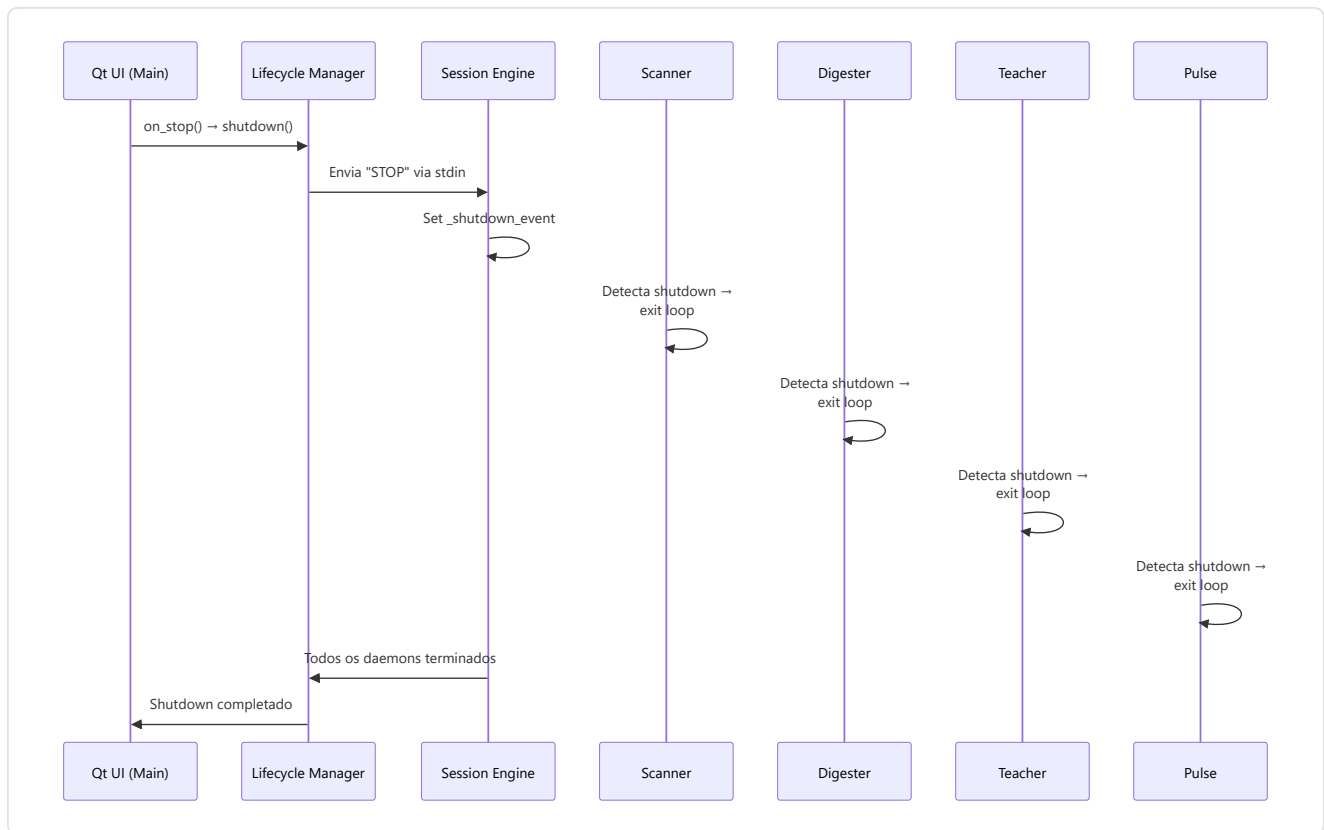
O Session Engine e o **coracao pulsante** da automacao do sistema. Vive como subprocess separado e hospeda **4 daemon threads** que trabalham em paralelo, cada um com uma responsabilidade bem definida. Esse design separa completamente o trabalho pesado (parsing demo, treinamento ML) da interface do usuario, garantindo que a GUI Qt permaneça sempre responsiva.



Ciclo de vida de cada daemon:

Daemon	Intervalo	Trabalho por ciclo	Trigger
Scanner	10 segundos	Escaneia pastas pro e usuario, cria <code>IngestionTask</code> para novos <code>.dem</code>	Sempre ativo (se estado = Scanning)
Digester	Contínuo	Pega 1 task da fila, executa parsing completo	<code>_work_available_event</code> (sinalizado por Scanner)
Teacher	300 segundos (5 min)	Verifica crescimento sample pro; se $\geq 10\%$ → <code>run_full_cycle()</code>	<code>pro_count ≥ last_count x 1.10</code>
Pulse	5 segundos	Atualiza <code>CoachState.last_heartbeat</code> no banco	Sempre ativo

Sequencia de shutdown:



Detalhes de implementacao dos daemons:

Daemon	Metodo Principal	Loop Interno	Condicao de Saida
Scanner	<code>_scanner_daemon_loop()</code>	<code>while not _shutdown_event.is_set() → scan_all_paths() → sleep(10)</code>	<code>_shutdown_event</code> set
Digester	<code>_digester_daemon_loop()</code>	<code>while not _shutdown_event.is_set() → _work_available_event.wait(2) → process_next_task()</code>	<code>_shutdown_event</code> set
Teacher	<code>_teacher_daemon_loop()</code>	<code>while not _shutdown_event.is_set() → _check_retrain_needed() → sleep(300)</code>	<code>_shutdown_event</code> set
Pulse	<code>_pulse_daemon_loop()</code>	<code>while not _shutdown_event.is_set() → state_mgr.heartbeat() → sleep(5)</code>	<code>_shutdown_event</code> set

Gestao de erros nos daemons:

Cada daemon e protegido por um `try/except` global. Se um daemon crasha:

1. O erro e logado com traceback completo
2. O `StateManager` registra o erro (`set_error(daemon, message)`)
3. Uma `ServiceNotification` e criada para o usuario
4. Um **watchdog** do Session Engine verifica a cada 30 segundos as threads daemon e **reinicia automaticamente** as que morreram
5. Os outros daemons continuam a funcionar independentemente

Zombie Task Cleanup: Na inicializacao, o Session Engine procura tasks com `status="processing"` restantes de um crash anterior e os reseta para `status="queued"` , permitindo o restauro automatico sem perda de dados.

Backup Automatico: Na inicializacao do Session Engine, `BackupManager.should_run_auto_backup()` verifica se um backup e necessario e, em caso afirmativo, cria um checkpoint com label `"startup_auto"` . O backup segue uma rotacao de 7 copias diarias + 4 semanais.

12.5 Interface Desktop

A interface desktop e construida com **Qt/PySide6** seguindo o pattern **MVVM** (Model-View-ViewModel).

12.5.1 Interface Qt — apps/qt_app/

Diretorio: `Programma_CS2_RENAN/apps/qt_app/` **Arquivos-chave:** `app.py`, `main_window.py`, `core/i18n_bridge.py`, `core/theme_engine.py`, `screens/`

A interface Qt e construida com **PySide6 (Qt 6)** e utiliza um pattern **MVVM com Qt Signals/Slots**. A `MainWindow` (`QMainWindow`) e composta por uma **sidebar de navegacao** e um `QStackedWidget` que hospeda as 15 telas.

Especificacao	Detalhe
Framework	PySide6 (Qt 6 para Python)
Pattern	MVVM com Qt Signals/Slots
Plataformas	Windows, macOS, Linux
Resolucao	Adaptativa, High-DPI nativo
Temas	3: CS2 (laranja), CSGO (azul-cinza), CS1.6 (verde) — um unico template QSS parametrizado por design tokens
i18n	3 linguas: EN, IT, PT — JSON + <code>QtLocalizationManager</code> , 572 chaves por lingua
Graficos	QPainter puro em todo lugar: mapa tatico e todos os widgets graficos

Sistema i18n: O `QtLocalizationManager` (`core/i18n_bridge.py`) carrega arquivos JSON por lingua (`en.json`, `it.json`, `pt.json`) e gerencia a troca de lingua em runtime atraves de sinais Qt. Cada string da UI e resolvida dinamicamente por uma chave de localizacao.

Sistema de temas: desde agosto de 2026 os tres temas nao sao mais tres folhas de estilo. A pasta `apps/qt_app/themes/` contem **um unico arquivo**, `base.qss.template`, e a cor chega por uma pipeline em tres estagios:

- `design/tokens/design-tokens.json` e a fonte de verdade das cores. `tools/gen_design_tokens.py` o compila em `core/design_tokens.py`, um modulo **gerado** — que nao se edita a mao — que expoe tres instancias congeladas de `DesignTokens`: `CS2_TOKENS`, `CSGO_TOKENS`, `CS16_TOKENS`.
- `core/qss_generator.py` executa `Template(base.qss.template).safe_substitute(asdict(tokens))` e mantem em cache uma folha de estilo ja renderizada para cada nome de tema.
- `core/theme_engine.py` aplica aquele QSS e depois constroi a `QPalette` **a partir da mesma instancia de tokens**, de modo que folha de estilo e palette nao possam mais divergir.

O ganho pratico e que acrescentar um tema significa acrescentar um bloco de tokens, nao reescrever uma folha de estilo de centenas de linhas; e uma cor corrigida num ponto se propaga junto para QSS, `QPalette` e `rating_color()`.

Tema	Acento	Superficie base	Inspiracao
CS2 (default)	#FF6A00 laranja	#0B1628 azul noite	UI moderna de Counter-Strike 2
CSGO	#617D8C azul-cinza	#1A1C21 cinza quase preto	Counter-Strike: Global Offensive
CS1.6	#4DB04F verde	#121A12 verde-preto	Counter-Strike 1.6 classico

O relay de tema. Um `ThemeEngine` vive o tempo de uma inicializacao da aplicacao. Os widgets que se recoloriam sozinhos nao tinham portanto um objeto estavel ao qual se inscrever e, apos uma troca de tema, ficavam com a cor antiga. A solucao e um **relay em nivel de modulo**, `get_theme_relay()`: os widgets com estilo aplicado por instancia — chips de filtro e de status, cards do roster, banners — se inscrevem no relay, que sobrevive aos motores individuais, e o Qt desconecta sozinho os destinatarios destruidos.

Web Views (`apps/qt_app/web/`):

A interface Qt inclui **3 web apps React** (React 18 + TypeScript + Vite, workspace pnpm) renderizadas em um `QWebEngineView` :

Web View	Diretorio	Descricao
Coach Chat	<code>web/coach-chat/</code>	Interface de chat com o coach de IA — formatacao rich text
Match Detail	<code>web/match-detail/</code>	Visualizacao detalhada de uma partida com timeline e estatisticas por round
Tactical Viewer	<code>web/tactical-viewer/</code>	Viewer tatico em camadas (<code>MapCanvas</code> , <code>PlayerLayer</code> , <code>GhostLayer</code> , <code>HeatmapLayer</code> , <code>TrailsLayer</code> , <code>RoundTimeline</code> , <code>ControlBar</code>)

A comunicacao bidirecional entre o backend Python e as web apps passa pelo `QWebChannel` (`web/shared/qwebchannel.ts` + um `bridge.ts` por app), expondo metodos Python como APIs JavaScript. Esta arquitetura hibrida permite usar bibliotecas de visualizacao web mantendo a logica de negocio no backend Python.

15 telas da interface:

Tela	Papel	Componentes chave
Wizard	Primeira configuracao	Nome do jogador, papel, caminhos das pastas de demo
Home	Dashboard	Quota mensal (X/10), status dos servicos (verde/vermelho), confianca das crenças (0-1), tarefas ativas, contador de partidas processadas
Coach	Insights de coaching	Cards coloridos por severidade, radar de skills multidimensional, tendencias historicas, chat de IA (Ollama/Claude), tarefas ativas
Tactical Viewer	Reproducao tatica	Mapa 2D com jogadores/granadas/fantasma, timeline com marcadores de eventos, sidebar de jogadores CT/T, controles de velocidade (0.25x→8x)
Settings	Personalizacao	Tema (CS2/CSGO/CS1.6), fonte, tamanho do texto, lingua, caminhos das demos, wallpaper
Help	Suporte ao usuario	Tutorial interativo, FAQ, troubleshooting
Match History	Historico de partidas	Lista das demos analisadas com filtros e ordenacao
Match Detail	Detalhe da partida	Estatisticas detalhadas de uma unica demo analisada
Performance	Progresso	Radar de skills de 5 eixos, graficos de tendencia, comparacoes temporais
User Profile	Perfil do usuario	Bio, papel preferido, sincronizacao Steam/FACEIT
Profile	Perfil publico	Visualizacao do perfil publico do jogador
Steam Config	Configuracao Steam	Insercao e validacao da API key da Steam
Pro Comparison	Comparacao com pros	Comparacao das estatisticas do usuario com jogadores pro da HLTV, benchmark de performance
Pro Player Detail	Detalhe do jogador pro	Perfil individual do jogador pro da HLTV com estatisticas completas de carreira
FACEIT Config	Configuracao FACEIT	Insercao e validacao da API key da FACEIT

Widgets personalizados:

Widget	Arquivo	Funcao
<code>PlayerSidebar</code>	<code>tactical/player_sidebar.py</code>	Lista CT/T com icones de papel, vida/colete, arma atual, dinheiro e status vivo/morto (<code>_PlayerItem</code> , <code>_StatBar</code>)
<code>TacticalMapWidget</code>	<code>tactical/map_widget.py</code>	Canvas 2D com renderizacao multicamada: textura do mapa → zonas → heatmap → jogadores → granadas → fantasma (QPainter)
<code>TimelineWidget</code>	<code>tactical/timeline_widget.py</code>	Scrubber horizontal com numeros de tick, glifos de evento (estrela, losango), drag-to-see e salto com duplo clique
<code>with_alpha</code>	<code>tactical/_paint_utils.py</code>	Helper de desenho compartilhado: aplica um canal alfa a um <code>QColor</code> sem duplicar a logica em cada widget

Widgets graficos (`widgets/charts/`):

O QtCharts foi **retirado** por uma razao de licenca, nao de gosto: e distribuido sob GPLv3, incompativel com a distribuicao deste projeto. Os graficos foram reescritos em QPainter puro preservando a API publica, de modo que as telas que os usavam nao precisaram mudar. Um teste de gate (`tests/test_charts.py` , classe `TestQtChartsRetired`) falha se uma unica referencia a `QtCharts` ou `QChart` voltar ao codigo.

Todos leem os tokens dentro de `paintEvent` , portanto se redesenham na cor certa apos uma troca de tema.

Widget	Arquivo	Funcao
<code>RadarChart</code>	<code>radar_chart.py</code>	Radar de habilidades de N eixos ($N \geq 3$), grade de poligonos concentricos, uma figura preenchida por serie — usado para sobrepor usuario e profissional
<code>RatingSparkline</code>	<code>rating_sparkline.py</code>	Tendencia do rating com linhas de referencia HLTV tracejadas em 0,90 / 1,00 / 1,10 e legendas na borda direita
<code>UtilityBarChart</code>	<code>utility_bar_chart.py</code>	Barras agrupadas usuario-vs-pro (<code>set_rows</code>) ou uma barra por linha com escala de marcacoes (<code>set_single</code>)
<code>EconomyChart</code>	<code>economy_chart.py</code>	Valor de equipamento por round, barras coloridas pelo lado daquele round; <code>set_half_marker()</code> traca a troca de metade
<code>MomentumChart</code>	<code>momentum_chart.py</code>	Barras do saldo kills-deaths por round, normalizadas pelo maior saldo
<code>MiniSparkline</code>	<code>mini_sparkline.py</code>	Forma de tendencia minima, sem eixos nem moldura, pequena o bastante para caber dentro de um card

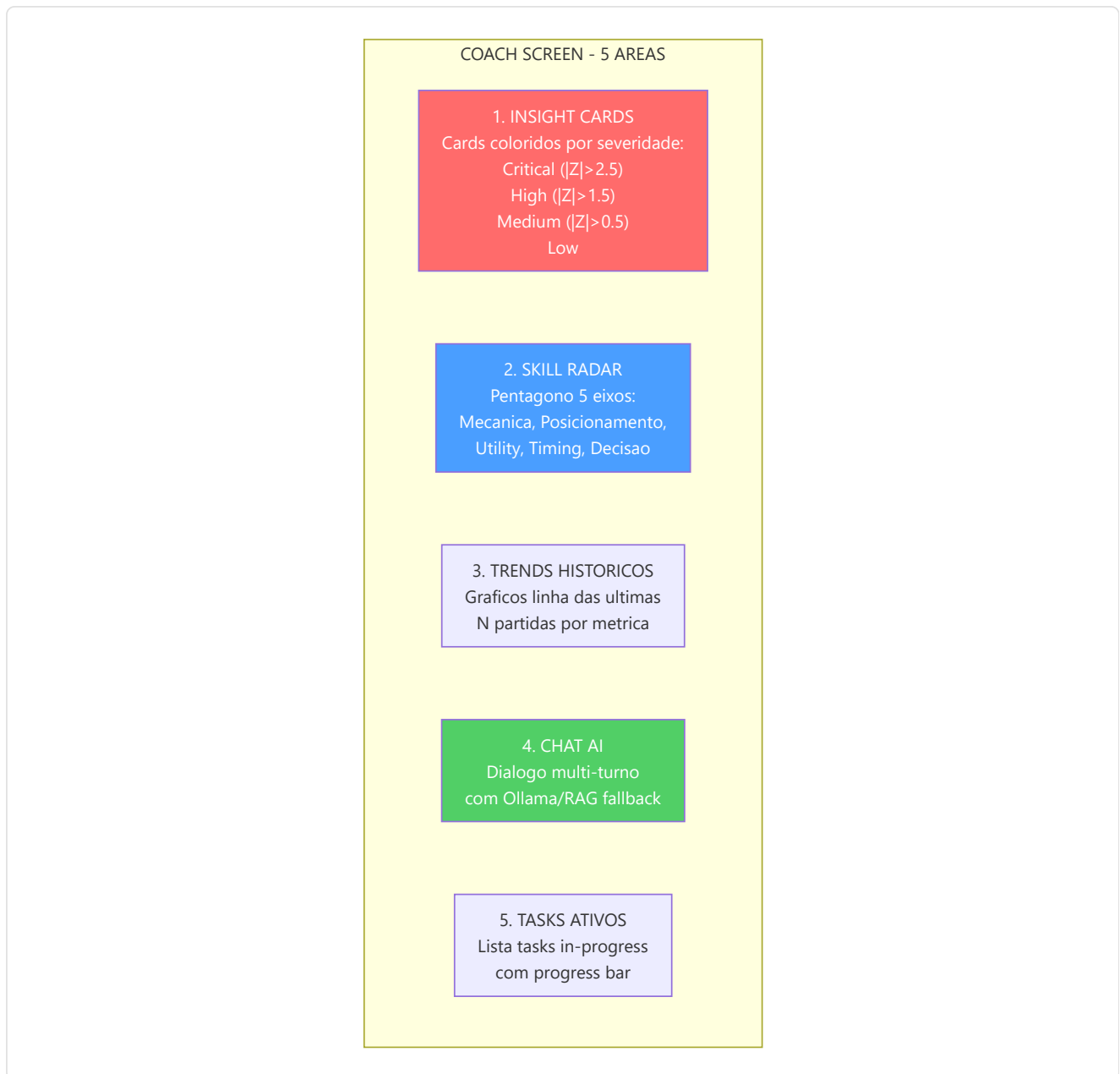
Biblioteca de componentes (`widgets/components/`):

26 componentes reutilizaveis construidos sobre o atlas de design. Os mais caracteristicos: `MapTile` (bloco por mapa com barra de progresso e rotulo de acessibilidade), `DeltaChip` (diferenca em relacao a uma referencia — *a comparacao e a informacao*), `ProBadge` , `MetricBarRow` , `DbRecordCard` , `TipBox` , `NumberedStep` , `DriversList` , `MonoFooter` (linha de proveniencia do dado: de qual tabela e de qual coluna vem o numero mostrado).

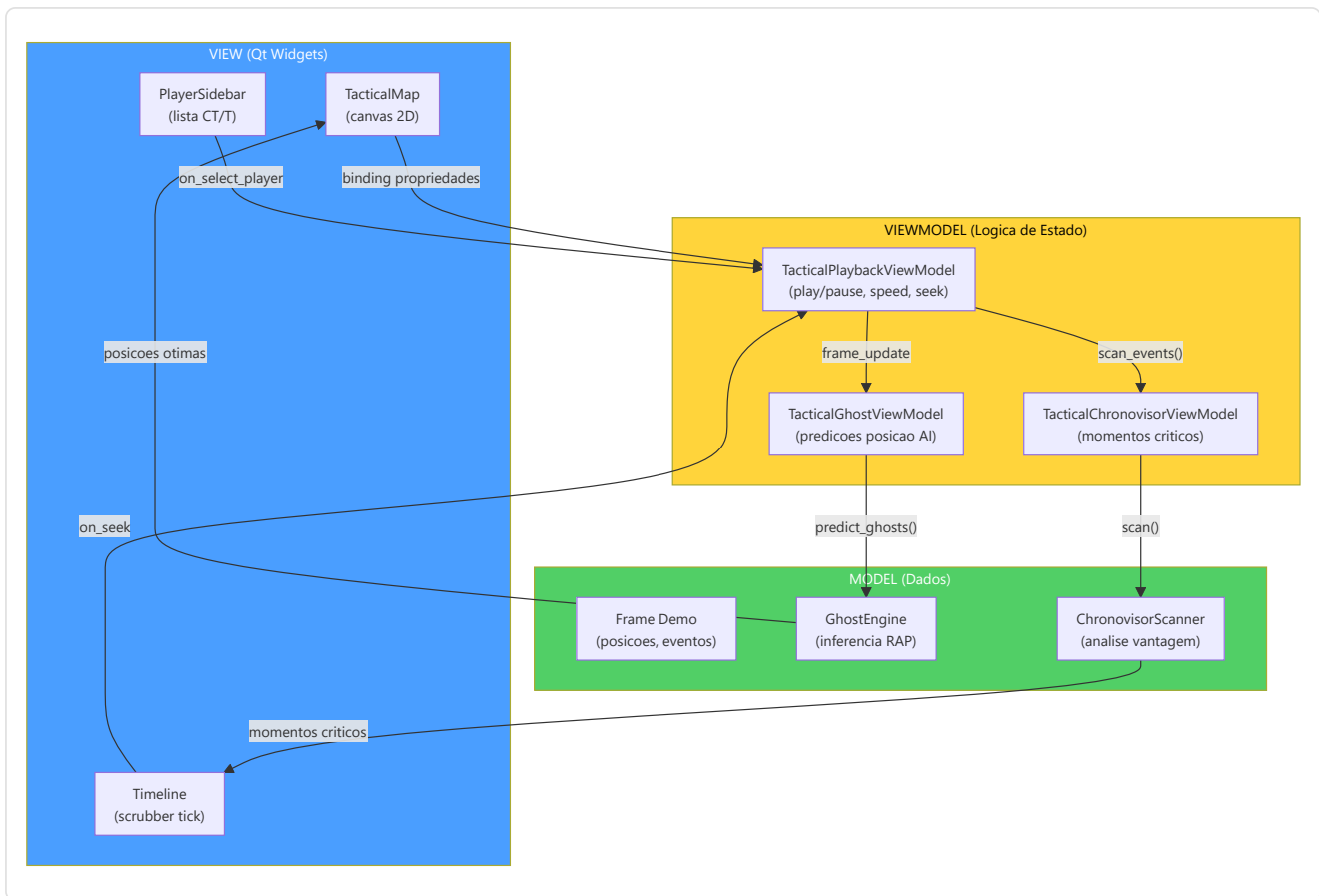
O painel de chat do coach (`widgets/coaching/chat_panel.py`) e deliberadamente **agnostico em relacao ao ViewModel**: nunca fala diretamente com um VM, e uma tela que o conecta (`panel.message_submitted` → `vm.send_message` , `vm.messages_changed` → `panel.add_message`). O mesmo painel pode portanto servir contextos diferentes sem saber nada sobre o que o alimenta.

Coach Screen — Layout detalhado:

A tela Coach e a mais complexa da aplicacao, com 5 areas funcionais:



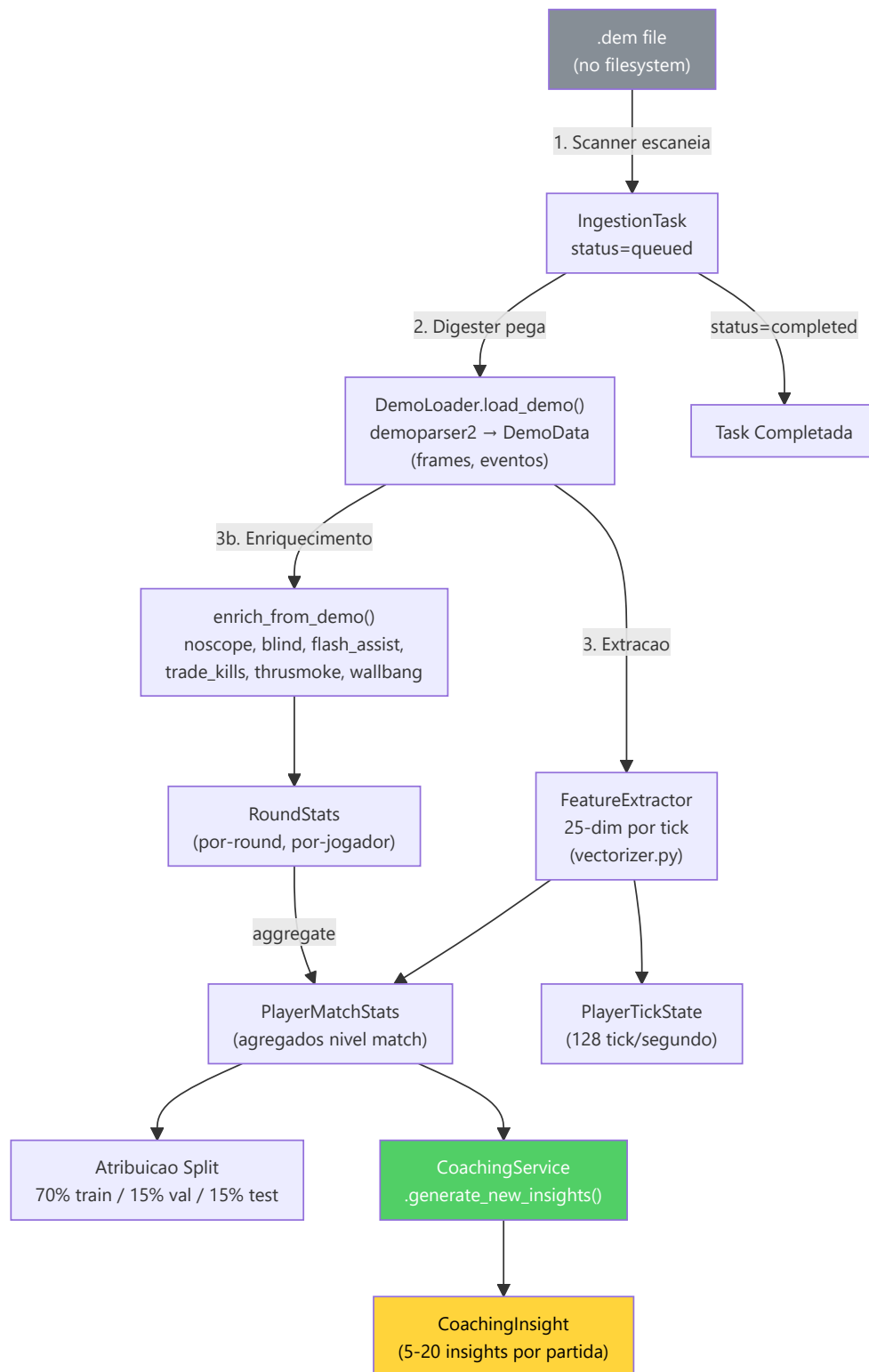
Pattern MVVM no Tactical Viewer:



12.6 Pipeline de Ingestao (`ingestion/`)

Diretorio: `Programma_CS2_RENAN/ingestion/` **Arquivos-chave:** `demo_loader.py`, `steam_locator.py`, `integrity.py`, `registry/`, `pipelines/user_ingest.py`, `pipelines/json_tournament_ingestor.py`

A pipeline de ingestao e o **caminho completo** que um arquivo `.dem` percorre do filesystem ate se tornar insight de coaching no banco. E orquestrada pelo daemon Scanner (descoberta) e pelo daemon Digester (processamento).



Quem assume a tarefa, e por que apenas um.

O passo 2 do diagrama — "o *Digester* retira" — esconde um problema de concorrência que custou dados duplicados. As superfícies que podem iniciar uma ingestão são **seis**: a tela Home, as configurações, o comando de ingest da console, **batch_ingest**, **ingest_pro_demos** e **run_worker**. O caminho antigo fazia um **SELECT** da fila e depois escrevia **status='processing'** sem condições: dois runners iniciados com pouca diferença liam a **mesma fotografia** da fila, ambos a consideravam sua, e a mesma demo era analisada duas vezes, escrevendo estatísticas duplicadas.

A correcao e uma unica instrucao SQL, e a sua forca esta na clausula `WHERE` :

```
_sa_update(IngestionTask)
    .where(IngestionTask.id == task_id, IngestionTask.status == "queued")
    .values(status="processing", updated_at=...)
# rowcount == 1 → a tarefa e nossa; rowcount == 0 → outro a pegou
```

O `UPDATE ... WHERE status='queued'` e atomico em nivel de banco de dados: **exatamente um runner vence cada tarefa**, os outros leem `rowcount == 0` e seguem em frente em silencio. Nao e preciso um lock de aplicacao, nao e preciso coordenacao entre processos — a condicao de corrida e eliminada em vez de gerenciada, o que e sempre a solucao preferivel. Um teste dedicado (`test_ingestion_atomic_claim.py`) verifica tanto o claim exclusivo quanto o fato de que uma tarefa ja reclamada seja pulada.

Fechando o circulo esta o `run_worker.py` : se uma tarefa e reclamada mas depois descartada, `_release_claim()` a devolve para `queued` em vez de deixa-la travada em `processing` para sempre.

DemoLoader — O parser do coracao da pipeline:

O `DemoLoader` e o wrapper em torno do **demoparser2** (biblioteca Rust de alto desempenho) que transforma um arquivo `.dem` binario em estruturas de dados Python:

Fase de Parsing	Output	Tamanho tipico
1. Header parsing	Metadata (map, server, duration)	~100 bytes
2. Frame extraction	Lista de frames (posicao, saude, arma por cada tick)	~100.000 frames/partida
3. Event extraction	Lista de eventos (kill, death, bomb_plant, round_start, etc.)	~500-2.000 eventos/partida
4. Player summary	Estatisticas agregadas por jogador	~10 registros

FeatureExtractor (`backend/processing/feature_engineering/vectorizer.py`):

O `FeatureExtractor` e o componente que transforma os dados brutos do demo em **vetores numericos de 25 dimensoes** (`METADATA_DIM=25`) utilizaveis pelas redes neurais. **Importante:** estas sao features **a nivel de tick** (128 Hz), nao estatisticas agregadas a nivel de partida. Cada frame individual do jogo produz um vetor 25-dim que captura o estado instantaneo do jogador:

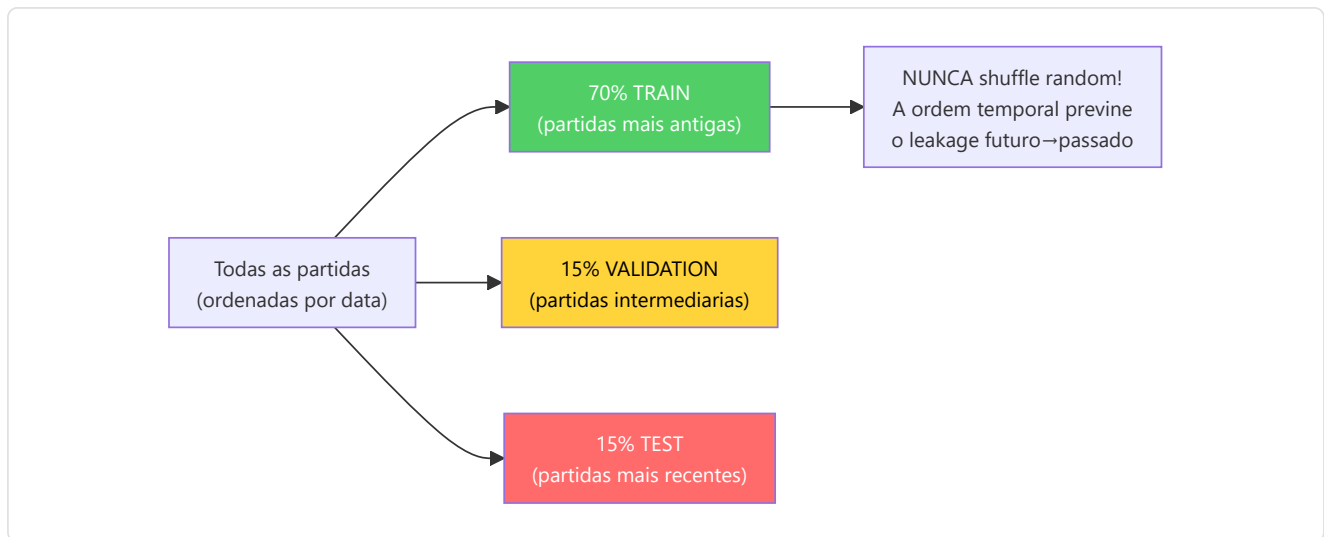
Dim	Feature	Tipo	Range	Descricao
0	health	Float	[0, 1]	Saude normalizada
1	armor	Float	[0, 1]	Armadura normalizada
2	has_helmet	Binary	0/1	Capacete equipado
3	has_defuser	Binary	0/1	Kit defuse equipado
4	equipment_value	Float	[0, 1]	Valor equipamento normalizado
5	is_crouching	Binary	0/1	Agachado
6	is_scoped	Binary	0/1	Scope ativo
7	is_blinded	Binary	0/1	Cego por flash
8	enemies_visible	Float	[0, 1]	Inimigos visiveis (normalizado, clamped)
9	pos_x	Float	[-1, 1]	Posicao X (normalizada +/-pos_xy_extent)
10	pos_y	Float	[-1, 1]	Posicao Y (normalizada +/-pos_xy_extent)
11	pos_z	Float	[0, 1]	Posicao Z (normalizada, trata Nuke/Vertigo)
12	view_x_sin	Float	[-1, 1]	sin(yaw) — continuidade ciclica angulo horizontal
13	view_x_cos	Float	[-1, 1]	cos(yaw) — continuidade ciclica angulo horizontal
14	view_y	Float	[-1, 1]	Pitch normalizado (angulo vertical)
15	z_penalty	Float	[0, 1]	Distincao nivel vertical (penalidade andar)
16	kast_estimate	Float	[0, 1]	Estimativa KAST (taxa participacao)
17	map_id	Float	[0, 1]	Hash deterministico do mapa
18	round_phase	Float	{0, 0.33, 0.66, 1}	Fase economica: pistol/eco/force/full_buy
19	weapon_class	Float	{0-1.0}	Classe arma: 0=faca, 0.2=pistola, 0.4=SMG, 0.6=rifle, 0.8=sniper, 1.0=pesada
20	time_in_round	Float	[0, 1]	Segundos no round / 115 (clamped)
21	bomb_planted	Binary	0/1	Bomba plantada
22	teammates_alive	Float	[0, 1]	Companheiros vivos (count / 4)
23	enemies_alive	Float	[0, 1]	Inimigos vivos (count / 5)
24	team_economy	Float	[0, 1]	Media dinheiro time / 16000 (clamped)

Normalizacao e bounds:

A normalizacao e integrada diretamente no FeatureExtractor, com bounds configuraveis via `HeuristicConfig` (externalizada em JSON). A codificacao ciclica dos angulos de vista (sin/cos para o yaw) previne descontinuidades nas bordas 0/360. O `z_penalty` distingue automaticamente os andares em mapas multi-nivel (Nuke, Vertigo). A classe arma utiliza um mapeamento categorico ordinal (6 classes) definido na constante `WEAPON_CLASS_MAP` (inclui tambem granadas=0.1 e equipamento especial=0.05).

Dataset Split Temporal:

A subdivisao do dataset segue uma **ordenacao cronologica** rigorosa para prevenir o data leakage temporal:



Enrich From Demo — Enriquecimento pos-parsing:

Apos o parsing base, `enrich_from_demo()` adiciona metricas avancadas calculadas dos eventos:

Metrica enriquecida	Calculo	Fonte
Trade kills	<code>TradeKillDetector.detect()</code> com janela de 3s ciente do tick-rate	Eventos kill/death
Flash assists	Contagem blind dentro de janela temporal antes de um kill	Eventos blind + kill
Noscope kills	Kill com arma sniper sem scope ativo	Evento kill + weapon state
Wallbang kills	Kill atraves de superficies penetraveis	Evento kill com flag penetration
Through-smoke kills	Kill com fumo ativo na linha de tiro	Evento kill + smoke position
Blind kills	Kill enquanto o jogador esta flashado	Evento kill + flash state

Componentes especificos:

Componente	Arquivo	Papel
DemoLoader	<code>demo_loader.py</code>	Wrappa <code>demoparser2</code> , extrai frames e eventos do arquivo <code>.dem</code>
SteamLocator	<code>steam_locator.py</code>	Localiza automaticamente a pasta demo do CS2 via registro Steam / <code>libraryfolders.vdf</code>
IntegrityChecker	<code>integrity.py</code>	Verifica que os arquivos demo sejam validos, completos e nao corrompidos antes do parsing
UserIngestPipeline	<code>pipelines/user_ingest.py</code>	Pipeline completa para demo usuario: parse → enrich → stats → coaching
JsonTournamentIngestor	<code>pipelines/json_tournament_ingestor.py</code>	Importa dados de torneio de arquivos JSON estruturados
Registry	<code>registry/registry.py</code>	<code>DemoRegistry</code> : rastreia todas as demos processadas, previne duplicatas
ResourceManager	<code>backend/ingestion/resource_manager.py</code>	Gestao de recursos hardware: CPU/RAM throttling, espaco disco
RegistryLifecycle	<code>registry/lifecycle.py</code>	<code>DemoLifecycleManager</code> : limpeza de demos antigas (default 30 dias)

SteamLocator (`ingestion/steam_locator.py`) — localizacao automatica demo CS2:

O SteamLocator implementa um algoritmo de **discovery cross-platform** para encontrar automaticamente a pasta das demos do CS2:

Plataforma	Estrategia	Path tipico
Windows	Registro do sistema → <code>libraryfolders.vdf</code>	<code>C:\Program Files (x86)\Steam\steamapps\common\Counter-Strike Global Offensive\game\csgo\replays</code>
Linux	<code>~/.steam/steam/</code> → <code>libraryfolders.vdf</code>	<code>~/.steam/steam/steamapps/common/ ...</code>
Fallback	Pergunta ao usuario via UI Settings	Path personalizado

IntegrityChecker (`ingestion/integrity.py`):

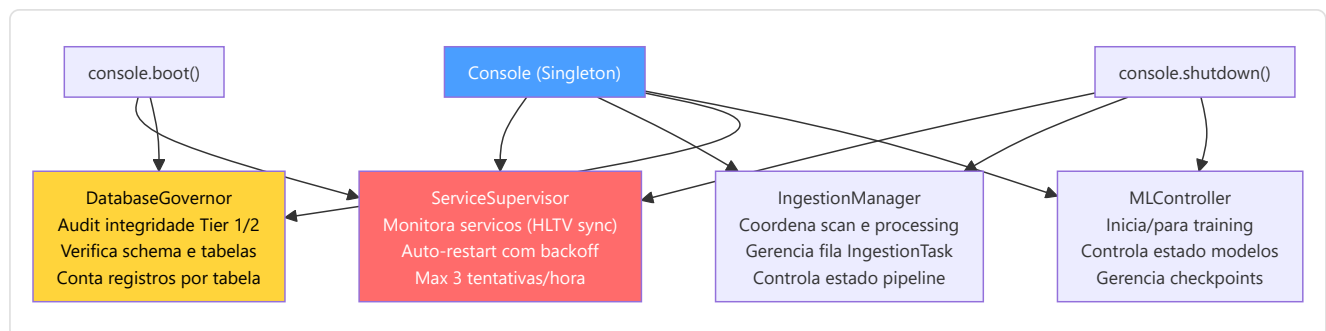
Verificacao preliminar de cada arquivo demo antes do parsing custoso:

- **Magic bytes:** `PBDEMS2\0` (CS2 Source 2) ou `HL2DEMO\0` (legacy Source 1)
- **Size bounds:** minimo 1KB (nao vazio), maximo 5GB (nao corrompido/excessivo)
- **Read test:** tenta ler os primeiros N bytes para verificar que o arquivo seja acessivel

12.7 Console de Controle Unificada (`backend/control/`)

Arquivo: `Programma_CS2_RENAN/backend/control/console.py` , `ingest_manager.py` , `db_governor.py` , `ml_controller.py`

A Console e um **Singleton** que atua como ponto de coordenacao central para todos os subsistemas backend. E o "quadro de comando" atraves do qual cada parte do sistema pode ser controlada.

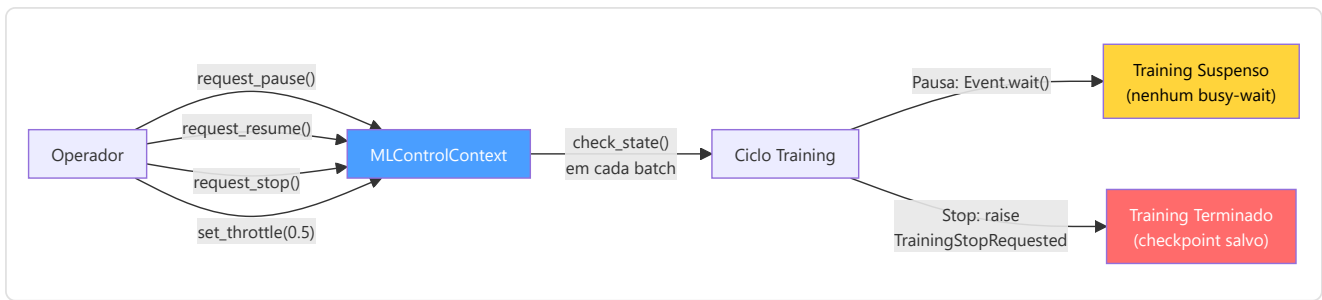


Sequencia de boot da Console:

1. `ServiceSupervisor` inicia o servico "hunter" (HLTV sync) como processo monitorado
2. `DatabaseGovernor` executa um audit de integridade: verifica todas as tabelas, conta registros, controla schema
3. `MLController` permanece em estado de espera — o training e gerenciado pelo daemon Teacher
4. `IngestionManager` permanece idle — o trabalho ativo e gerenciado pelos daemons Scanner/Digester

MLControlContext — Controle Live do Treinamento:

O `MLController` utiliza um token de controle chamado `MLControlContext` (`backend/control/ml_controller.py`) que e passado aos ciclos de treinamento para permitir **intervencao em tempo real** pelo operador. Isso substitui a abordagem anterior baseada em `StopIteration` por um sistema thread-safe mais robusto.

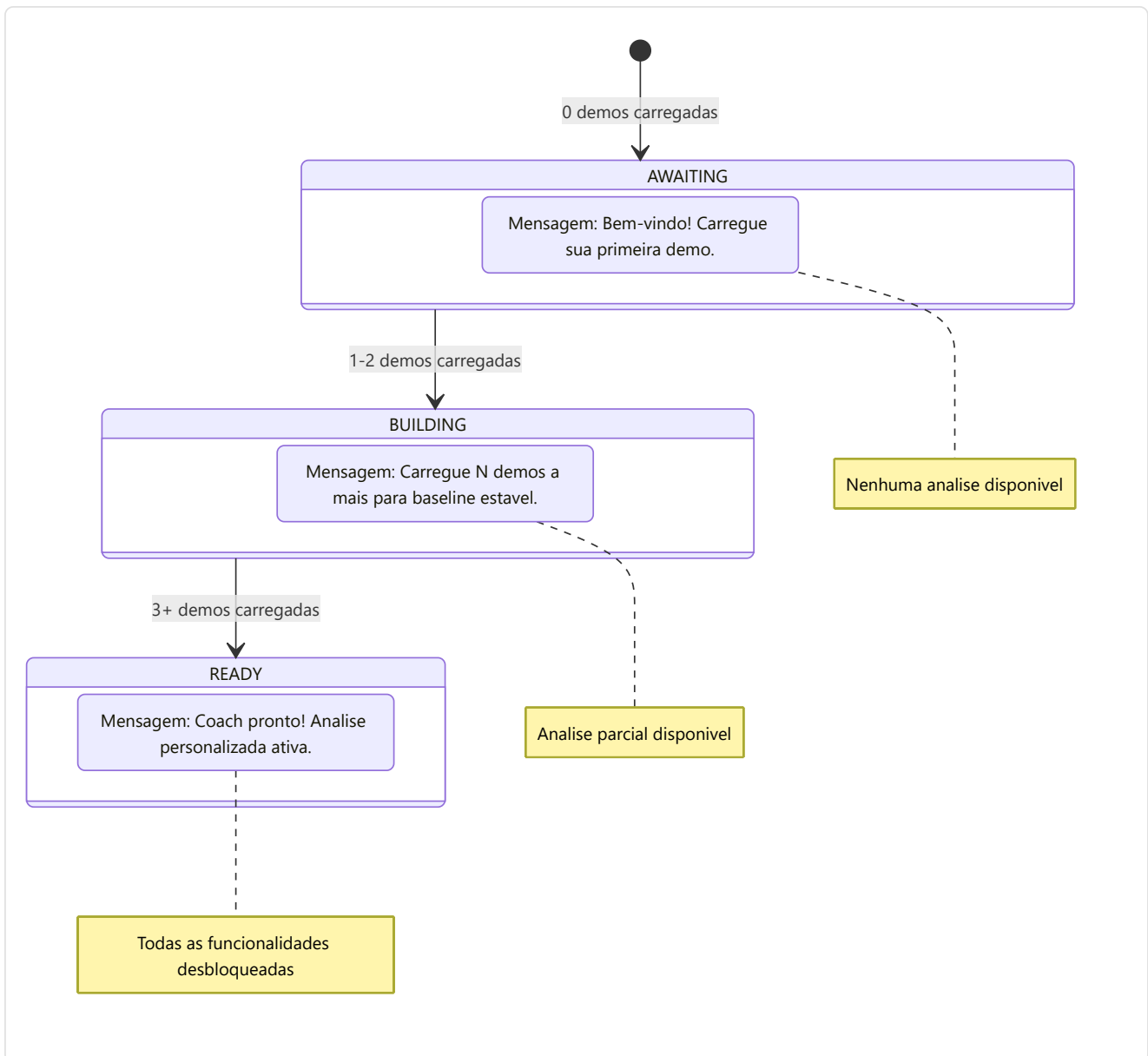


Comando	Metodo	Efeito
Pausa	<code>request_pause()</code>	<code>_resume_event.clear()</code> → bloqueia <code>check_state()</code>
Retomar	<code>request_resume()</code>	<code>_resume_event.set()</code> → desbloqueia o training
Stop	<code>request_stop()</code>	Lança <code>TrainingStopRequested</code> (excecao custom)
Throttle	<code>set_throttle(factor)</code>	Adiciona <code>time.sleep(factor)</code> apos cada batch

12.8 Onboarding e Fluxo de Novo Usuario

Arquivo: `Programma_CS2_RENAN/backend/onboarding/new_user_flow.py`

O `OnboardingManager` guia os novos usuarios atraves de uma **progressao de 3 fases** que se adapta automaticamente a quantidade de dados disponiveis.



Wizard Screen (Primeira Configuracao):

Na primeira execucao (`SETUP_COMPLETED = False`), o usuario e guiado atraves do Wizard:

1. **Nome jogador** — O nome que aparecera nas analises
2. **Papel preferido** — Entry Fragger, AWPPer, Lurker, Support ou IGL
3. **Paths pastas demo** — Onde o sistema procura automaticamente as demos
4. Ao completar: `SETUP_COMPLETED = True` , redirect para a Home

Cache das quotas (Task 2.16.1): A contagem de demos e cacheada por 60 segundos para evitar queries DB repetidas. `invalidate_cache()` e chamado apos cada novo upload, garantindo que a UI sempre mostre a contagem correta sem sobrecarregar o banco.

Help System (`backend/knowledge/help_system.py`):

O sistema de ajuda integrado fornece suporte contextual ao usuario:

Funcionalidade	Implementacao
Tutorial interativo	Guias step-by-step para as funcionalidades principais
FAQ contextuais	Perguntas frequentes filtradas por tela atual
Troubleshooting	Arvore decisoria para problemas comuns (Steam path nao encontrado, demo nao parsed, coaching vazio)
Tooltips	Explicacoes inline para metricas complexas (KAST, HLTV 2.0 Rating, ADR)

12.9 Arquitetura de Storage (`backend/storage/`)

Diretorio: `Programma_CS2_RENAN/backend/storage/` **Arquivos-chave:** `database.py` , `db_models.py` , `match_data_manager.py` , `storage_manager.py` , `maintenance.py` , `state_manager.py` , `stat_aggregator.py` , `backup.py`

O sistema de storage utiliza uma arquitetura **three-tier storage** baseada em SQLite em modo WAL (Write-Ahead Logging), que permite leituras e escritas concorrentes sem bloqueios.





As 21 tabelas SQLModel:

#	Tabela	Banco	Categoria	Descricao
1	PlayerMatchStats	database.db	Core	Estatisticas agregadas por jogador/partida
2	PlayerTickState	database.db	Core	Estado por-tick, tambem arquivado em DBs por-match separados
3	PlayerProfile	database.db	Usuario	Perfil do usuario (nome, papel, Steam ID, quota mensal)
4	RoundStats	database.db	Core	Estatisticas isoladas por round (kills, rating, enriquecimento)
5	CoachingInsight	database.db	Coaching	Conselhos gerados pelo servico de coaching
6	CoachingExperience	database.db	Coaching	Banco de experiencias COPER (contexto, resultado, eficacia, TrueSkill μ/σ)
7	IngestionTask	database.db	Sistema	Fila de trabalho para o daemon Digester
8	CoachState	database.db	Sistema	Estado global (training metrics, heartbeat, status)
9	ServiceNotification	database.db	Sistema	Mensagens de erro/evento dos daemons → UI
10	TacticalKnowledge	database.db	Conhecimento	Base RAG (embedding 384-dim em JSON)
11	ProPlayer	hlvtv_metadata.db	Pro	Perfis dos jogadores profissionais
12	ProTeam	hlvtv_metadata.db	Pro	Metadados dos times profissionais
13	ProPlayerStatCard	hlvtv_metadata.db	Pro	Estatisticas sazonais por jogador pro
14	ProEvent	hlvtv_metadata.db	Pro	Eventos/competicoes HLTV
15	ProTournament	hlvtv_metadata.db	Pro	Torneios HLTV
16	ProHead2Head	hlvtv_metadata.db	Pro	Confrontos diretos entre jogadores
17	ProMapRecord	hlvtv_metadata.db	Pro	Records por mapa
18	Ext_PlayerPlaystyle	database.db	Externo	Dados de estilo de jogo de CSV (para o NeuralRoleHead)
19	Ext_TeamRoundStats	database.db	Externo	Estatisticas de torneio externas
20	MatchResult	database.db	Partidas	Resultados das partidas
21	MapVeto	database.db	Partidas	Historico de selecao de mapas
22	CalibrationSnapshot	database.db	Sistema	Registro de calibracao do modelo de crenças (timestamp, amostras, resultado)
23	RoleThresholdRecord	database.db	Sistema	Limiars aprendidos para a classificacao de papeis (persistidos entre reinicios)
24	DataLineage	database.db	Proveniencia	Registro append-only de proveniencia de dados: entity_type, entity_id, source_demo, pipeline_version
25	DataQualityMetric	database.db	Proveniencia	Metricas de qualidade append-only por run: run_id, run_type, metric_name, metric_value

Enum de suporte (nao tabelas):

Enum	Tipo	Descricao
DatasetSplit	str, Enum	Categorias split (train/val/test/unassigned) — usado como constraint em <code>PlayerMatchStats.dataset_split</code>
CoachStatus	str, Enum	Estados do coach (Paused/Training/Idle/Error) — usado como constraint em <code>CoachState.status</code>

Componentes de Storage detalhados:

MatchDataManager (`backend/storage/match_data_manager.py`) — o componente maior do storage layer:

O MatchDataManager e responsavel pela gestao dos dados por-partida de alta densidade (PlayerTickState com ~100.000 linhas por partida). Para evitar que o banco principal cresca de forma descontrolada, cada partida tem seu proprio banco SQLite separado (`match_XXXX.db`).

Metodo	Descricao
<code>get_match_session(match_id)</code>	Context manager transacional sobre o DB por-match (engine de um cache LRU)
<code>store_tick_batch(match_id, ticks)</code>	Bulk insert de <code>MatchTickState</code> no DB dedicado
<code>store_event_batch(match_id, events)</code>	Bulk insert de <code>MatchEventState</code>
<code>store_metadata(match_id, metadata)</code>	Upsert dos metadados da partida
<code>list_available_matches()</code>	Lista todas as partidas com DB disponivel
<code>delete_match(match_id)</code>	Remove o DB por-match

StorageManager (`backend/storage/storage_manager.py`):

O StorageManager e o **coordenador de alto nivel** do storage que gerencia quotas, uploads e ciclo de vida dos dados:

Responsabilidade	Implementacao
Quota mensal	<code>can_user_upload()</code> → verifica <code>MAX_DEMOS_PER_MONTH=10</code> e <code>MAX_TOTAL_DEMOS=100</code>
Upload flow	<code>handle_demo_upload(path)</code> → validacao → copia em working dir → cria IngestionTask
Limpeza	<code>cleanup_old_data(days)</code> → remove match DB e tasks antigos
Espaco disco	<code>get_storage_usage()</code> → report dimensoes para cada banco e diretorio

StateManager (`backend/storage/state_manager.py`):

O StateManager e um **Singleton** que mantem o estado runtime do sistema e o persiste no banco via a tabela `CoachState` :

Metodo	Proposito
<code>update_status(daemon, text)</code>	Atualiza o estado de um daemon especifico
<code>heartbeat()</code>	Atualiza o timestamp <code>last_heartbeat</code>
<code>get_state()</code>	Retorna o estado atual como objeto <code>CoachState</code>
<code>set_error(daemon, message)</code>	Registra um erro para um daemon com timestamp
<code>update_training_metrics(epoch, loss, val_loss, eta)</code>	Atualiza as metricas de training em tempo real
<code>get_belief_confidence()</code>	Retorna o nivel de confianca do modelo de crenca (0.0-1.0)

StatAggregator (`backend/storage/stat_aggregator.py`):

Calcula estatisticas agregadas a partir dos dados brutos por-round:

Agregacao	Formula	Uso
<code>avg_kills</code>	<code>mean(RoundStats.kills)</code>	Dashboard, radar chart
<code>avg_adr</code>	<code>mean(RoundStats.damage_dealt / rounds)</code>	Comparacao pro
<code>avg_kast</code>	<code>mean(rounds_with_kast / total_rounds)</code>	Metrica HLTV
<code>accuracy</code>	<code>sum(hits) / sum(shots_fired)</code>	Performance mecanica
<code>trade_kill_rate</code>	<code>trade_kills / team_deaths</code>	Trabalho de equipe

BackupManager (`backend/storage/backup.py`):

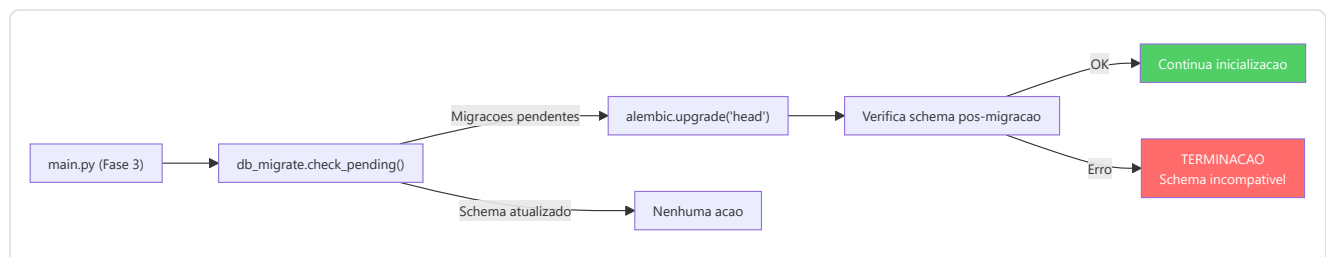
Caracteristica	Detalhe
Rotacao diaria	7 copias — a mais antiga e sobrescrita
Rotacao semanal	4 copias — backup semanal adicional
Trigger automatico	Na inicializacao do Session Engine via <code>should_run_auto_backup()</code>
Trigger manual	Via Console ou UI Settings
Formato	Copia completa do arquivo <code>.db</code> (nao dump SQL)
Etiquetagem	<code>startup_auto</code> , <code>manual</code> , <code>pre_migration</code>

Maintenance (`backend/storage/maintenance.py`):

Operacao	Frequencia	Proposito
<code>vacuum()</code>	Mensal	Compacta o banco, recupera espaco de registros deletados
<code>analyze()</code>	Apos cada bulk insert	Atualiza as estatisticas do otimizador query SQLite
<code>wal_checkpoint()</code>	Na inicializacao	Forca o merge do WAL no banco principal
<code>integrity_check()</code>	Na inicializacao	<code>PRAGMA integrity_check</code> — verifica coerencia estrutural

DbMigrate (`backend/storage/db_migrate.py`):

Wrapper em torno do Alembic que automatiza a execucao das migracoes:



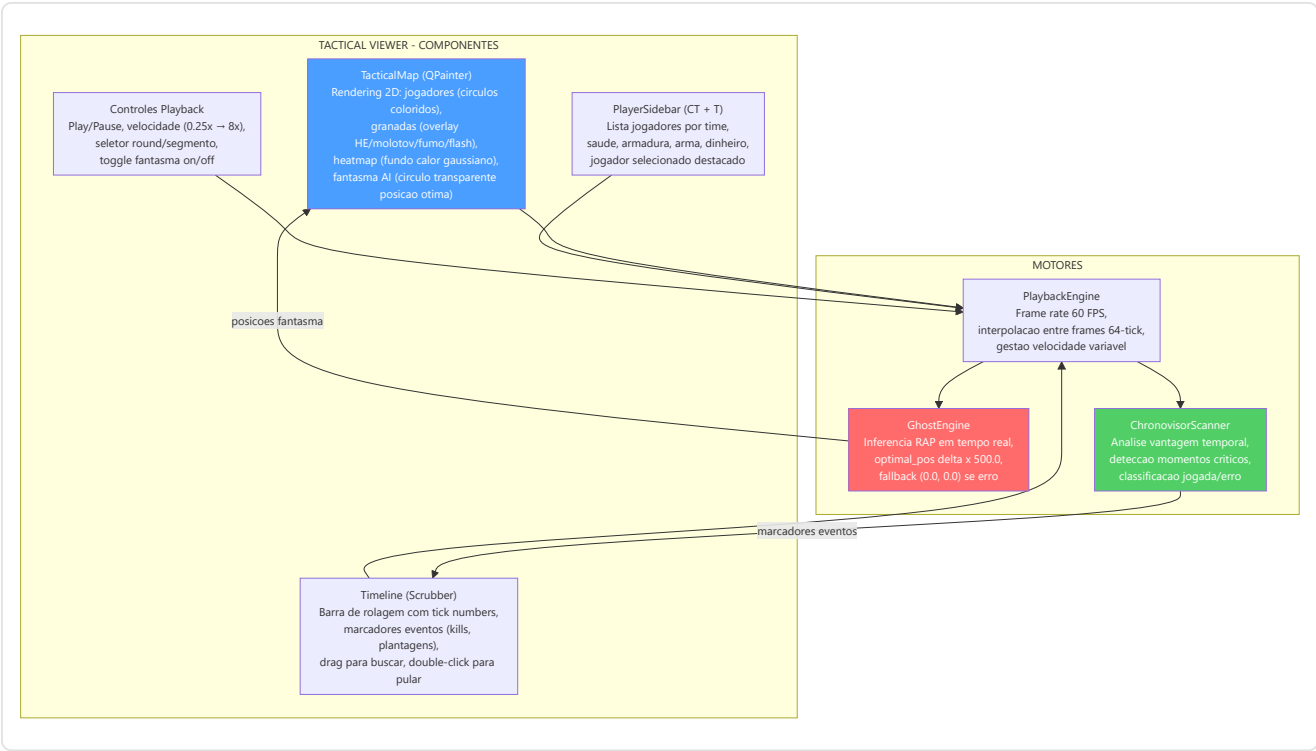
Connection Pooling e Concorrencia:

Parametro	Valor	Proposito
<code>check_same_thread</code>	<code>False</code>	Permite acesso multi-thread
<code>timeout</code>	30 segundos	Busy timeout para contencao WAL
<code>pool_size</code>	1	Unico escritor SQLite (seguranca single-writer)
<code>max_overflow</code>	4	Conexoes overflow para picos de carga
WAL mode	Habilitado	Leituras concorrentes ilimitadas

12.10 Motor de Playback e Viewer Tatico

Arquivos: `Programma_CS2_RENAN/core/playback_engine.py`, `apps/qt_app/screens/tactical_viewer_screen.py`, `apps/qt_app/widgets/tactical/map_widget.py`, `timeline_widget.py`, `player_sidebar.py`

O sistema de playback tatico permite ao usuario **reviver as proprias partidas** em um mapa 2D interativo, com overlay AI (posicao fantasma otima), marcadores de eventos (kills, plantagens de bomba) e controles de reproducao completos.



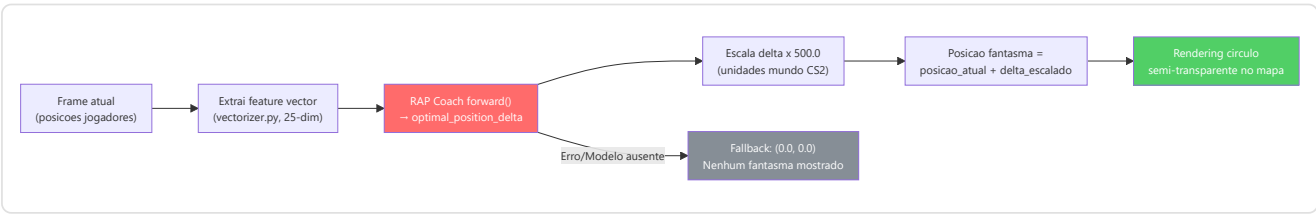
PlaybackEngine — Arquitetura interna:

O PlaybackEngine gerencia a reprodução frame-by-frame com interpolação temporal. O timer de atualização utiliza `QTimer`, mantendo a mesma lógica de interpolação e buffering:

Característica	Detalhe
Frame rate	60 FPS (interpolação de 64-tick nativo do demo)
Velocidade variável	0.25x (slow-mo), 0.5x, 1x (normal), 2x, 4x, 8x (fast-forward)
Interpolação	Linear entre frames adjacentes para movimentos fluidos
Buffering	Pre-carrega 120 frames antecipadamente para evitar lag
Seek	Acesso direto a qualquer tick via índice
Round selection	Pula diretamente ao início de um round específico

GhostEngine — Inferência em tempo real:

O GhostEngine é o componente que transforma as previsões do RAP Coach em **posições fantasma visíveis no mapa**:



ChronovisorScanner — Detecção de momentos críticos:

O ChronovisorScanner analisa a partida inteira e identifica os **momentos decisivos** baseando-se na mudança de vantagem:

Tipo momento	Condicao	Cor marcador
Erro critico	Morte em vantagem numerica (ex. 4v3 → 3v3)	Vermelho
Jogada excelente	Kill em desvantagem numerica (ex. 2v3 → 2v2)	Verde
Plantagem bomba	Evento bomb_planted com timer	Amarelo
Clutch	Vitoria 1vN (N >= 2)	Ouro
Eco round win	Vitoria com equipamento < \$2.000	Azul

Cada momento e posicionado na Timeline como um marcador clicavel. O clique pula o playback diretamente para aquele tick.

Fluxo de carregamento do viewer (One-Click):

1. O usuario clica "Tactical Viewer" da Home
2. Se nenhuma demo esta carregada → `trigger_viewer_picker()` abre o file picker automaticamente
3. O usuario seleciona um arquivo `.dem`
4. Aparece um dialogo "Reconstrucao Dinamica 2D em Andamento..."
5. Uma thread em background executa `_execute_viewer_parse(path)` → `DemoLoader.load_demo()`
6. Ao completar: dismissao do dialogo, carregamento frames no PlaybackEngine
7. O mapa e renderizado com os jogadores no frame 0

12.11 Dados Espaciais e Gestao de Mapas

Arquivos: `Programma_CS2_RENAN/core/spatial_data.py`, `spatial_engine.py`, `data/map_config.json`

O sistema de gestao de mapas traduz as **coordenadas mundo do CS2** (valores tipicos: -2000 a +2000 em X/Y) em **coordenadas pixel** na textura do mapa (0.0 a 1.0 normalizado), e vice-versa.

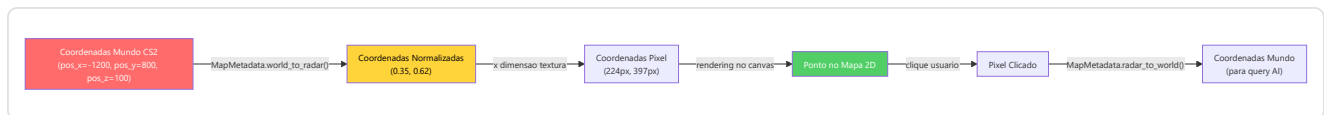
Antes da geometria, a identidade: quais mapas existem. Um censo de agosto de 2026 encontrou **doze** listas de mapas conhecidos declaradas em pontos diferentes do projeto, com conteudos divergentes: uma conhecia onze, outra nove, outra oito. A consequencia nao era teorica — a mesma demo podia ser reconhecida por uma ferramenta e ignorada por outra, dependendo de qual lista aquele modulo carregava consigo.

`core/known_maps.py` e hoje a autoridade unica: `KNOWN_MAP_NAMES` (os nomes nus), `KNOWN_MAP_IDS` (os mesmos com o prefixo `de_`), a expressao regular para reconhecer um mapa dentro de um nome de arquivo, e os tres helpers `bare_name()`, `is_known_map()`, `sniff_map_from_text()`. O conjunto e deliberadamente um **superconjunto** de todas as doze listas encontradas: melhor reconhecer um mapa que nao esta na rotacao competitiva do que ignorar um que esta.

E preciso dizer com precisao ate onde ela chega, porque uma SSOT declarada mais ampla do que e vale menos do que nenhuma SSOT. **Sete consumidores** foram convertidos e sao guardados pelo teste `test_known_maps_ssot.py`, que proibe redeclarar o trio mirage/inferno/nuke e verifica que cada um ainda importe o modulo:

`apps/qt_app/core/match_utils.py` · `apps/qt_app/screens/coach_screen.py` · `tools/d3_recover_shard_metadata.py` · `tools/mine_coaching_experience.py` · `tools/mine_shard_strategies.py` · `tools/populate_match_results.py` · `tools/rebuild_monolith.py`

Ficam de fora quatro listas locais — em `coaching_dialogue.py`, `reporting/analytics.py`, `knowledge/pro_demo_miner.py` e em `Goliath_Hospital.py` — e duas destas sao exclusoes **deliberadas**: `REQUIRED_MAPS` do Goliath verifica a presenca de arquivos de asset, nao a identidade de um mapa, e o registro espacial carrega geometrias de radar calibradas a mao que nao fazem sentido fora do seu contexto. As outras duas sao debito residual, nao projeto.



MapMetadata (dataclass imutavel para cada mapa):

Campo	Exemplo (Dust2)	Proposito
<code>pos_x</code>	-2476	X do canto superior esquerdo em unidades mundo
<code>pos_y</code>	3239	Y do canto superior esquerdo em unidades mundo
<code>scale</code>	4.4	Unidades de jogo por pixel da textura
<code>z_cutoff</code>	None	Separador de nivel (apenas mapas multi-nivel)
<code>level</code>	"single"	Tipo: "single", "upper", "lower"
<code>texture_width</code>	1024	Largura textura radar em pixel
<code>texture_height</code>	1024	Altura textura radar em pixel

MapManager (`core/map_manager.py`):

O MapManager e o componente de alto nivel que coordena o carregamento dos mapas para a interface:

Metodo	Proposito
<code>get_map_metadata(map_name)</code>	Retorna MapMetadata para o mapa requisitado
<code>load_radar_texture(map_name)</code>	Carrega textura PNG do mapa radar do cache ou disco
<code>get_available_maps()</code>	Lista dos mapas suportados com metadata
<code>world_to_pixel(pos, map_name)</code>	Shortcut para conversao coordenadas mundo → pixel

Mapas multi-nivel suportados:

Mapa	z_cutoff	Niveis	Notas
Nuke	-495	upper / lower	Dois plant sites em andares diferentes
Vertigo	11700	upper / lower	Arranha-ceu com duas areas jogaveis
Todos os outros	None	single	Mapas de nivel unico

SpatialEngine (`core/spatial_engine.py`):

O SpatialEngine adiciona capacidades de **raciocinio espacial** ao sistema de coordenadas base:

Metodo	Input	Output	Uso
<code>distance_2d(pos_a, pos_b)</code>	Duas coordenadas mundo	Float (unidades mundo)	Calculo distancia de engajamento
<code>is_visible(pos_a, pos_b, obstacles)</code>	Duas pos + obstaculos	Bool	Linha de visao (simplificada)
<code>get_zone(pos, map_name)</code>	Coordenada + mapa	String (ex. "A_site")	Classificacao zona tatica
<code>nearest_cover(pos, map_name)</code>	Coordenada + mapa	Coordenada cobertura	Sugestao posicional

AssetManager (`core/asset_manager.py`):

Gerencia o carregamento das texturas dos mapas e dos assets UI:

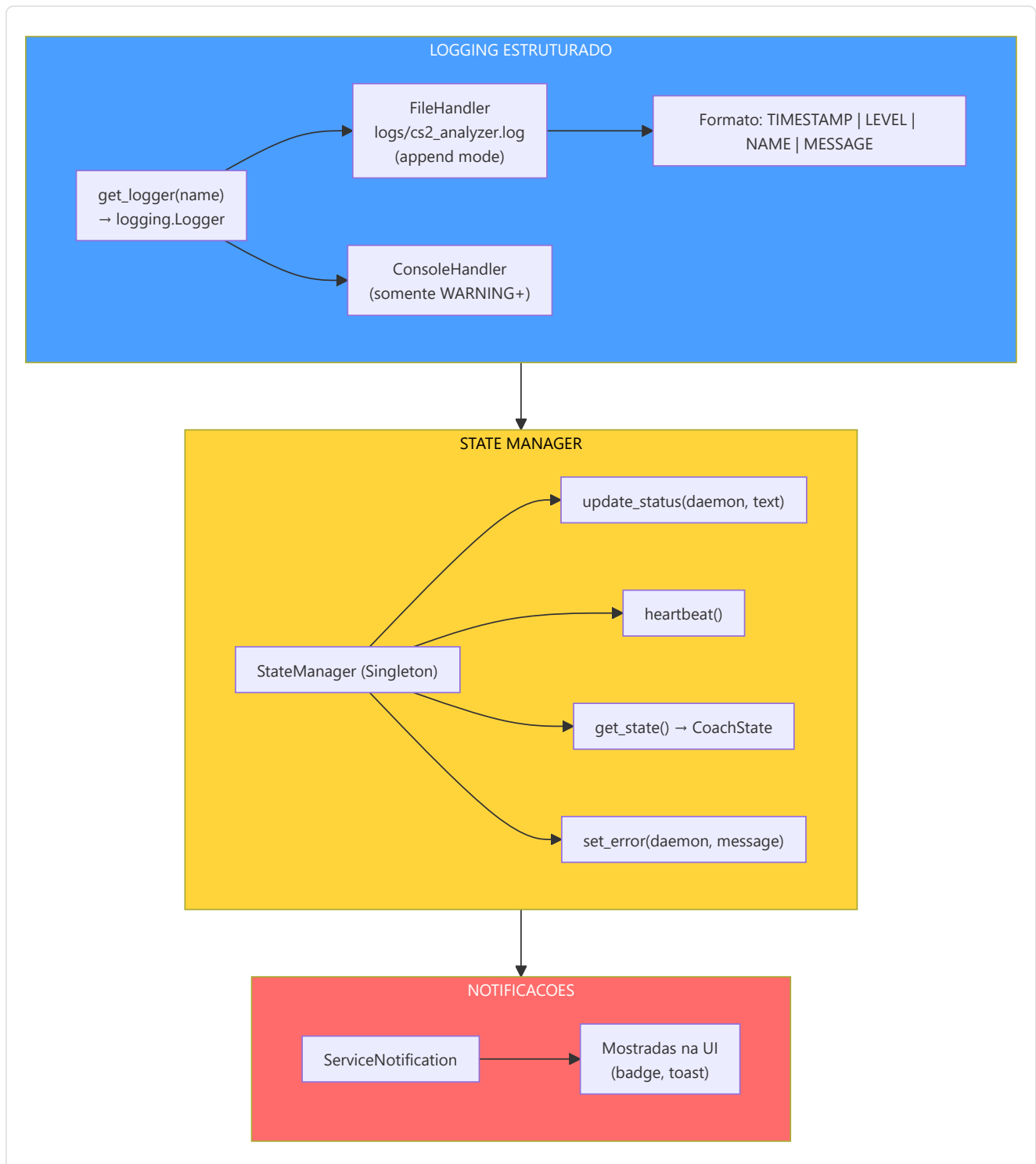
Asset	Formato	Tamanho tipico	Cache
Textura mapa (radar)	PNG 1024x1024	~500KB	Sim (in-memory)
Icones jogador	PNG 32x32	~2KB	Sim
Fontes (JetBrains Mono, YUPIX)	TTF	~200KB	Sim (registradas via Qt)
Wallpapers temas	PNG 1920x1080	~2MB	Lazy load

12.12 Observabilidade e Logging

Arquivo: `Programma_CS2_RENAN/observability/logger_setup.py` **Arquivos relacionados:**

`backend/storage/state_manager.py` , `backend/services/telemetry_client.py`

O sistema de observabilidade garante que cada evento significativo seja **rastreavel, estruturado e persistente**. O client de telemetria (`telemetry_client.py`) utiliza `httpx` (HTTP assincrono) para o envio nao-bloqueante de metricas e eventos, evitando que latencias de rede afetem o desempenho da aplicacao.

**Daemons monitorados pelo StateManager:**

Daemon	Campo em CoachState	Valores tipicos
Scanner	<code>hlvtv_status</code>	"Scanning", "Paused", "Error"
Digester (worker)	<code>ingest_status</code>	"Processing", "Idle", "Error"
Teacher (trainer)	<code>ml_status</code>	"Learning", "Idle", "Error"
Global	<code>status</code>	"Paused", "Training", "Idle", "Error"

Sentry Integration (`observability/sentry_setup.py`):

O sistema inclui integracao opcional com **Sentry** para error tracking remoto, com uma abordagem **double opt-in** para a privacidade:

Caracteristica	Implementacao
Double opt-in	O usuario deve (1) configurar <code>SENTRY_DSN</code> como variavel de ambiente E (2) ativar o flag nas configuracoes
PII Scrubbing	Todos os dados pessoais (nomes jogadores, Steam ID, paths) sao removidos antes do envio
Breadcrumb sanitization	Os breadcrumbs de navegacao sao limpos de informacoes sensiveis
Nao bloqueante	Se Sentry nao esta configurado, o sistema prossegue normalmente (Fase 4 da inicializacao)
Contexto enriquecido	Cada erro inclui: versao app, estado daemon, contagem demo, sistema operacional

Logger Setup (`observability/logger_setup.py`):

O sistema de logging centralizado fornece logs estruturados com:

Caracteristica	Detalhe
Formato	<code>`TIMESTAMP</code>
File handler	<code>logs/cs2_analyzer.log</code> (append mode, rotacao automatica)
Console handler	Somente <code>WARNING+</code> para nao poluir o output
Naming convention	<code>get_logger("cs2analyzer.<module>")</code> — namespace hierarquico
Niveis usados	<code>DEBUG</code> (desenvolvimento), <code>INFO</code> (operacoes normais), <code>WARNING</code> (anomalias nao criticas), <code>ERROR</code> (falhas), <code>CRITICAL</code> (terminacao)

Metricas de training expostas em CoachState: `current_epoch`, `total_epochs`, `train_loss`, `val_loss`, `eta_seconds`, `belief_confidence`, `system_load_cpu`, `system_load_mem`.

Registro Centralizado de Codigos de Erro (`observability/error_codes.py`):

O sistema implementa um registro formal de **24 codigos de erro** classificados por severidade e modulo. Cada codigo e definido como `ErrorCodeDef(NamedTuple)` com: `code`, `severity`, `module`, `description`, `remediation`.

Prefixo	Modulo	Exemplo	Severidade tipica
<code>LS</code>	Logger Setup	LS-01: RotatingFileHandler indisponivel	MEDIUM
<code>RP</code>	RASP Guard	RP-01: CS2_MANIFEST_KEY nao definido	HIGH
<code>DA</code>	Data Access	DA-01-03: JSON malformado do DB	LOW
<code>P</code>	Pipeline	P7-01/P7-02: Erros criticos de pipeline	HIGH
<code>F</code>	Feature/Fix	F6-SE: Erro de session engine	HIGH
<code>SE</code>	Session Engine	SE-05: Erro critico de sessao	HIGH
<code>IM</code>	Ingestion	IM-03: Erro de ingestao	MEDIUM
<code>NN</code>	Neural Network	NN-02: Erro de rede neural	MEDIUM
<code>CO</code>	Console Control	CO-01/CO-03: Erros criticos de controle	HIGH
<code>R1</code>	Release/Manifest	R1-12: Erro de manifesto de release	HIGH

Severidade (`Severity(Enum)`): `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.

Utilidades:

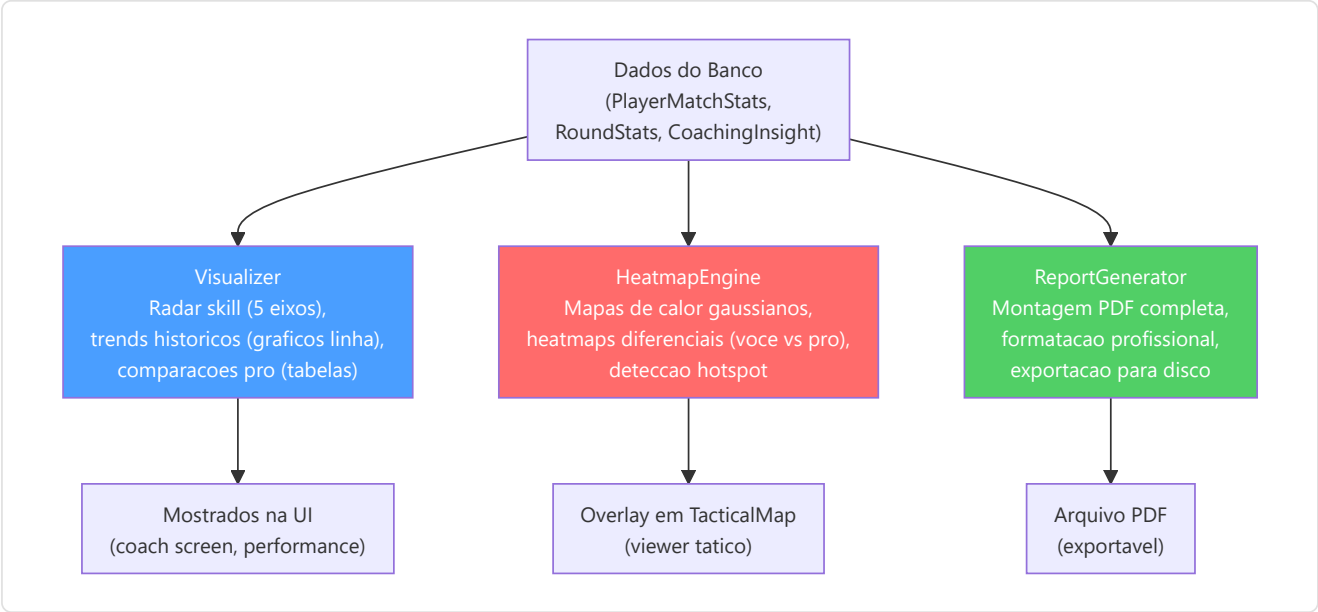
- `log_with_code(error_code, message) → str` — Prefixa a mensagem com o codigo formal (ex. `[LS-01] message`)
- `get_all_codes() → list[dict]` — Todos os codigos como lista de dicionarios para acesso programatico

Hierarquia de Excecoes (`observability/exceptions.py`): Classe base `CS2AnalyzerError(Exception)` com subclasses: `ConfigurationError`, `DatabaseError`, `IngestionError`, `TrainingError`, `IntegrationError`, `UIError`. Cada excecao pode carregar um `error_code: ErrorCode | None` para correlacao com o registro.

12.13 Reporting e Visualizacao

Arquivos: `Programma_CS2_RENAN/reporting/visualizer.py`, `report_generator.py` **Arquivos relacionados:** `backend/processing/heatmap_engine.py`

O sistema de reporting transforma os dados brutos em **visualizacoes compreensiveis** para o usuario.



HeatmapEngine — Detalhes tecnicos:

Caracteristica	Detalhe
Thread-safety	<code>generate_heatmap_data()</code> roda em thread separada (nao bloqueia UI)
Texture creation	Apenas no thread principal (requisito do thread Qt)
Tipo	Ocupacao gaussiana 2D (blur kernel parametrico)
Diferencial	Subtrai heatmap pro de heatmap usuario → vermelho (tempo demais), azul (pouco tempo)
Hotspot	Identifica clusters de posicao para training posicional

MatchVisualizer (`reporting/visualizer.py`) — metodos de rendering especializados:

O MatchVisualizer estende a capacidade de reporting com 6 metodos de rendering de alta qualidade (cf. secao 12.26 para os detalhes algoritmicos):

Metodo	Input	Output	Descricao
<code>render_skill_radar(user, pro)</code>	Dict metricas	PNG radar chart	Pentagono 5 eixos com overlay pro baseline
<code>render_trend_chart(history, metric)</code>	Lista historica	PNG line chart	Andamento temporal com media movel
<code>render_heatmap(positions, map_name)</code>	Coordenadas tick	PNG heatmap	Gaussiana 2D em textura mapa
<code>render_differential(user_pos, pro_pos)</code>	Dois sets posicoes	PNG heatmap diff	Vermelho (excesso) / Azul (deficit) / Verde (alinhamento)
<code>render_critical_moments(events)</code>	Lista eventos round	PNG timeline	Marcadores coloridos para kill/death/bomb
<code>render_comparison_table(user, pro)</code>	Dois perfis stats	HTML/PNG tabela	Delta percentual para cada metrica

ReportGenerator (`reporting/report_generator.py`):

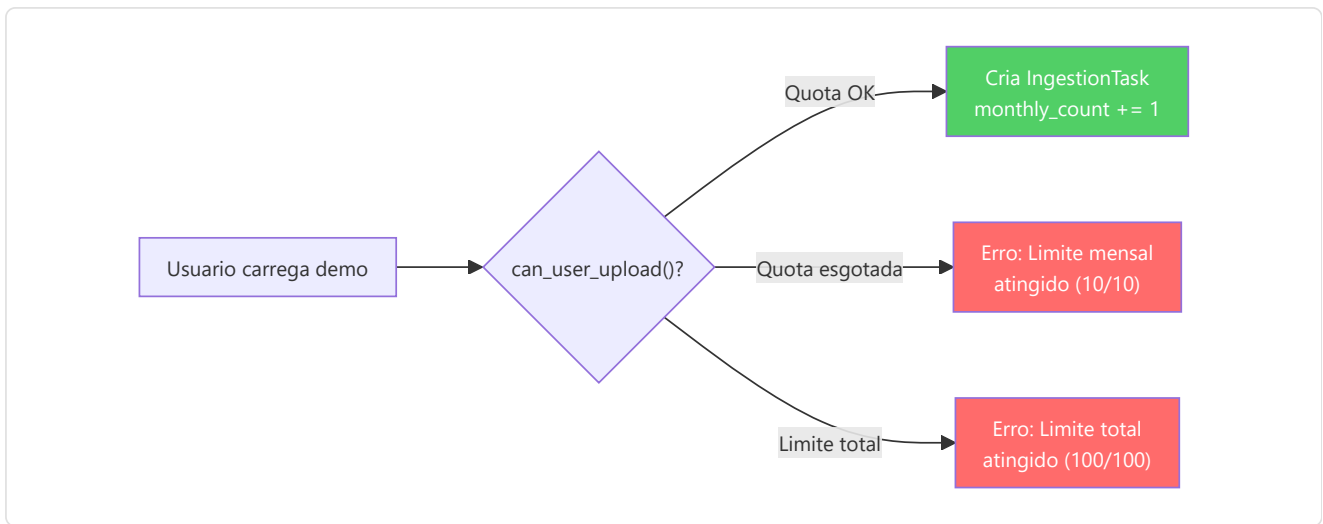
Monta todos os elementos visuais em um **documento PDF completo**:

1. **Header**: Nome jogador, data, demo analisada
2. **Executive Summary**: Metricas chave (KPR, ADR, KAST, Rating)
3. **Radar Chart**: Comparacao 5 eixos com pro baseline
4. **Trend Charts**: Tendencias das ultimas N partidas
5. **Heatmap Diferencial**: Onde se posicionar melhor
6. **Momentos Criticos**: Os 5 momentos decisivos da partida
7. **Conselhos do Coach**: Top-3 insights priorizados
8. **Footer**: Gerado por Macena CS2 Analyzer v.X.X

12.14 Gestao de Quotas e Limites

O sistema implementa um mecanismo de **quota mensal** para prevenir o abuso dos recursos de processamento e garantir uma distribuicao justa da carga.

Limite	Valor	Enforcement
MAX_DEMOS_PER_MONTH	10	<code>StorageManager.can_user_upload()</code>
MAX_TOTAL_DEMOS	100	<code>StorageManager.can_user_upload()</code>
MIN_DEMOS_FOR_COACHING	10	Limite para coaching personalizado completo
Reset mensal	Automatico	<code>PlayerProfile.last_upload_month</code> vs. mes atual



12.15 Tolerancia a Falhas e Recuperacao

O sistema e projetado para **nunca perder dados** e **se recuperar automaticamente** de quase todos os tipos de falha.

MECANISMOS DE TOLERANCIA A FALHAS

Zombie Task Cleanup
 Na inicializacao: tasks 'processing'
 → reset para 'queued'
 Restauro automatico sem perda

Backup Automatico
 Na inicializacao: checkpoint
 'startup_auto'
 Rotacao: 7 diarias + 4 semanais
 Copia completa banco

Service Supervisor
 Monitora servicos externos
 Auto-restart com backoff exponencial
 Max 3 tentativas/hora

Circuit Breaker HLTV
 MAX_FAILURES=10, RESET=3600s
 Estado: CLOSED→OPEN→HALF_OPEN
 Previne cascade failure

Connection Pooling
 20 conexoes persistentes
 Timeout 30s para contencao WAL
 Health checks automaticos

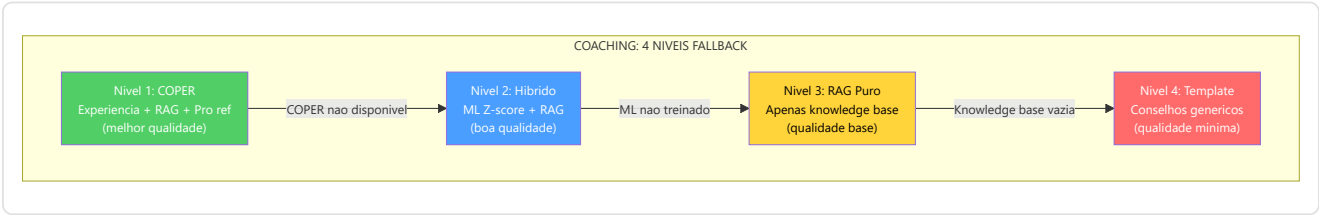
Degradacao Gradual
 Coaching: 4 niveis fallback
 GhostEngine: (0,0) se erro
 Cada servico tem um plano B

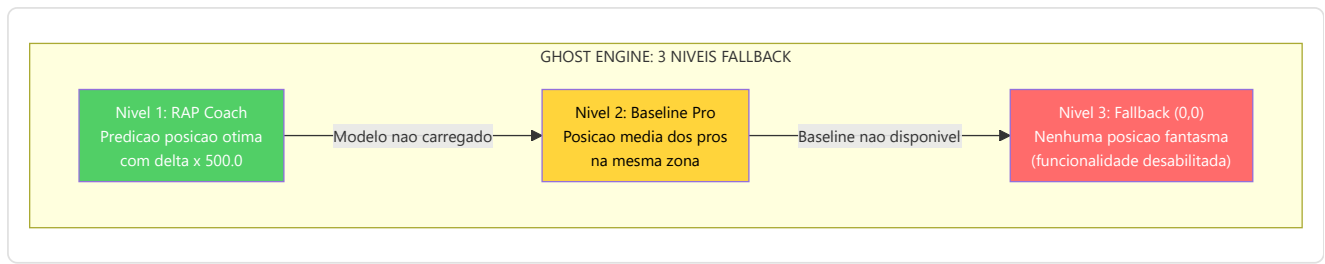
Matriz de falha e recuperacao:

Cenario de falha	Mecanismo de recuperacao	Perda de dados
Crash da aplicacao durante parsing	Zombie Task Cleanup no reinicio	Nenhuma — task recomecada
Corrupcao banco	Restore de backup automatico mais recente	Maximo 24 horas de dados

Cenario de falha	Mecanismo de recuperacao	Perda de dados
Servico HLTV nao alcancavel	Circuit Breaker <code>_CircuitBreaker</code> (MAX_FAILURES=10, RESET_WINDOW_S=3600): apos 10 falhas consecutivas o circuito se abre e bloqueia as requisicoes por 1 hora, depois passa para HALF_OPEN para um teste. Previne cascade failure em APIs externas.	Nenhuma — dados pro atrasados
Modelo ML nao carregavel	Fallback para pesos aleatorios (GhostEngine) ou coaching base	Nenhuma — qualidade degradada
RAM insuficiente durante training	Early stopping automatico, checkpoint salvo	Nenhuma — ultimo checkpoint valido
Disco cheio	Database Governor detecta e notifica via ServiceNotification	Prevencao — nenhum dado escrito
Demo corrompida/truncada	IntegrityChecker rejeita antes do parsing	Nenhuma — demo ignorada com warning
Ollama nao disponivel	LLM fallback para RAG puro, depois coaching base	Nenhuma — qualidade narrativa degradada
API Steam/FACEIT timeout	Retry com backoff exponencial (max 3 tentativas)	Nenhuma — perfil nao atualizado
Browser Playwright crash	BrowserManager: recria instancia, retry operacao	Nenhuma — scraping HLTV atrasado
Conflito WAL (lock contention)	Timeout 30s com retry automatico	Nenhuma — operacao atrasada
Checkpoint ML corrompido	Fallback para checkpoint anterior (versionamento)	Parcial — perde ultimo training
Vetor query norma-zero (M-07)	<code>VectorIndex.search()</code> retorna <code>None</code> com warning — pipeline RAG continua sem crash	Nenhuma — resultado RAG vazio
Arquivo settings.json corrompido (M-08)	<code>load_user_settings()</code> valida estrutura, fallback para default se nao-dict	Nenhuma — configuracoes padrao
Logger nao inicializado (C-09)	Bootstrap antecipado usa <code>print()</code> antes da init do logger	Nenhuma — mensagens em stdout
DB handle leak em integrity check (H-02)	<code>_verify_integrity()</code> usa <code>try/finally</code> + <code>PRAGMA quick_check</code>	Nenhuma — conexao sempre fechada
Erros batch mascarados em RAP training (H-03)	Removido <code>except (KeyError, TypeError): continue</code> — <code>train_step()</code> requer dict com key <code>'loss'</code>	Nenhuma — erros agora visiveis

Degradacao gradual — A Cadeia de Fallback:

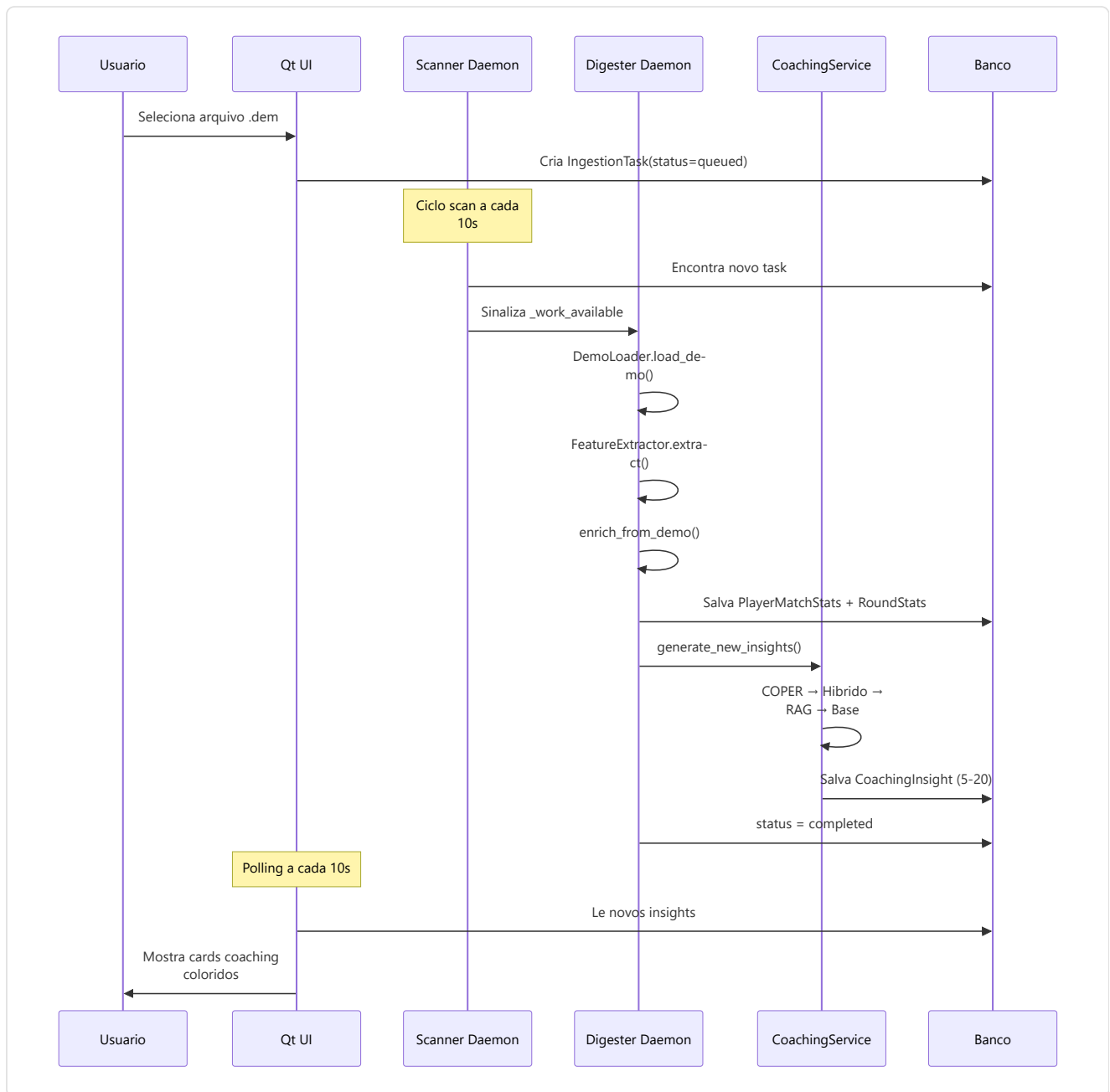




12.16 Jornada Completa do Usuario — 4 Fluxos Principais

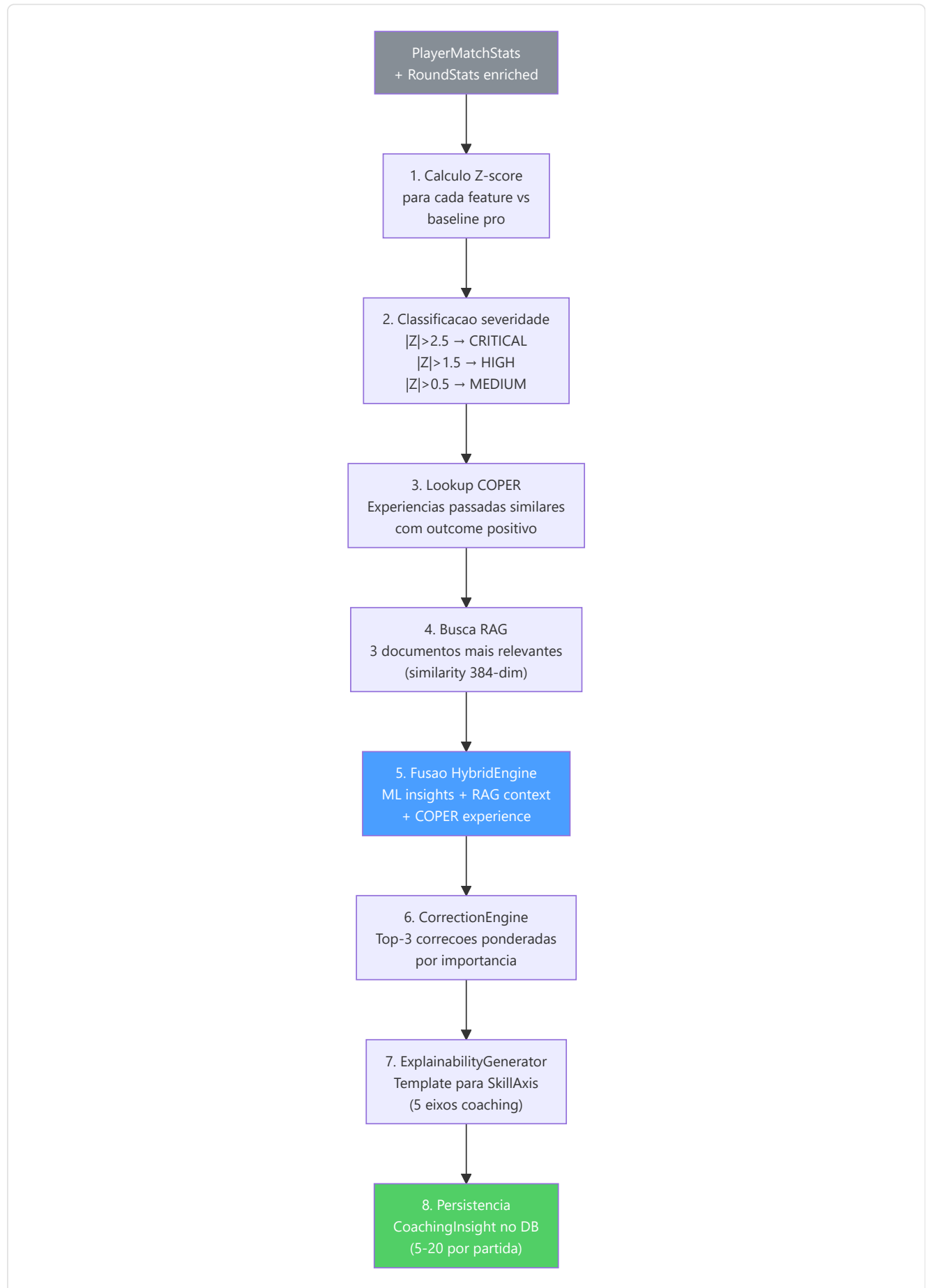
Esta secao descreve os **4 fluxos principais** que um usuario percorre durante o uso do Macena CS2 Analyzer.

Fluxo 1: Upload e Analise de uma Demo



Detalhe: Pipeline CoachingService interna

Quando `generate_new_insights()` é invocado, o CoachingService executa internamente:

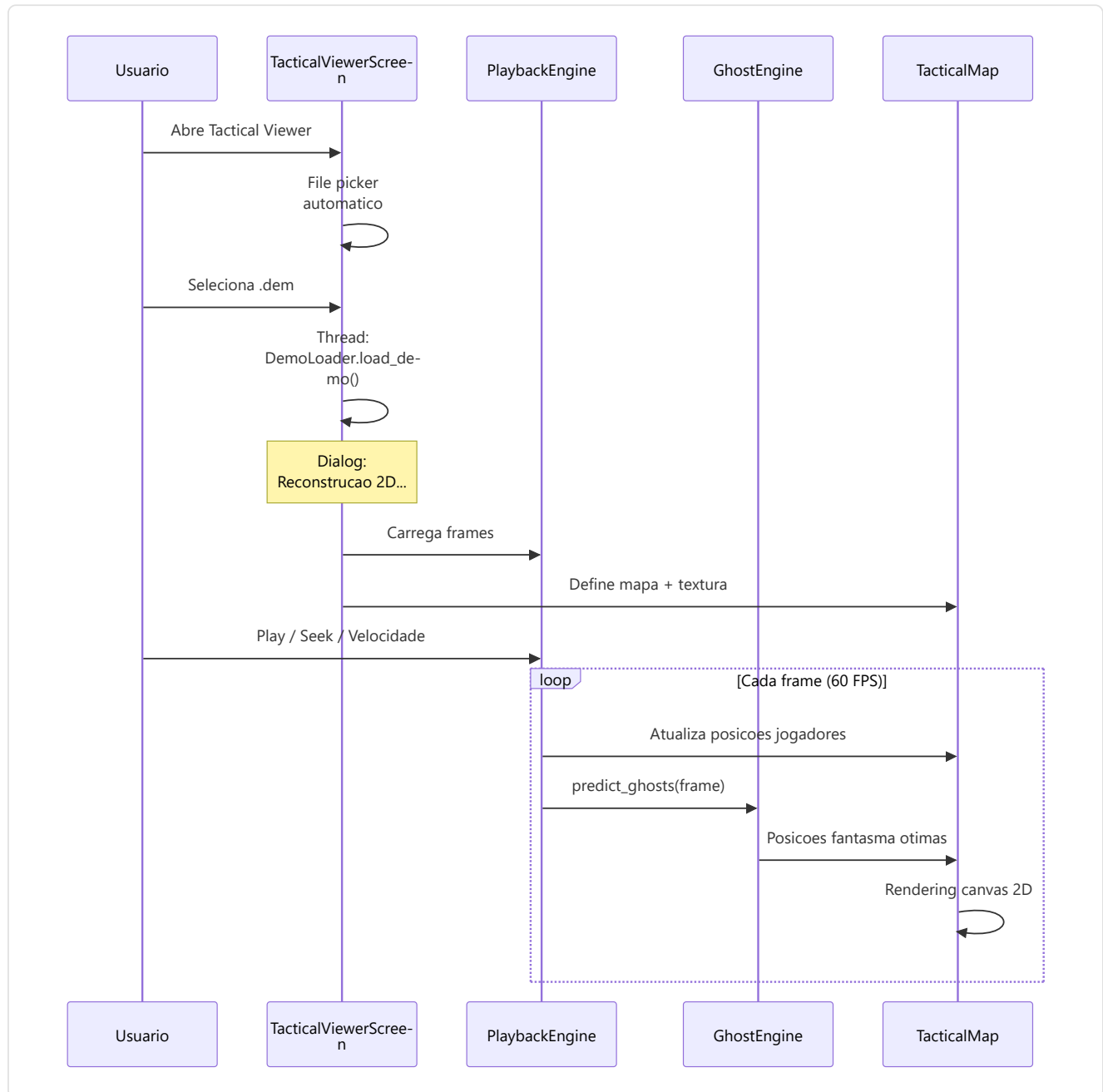


LessonGenerator — geracao de licoes estruturadas de demo:

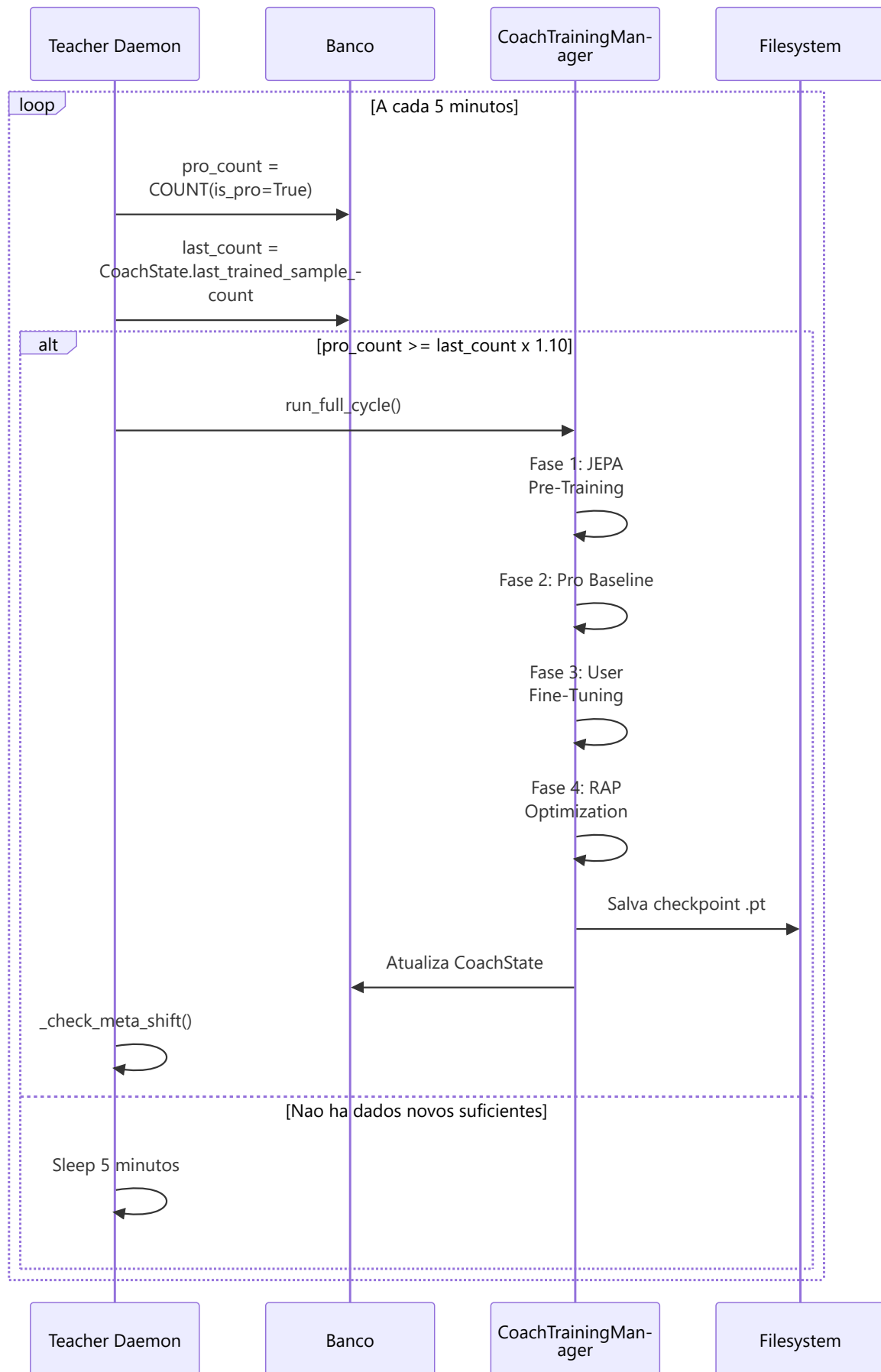
Parametro	Valor	Significado
ADR_STRONG	75	ADR >= 75 → "Bom dano medio"
HS_STRONG	0.40	HS% >= 40% → "Boa precisao na cabeca"
KAST_STRONG	0.70	KAST >= 70% → "Boa contribuicao ao round"
Formato output	Secao 1 (Overview) + Secao 2 (Pontos fortes) + Secao 3 (Areas de melhoria) + Secao 4 (Pro tip)	Estrutura licao padrao

O LessonGenerator opera em modo duplo: se Ollama esta disponivel, gera licoes em linguagem natural via `LLMService.generate_lesson()`. Se nao esta disponivel, usa templates estruturados com os dados numericos inseridos em frases preformadas.

Fluxo 2: Visualizacao Tatica



Fluxo 3: Retreinamento ML Automatico



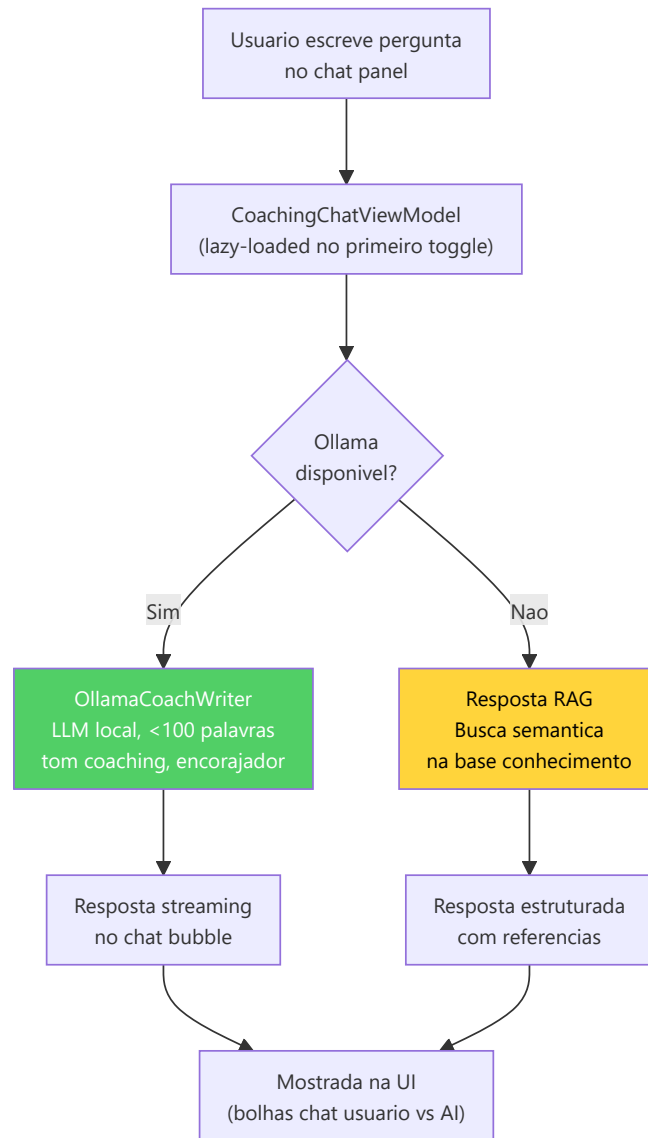
Fluxo 4: Chat AI com o Coach

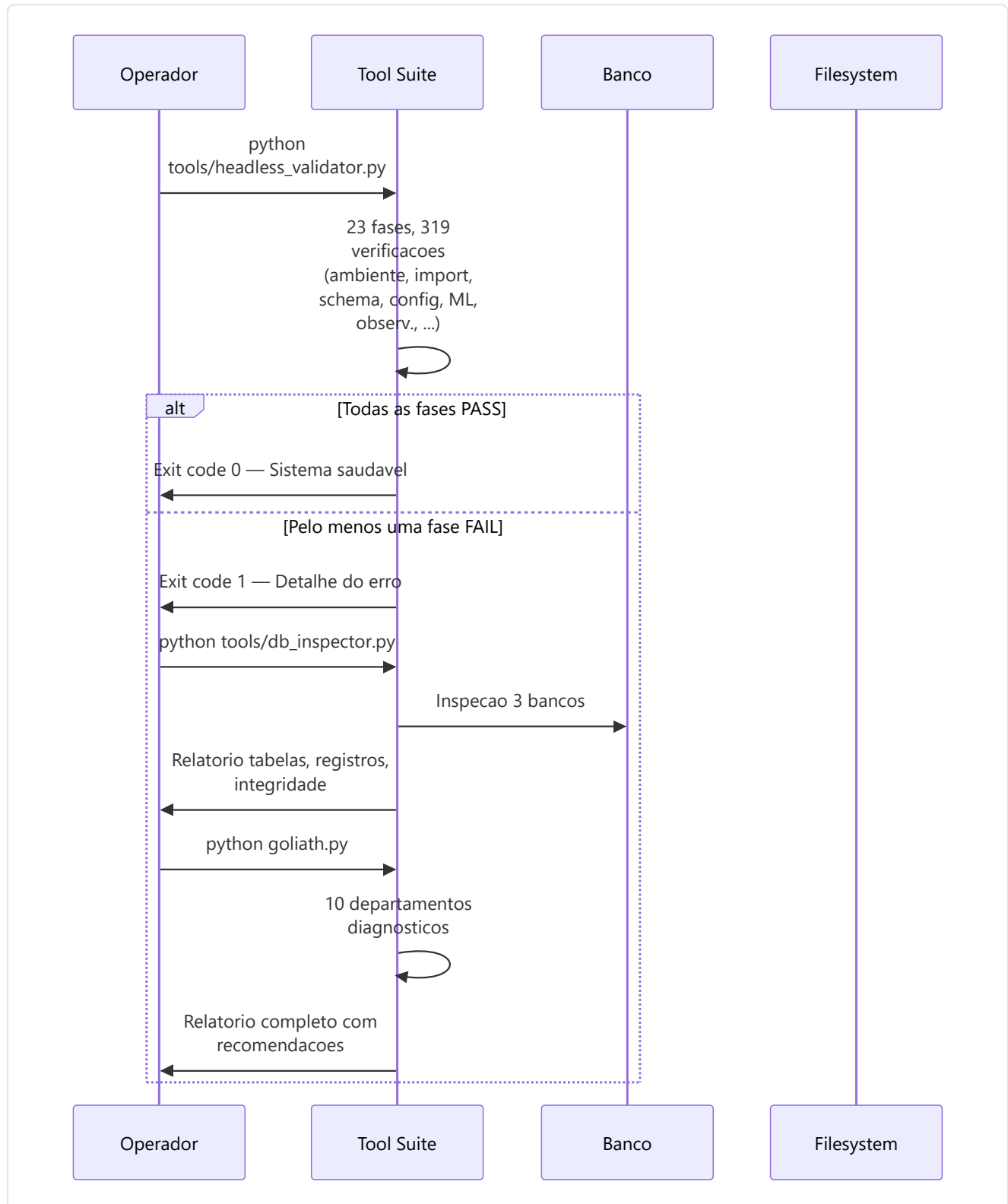
CoachingDialogueEngine — Pipeline Multi-Turno:

O dialogo AI segue uma pipeline de 5 fases para cada mensagem do usuario:

Fase	Componente	Descricao
1	Intent Classification	Classifica a pergunta: coaching, stats query, comparison, general
2	Context Retrieval	Recupera as ultimas N partidas, insights recentes, perfil jogador
3	RAG Augmentation	Busca na knowledge base os 3 documentos mais relevantes (similarity search 384-dim)
4	COPER Augmentation	Se disponivel, adiciona experiencias de coaching passadas com outcome positivo
5	Response Generation	Gera resposta via Ollama (se disponivel) ou template RAG estruturado

Parametro	Valor	Proposito
MAX_CONTEXT_TURNS	6	Numero maximo de turnos mantidos no contexto
MAX_RESPONSE_WORDS	100	Limite de palavras para resposta LLM
SIMILARITY_THRESHOLD	0.5	Limite minimo para resultados RAG
Tone	"coaching, encouraging"	Prompt system para tom positivo



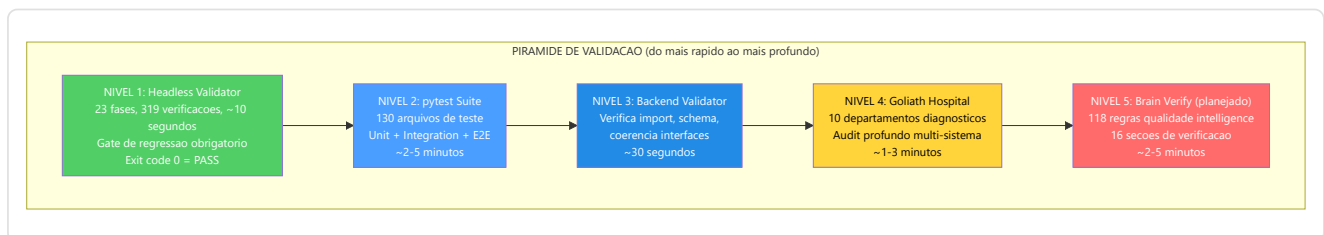
Fluxo 5: Diagnostico e Manutencao**Matriz ferramentas por cenario:**

Cenario	Ferramenta recomendada	Tempo
Quick check pos-deploy	<code>headless_validator.py</code>	~10s
Banco suspeito	<code>db_inspector.py</code>	~30s
Demo nao parsed	<code>demo_inspector.py</code>	~5s
Training diverge	<code>Ultimate_ML_Coach_Debugger.py</code>	~2min
Check-up completo	<code>Goliath_Hospital.py</code>	~3min
Audit ML profundo	<code>brain_verify.py</code>	~5min

12.17 Suite de Ferramentas — Validacao e Diagnostico (`tools/`)

Diretorio: `tools/` (root) + `Programma_CS2_RENAN/tools/` **Arquivos principais:** `headless_validator.py`, `db_inspector.py`, `demo_inspector.py`, `brain_verify.py` (*planejado, ainda nao implementado*), `Goliath_Hospital.py`, `Ultimate_ML_Coach_Debugger.py`, `_infra.py`

A suite de ferramentas e uma **colecão de 41 scripts** distribuidos em duas diretorias (`tools/` root com 29 scripts e `Programma_CS2_RENAN/tools/` com 12 scripts) que formam uma piramide de validacao multi-nivel. Cada ferramenta tem um proposito preciso e pode ser executada independentemente, mas juntas formam um sistema de garantia de qualidade que cobre cada aspecto do projeto.



Headless Validator (`tools/headless_validator.py`) — o gate de regressao obrigatorio (Dev Rule 9). Executado apos **cada** task de desenvolvimento com 23 fases e 319 verificacoes:

Grupo de fases	Verificacao
Ambiente / Deps / GPU / Platform	Python e dependencias criticas presentes (torch, pyside6, sqlmodel, demoparser2), dispositivo e plataforma
Core / NewlImport / Structure	Os modulos fundamentais (<code>config.py</code> , <code>spatial_data.py</code> , <code>lifecycle.py</code>) carregam sem erros
NN / ML / ML-Deep / RAP / Training	Instanciacao dos modelos com pesos aleatorios — dimensoes e forward pass
DB / DB-Deep / Schema / Storage	As tabelas SQLAlchemy (25 + 3 por-match) sao criadas corretamente
Config / Config-Deep / Features / Processing	<code>METADATA_DIM</code> , caminhos, constantes coerentes
Coaching / Knowledge / Services / Analysis / Belief / Baseline	Contratos dos subsistemas de coaching e analise
Ingestion / DataSrc / Adapter / Integrity / Security	Pipeline de ingestao, validacao de demos, seguranca
UI / Qt-Import / Design-Tokens / Reporting / Quality / Quality-Adv	Interface, design tokens, qualidade

Infraestrutura Compartilhada (`tools/_infra.py`):

Componente	Papel
<code>BaseValidator</code>	Classe base para todos os validadores — output console unificado, contagem pass/fail/warning
<code>path_stabilize()</code>	Normalizacao <code>sys.path</code> para evitar imports relativos inconsistentes
<code>Console</code> (Rich)	Output colorido com tabelas, progress bar, emoji de estado
<code>Severity</code> enum	4 niveis: <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code>

DB Inspector (`tools/db_inspector.py`) — inspecao profunda do banco:

- Abre `database.db` e `hltv_metadata.db` separadamente
- Para cada tabela: conta registros, verifica schema, controla integridade indices
- Mostra metricas de espaco (tamanho arquivo, paginas WAL, fragmentacao)
- Detecta anomalias: tabelas vazias inesperadas, registros orfaos, timestamps fora do range
- Output: tabela Rich colorida com estado para cada tabela

Demo Inspector (`tools/demo_inspector.py`) — inspecao arquivo demo:

- Verifica magic bytes (`PBDEMS2\0` para CS2, `HL2DEMO\0` para legacy)
- Controla tamanho arquivo (1KB min, 5GB max)
- Extrai metadata header sem parsing completo
- Detecta corrupcao parcial (truncamento, header danificado)
- Output: relatorio sobre validade da demo com detalhes erro se invalida

Brain Verify (`tools/brain_verify.py` + `brain_verification/` , 16 secoes) — *(planejado, ainda nao implementado)*:

O sistema de verificacao da inteligencia e projetado com **16 secoes tematicas** que cobrirao **118 regras** de qualidade:

BRAIN VERIFY — 16 SECOES (118 REGRAS)

Secao 1 Dimensional Contracts
METADATA_DIM, latent_dim,
input shapes

Secao 2 Forward Pass
Smoke test todos os modelos
com input sintetico

Secao 3 Loss Functions
Gradient flow, NaN check,
loss monotonicity

Secao 4 Training Pipeline
DataLoader, optimizer,
scheduler config

Secao 5 Feature Engineering
Vectorizer alignment,
normalization bounds

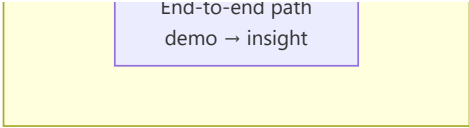
Secao 6 Coaching Pipeline
COPER → Hybrid → RAG
fallback chain

Secao 7 Data Integrity
DB schema, FK constraints,
orphan detection

Secao 8 Concept Alignment
16 coaching concepts,
label correctness

...

Secao 16 Integration



End-to-end path
demo → insight

Goliath Hospital ([tools/Goliath_Hospital.py](https://github.com/UltimateMLCoach/Goliath_Hospital.py)) — diagnostico multi-departamento:

O "Goliath Hospital" e o **sistema de diagnostico mais completo** do projeto, organizado como um hospital com 10 departamentos especializados:

Departamento	Nome	Verificacoes
1	Pronto-Socorro (ER)	Sintaxe, padroes proibidos, namespace
2	Radiologia	Assets, estrutura de arquivos/diretorios
3	Patologia	Qualidade dos dados, deteccao de dados mock
4	Cardiologia	Modulos core, DB, config, motores de analise
5	Neurologia	ML/IA, forward pass
6	Oncologia	Debito tecnico
7	Pediatria	Arquivos modificados recentemente
8	Terapia Intensiva (ICU)	Integracao, imports
9	Farmacia	Dependencias, versoes, compatibilidade
10	Clinica dos Instrumentos	Validacao dos outros instrumentos (meta-teste)
11	Endocrinologia	Entry points, migracoes, validacao JSON

Ultimate ML Coach Debugger ([tools/Ultimate_ML_Coach_Debugger.py](https://github.com/UltimateMLCoach/Ultimate_ML_Coach_Debugger.py)) — falsificacao das crenças neurais:

Esta ferramenta executa um audit em **3 fases** na pipeline ML:

1. **Verificacao Estrutural** — Dimensoes tensor, arquitetura modelos, parametros learnable
2. **Verificacao Comportamental** — Forward pass com dados reais, gradient flow, loss convergence
3. **Falsificacao Crenças** — Testa se o modelo "acredita" em coisas erradas: predicoes overconfident, bias sistematicos, patterns degenerados

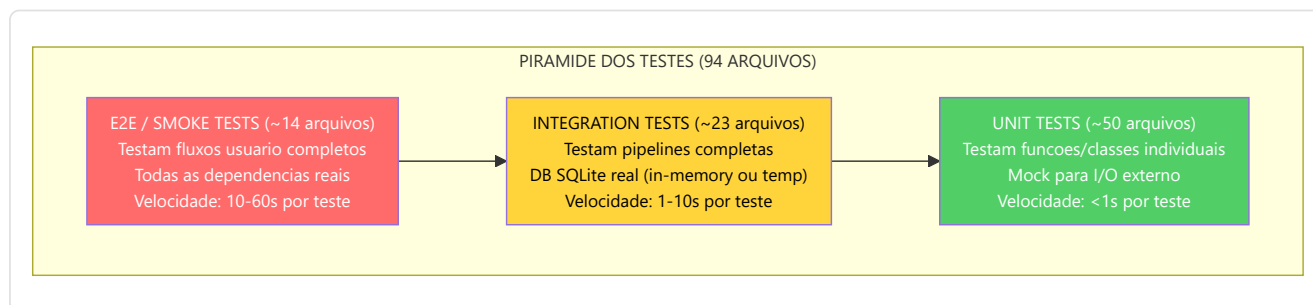
Outros instrumentos:

Instrumento	Arquivo	Proposito
<code>backend_validator.py</code>	<code>tools/</code>	Verifica coerencia import e interfaces backend
<code>build_tools.py</code>	<code>tools/</code>	Automacao build PyInstaller
<code>user_tools.py</code>	<code>tools/</code>	Utilidades para o usuario final (reset, export)
<code>dev_health.py</code>	<code>tools/</code>	Quick check saude desenvolvimento
<code>context_gatherer.py</code>	<code>tools/</code>	Coleta contexto para debug report
<code>dead_code_detector.py</code>	<code>tools/</code>	Identifica codigo morto nao referenciado
<code>sync_integrity_manifest.py</code>	<code>tools/</code>	Atualiza <code>integrity_manifest.json</code> com hash SHA-256
<code>project_snapshot.py</code>	<code>tools/</code>	Snapshot completo do estado do projeto
<code>ui_diagnostic.py</code>	<code>tools/</code>	Diagnostico especifico UI (Qt)
<code>build_pipeline.py</code>	Root <code>tools/</code>	Pipeline de build completa
<code>Feature_Audit.py</code>	Root	Audit do feature vector 25-dim
<code>Sanitize_Project.py</code>	Root	Limpeza arquivos temporarios, cache, artifacts
<code>audit_binaries.py</code>	Root	Verifica integridade executaveis e .pt

12.18 Arquitetura da Test Suite (`tests/`)

Diretorio: `Programma_CS2_RENAN/tests/` **Arquivos totais:** 130 arquivos de teste + `conftest.py` + 10 scripts forensics + 15 scripts de verificacao

A test suite e organizada segundo o **princípio da piramide dos testes**: muitos unit tests (rapidos, isolados), menos integration tests (mais lentos, com dependencias reais), e poucos end-to-end tests (completos mas custosos).



Conftest (`tests/conftest.py`) — fixtures compartilhadas:

Fixture	Scope	Descricao
<code>test_db</code>	<code>function</code>	Banco SQLite in-memory com todas as tabelas criadas, auto-cleanup
<code>sample_match_stats</code>	<code>function</code>	<code>PlayerMatchStats</code> com dados realísticos derivados de partidas reais
<code>mock_demo_data</code>	<code>function</code>	<code>DemoData</code> mock com frames e eventos para testes de parsing
<code>tmp_models_dir</code>	<code>function</code>	Diretorio temporario para checkpoints <code>.pt</code>
<code>coaching_service</code>	<code>function</code>	<code>CoachingService</code> configurado com DB de teste

Automated Suite (`tests/automated_suite/`):

Categoria	Arquivo	Cobertura
Smoke	<code>test_smoke_*.py</code>	Imports criticos, instanciacao modelos, DB connection
Unit	<code>test_unit_*.py</code>	Funcoes puras, calculos, transformacoes
Functional	<code>test_functional_*.py</code>	Pipelines completas, fluxos de trabalho
E2E	<code>test_e2e_*.py</code>	Demo upload → parsing → coaching → insight
Regression	<code>test_regression_*.py</code>	Bug fixes especificos (G-01 label leakage, G-07 Bayesian, etc.)

Testes por subsistema:

Subsistema	Arquivos de teste	O que testam
NN Core	<code>test_jepa_model.py</code> , <code>test_rap_model.py</code> , <code>test_advanced_nn.py</code>	Forward pass, dimensoes tensor, gradient flow
Coaching	<code>test_coaching_service.py</code> , <code>test_hybrid_engine.py</code> , <code>test_correction_engine.py</code>	Pipeline coaching, priorizacao insights, fallback chain
Processing	<code>test_vectorizer.py</code> , <code>test_heatmap.py</code> , <code>test_feature_eng.py</code>	Feature extraction, normalizacao, METADATA_DIM=25
Storage	<code>test_database.py</code> , <code>test_models.py</code> , <code>test_backup.py</code>	CRUD, schema, backup/restore, WAL mode
Data Sources	<code>test_demo_parser.py</code> , <code>test_hltv_scraper.py</code> , <code>test_steam_api.py</code>	Parsing, scraping, API timeout/retry
Analysis	<code>test_game_theory.py</code> , <code>test_belief.py</code> , <code>test_role_engine.py</code>	Motores analise, calibracao, heuristica
Knowledge	<code>test_rag.py</code> , <code>test_experience_bank.py</code> , <code>test_knowledge_graph.py</code>	Retrieval, COPER eficacia, KG query
Ingestion	<code>test_ingest_pipeline.py</code> , <code>test_registry.py</code>	Pipeline completa, deduplicacao

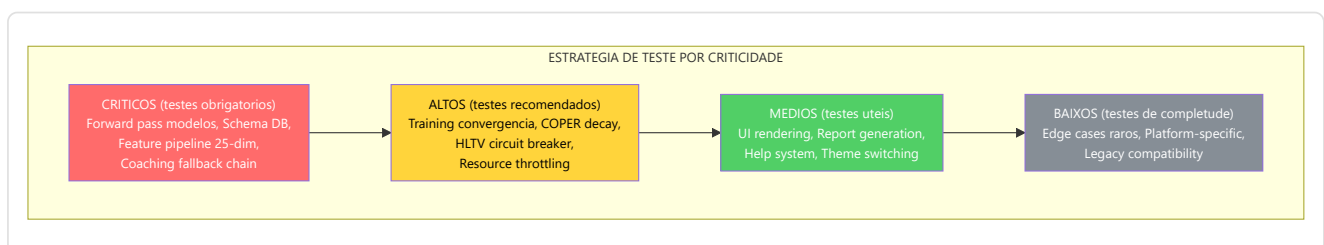
Forensics (`tests/forensics/` na raiz, 10 scripts):

Os scripts forensics sao ferramentas de diagnostico para investigacoes post-mortem: `check_db_status.py`, `check_failed_tasks.py`, `debug_env.py`, `debug_nade_cols.py`, `debug_parser_fields.py`, `probe_missing_tables.py`, `test_forensic_parser.py`, `test_skill_logic.py`, `verify_map_dimensions.py`, `verify_spatial_integrity.py`.

Verification Scripts (6 arquivos `verify_*.py` na raiz `tests/`):

Scripts de verificacao one-shot para validar aspectos especificos do sistema (coerencia espacial, pipeline de dados, integridade), acompanhados pelos testes da raiz `test_d3_rederive.py`, `test_eval_harness.py`, `test_lock_files.py`, `test_rescrape_placeholder_pros.py`, `test_sync_pro_players.py`.

Estrategia de teste por criticidade:



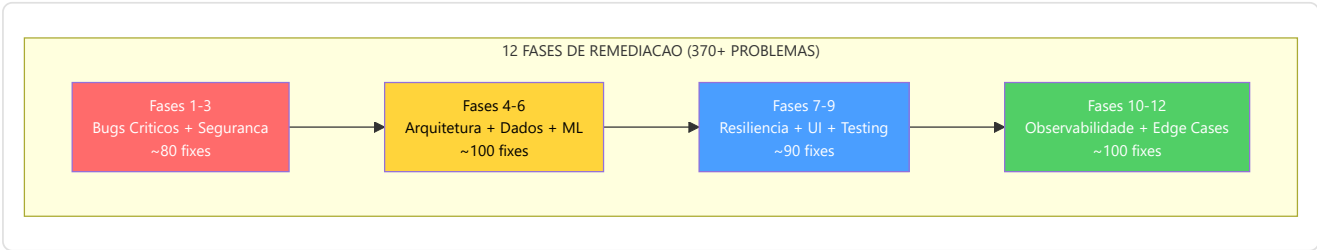
Gaps de cobertura identificados:

Os seguintes modulos tem alta complexidade (>500 LOC) mas cobertura de teste limitada:

Modulo	LOC	Cobertura teste	Risco
training_orchestrator.py	733	Baixa	ALTO — orquestracao multi-fase
session_engine.py	538	Media	MEDIO — testavel somente com subprocess
coaching_service.py	585	Media	MEDIO — 4 niveis fallback
experience_bank.py	751	Baixa	ALTO — logica COPER complexa
tensor_factory.py	686	Baixa	ALTO — DataLoader/Dataset
coach_manager.py	878	Baixa	ALTO — estado global training

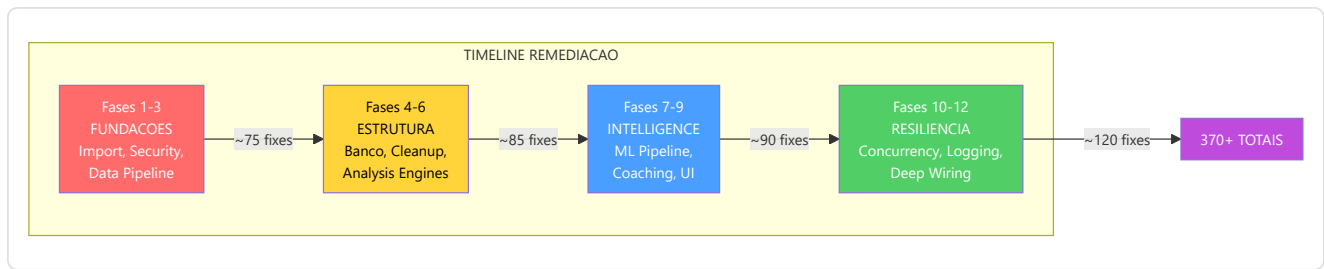
12.19 As Fases de Remediacao Sistematica

O projeto atravessou um processo de **remediacao em 12 fases** que resolveu no total **370+ problemas** identificados durante audits de qualidade progressivos. Cada fase se concentrou em uma categoria especifica de problemas, da correcao de bugs criticos a reestruturacao arquitetural. A estas se soma a campanha de auditoria de agosto de 2026, que mudou o metodo — ler tudo antes de decidir o que e um problema — e esta descrita em §12.19.1.



Detalhe por fase:

Fase	Foco	Problemas resolvidos	Exemplo chave
1	Import e estrutura base	~20	Circular imports, path resolution, missing <code>__init__.py</code>
2	Seguranca e secrets	~25	API keys hard-coded → env vars/keyring, input validation
3	Data pipeline e features	~30	G-01 label leakage, G-02 normalizacao bounds, feature alignment
4	Banco e schema	~30	WAL mode enforcement, missing indices, schema migration safety
5	Dead code e cleanup	~25	G-06 eliminacao <code>nn/advanced/</code> , imports nao usados, arquivos duplicados
6	Analysis engines	~30	Graceful degradation para todos os 11 motores, edge case handling
7	ML pipeline	~35	G-07 Bayesian calibration, gradient clipping, checkpoint versioning
8	Coaching e COPER	~30	G-08 experience decay, RAG index validation, fallback chain
9	UI e UX	~25	F8-XX feedback visual, state consistency, error prevention
10	Resiliencia e concorrencia	~40	F5-35 threading.Event, timeout enforcement, circuit breaker
11	Observabilidade e logging	~35	F5-33 structured logging, correlation IDs, Sentry integration
12	Deep debug e wiring	~45	18 issues: wiring verification, integration testing, edge case audit

**Diretorio `reports/` :**

Cada fase de remediacao produziu um relatorio detalhado salvo na diretorio `reports/`. Cada relatorio inclui:

- Lista numerada dos problemas encontrados com codigo (G-XX ou F-XX)
- Descricao do problema com arquivo e linhas afetadas
- Solucao implementada com diff conceitual
- Verificacao: confirma que o fix nao introduziu regressoes

Correcoes chave por codigo (G-XX):

Codigo	Fase	Descricao	Impacto
G-01	3	Label leakage em <code>ConceptLabeler.label_tick()</code> — features futuras vazadas nas labels de training	CRITICO — invalidava o treinamento VL-JEPA
G-02	4	Danger zone em <code>vectorizer.py</code> — normalizacao sem bounds check	ALTO — NaN propagation em training
G-06	5	Dead code em <code>backend/nn/advanced/</code> — 3 arquivos nao referenciados	MEDIO — confusao e imports acidentais
G-07	7	Bayesian death estimator nao calibrado — modelo estatico sem auto-calibration	ALTO — predicoes imprecisas
G-08	8	COPER experience bank sem decay — experiencias obsoletas nunca removidas	MEDIO — coaching baseado em dados envelhecidos

Correcoes por feature (F-XX):

Codigo	Area	Descricao
F3-29	Processing	ResNet resize inconsistente — dimensoes imagem nao normalizadas
F5-19	Services	Matplotlib rendering sem error handling — crash em stats vazias
F5-21	Storage	<code>session.commit()</code> explicito dentro de context manager — commit duplo
F5-22	Security	API keys hard-coded em source — migrate para env vars + keyring
F5-33	Observability	<code>print()</code> debugging — migrado para structured logging
F5-35	Control	<code>time.sleep()</code> loop para stop — migrado para <code>threading.Event</code>
F6-XX	Analysis	Motores analise sem graceful degradation — crash em inputs incompletos
F7-XX	Knowledge	RAG sem index validation — embedding dimensoes incoerentes
F8-XX	UI	Widgets Qt sem feedback visual — acoes silenciosas confundem o usuario

12.19.1 A campanha de auditoria integral (agosto de 2026)

As ondas anteriores partiam de um sintoma: algo quebrava, procurava-se a causa, corrigia-se. A campanha de agosto de 2026 inverteu o metodo — leu **todos** os arquivos do repositorio, em duas passagens, antes de decidir o que era um problema.

A primeira passagem foi por arquivo: 618 arquivos lidos em 76 lotes, cada um com o seu proprio dossie em `docs/audit/dossiers/`. A segunda foi transversal, por lente: dez contratos que cortam o codigo de travessia em vez de por pasta — ticks e tensores, seguranca das threads Qt, ciclo de vida das sessoes de DB, tratamento de erros, recursos, configuracao e caminhos, internacionalizacao, correcao numerica e de ML, seguranca, codigo morto (`docs/audit/CONTRACTS.md`).

Dali sairam **44 achados**: nenhum P0, 12 P1 (correcao, threading ou recursos sob uso real), 32 P2 (deriva de contrato e codigo morto). Trinta e um foram corrigidos, **cada um com o seu proprio teste de regressao no mesmo commit**; treze foram adiados, cada um com a condicao bloqueante escrita por extenso — dados de referencia ausentes, verdade visual faltante, ou uma pergunta de pesquisa que um patch nao resolve.

O que sobrevive a campanha, porem, nao sao as 31 correcoes: sao os **testes de doutrina**, que nao verificam um comportamento mas proíbem o retorno de uma classe inteira de erro.

Invariante	Teste	Como morde
Nenhum acesso ao DB pela thread da GUI	<code>test_screens_no_gui_thread_db.py</code>	Inspeciona o codigo das telas: a funcao que toca o DB deve ser um <code>staticmethod</code> executado por um <code>Worker</code>
Nenhum tick rate escrito a mao	<code>test_tick_rate_ssot.py</code>	Varredura AST (imune a comentarios e docstrings) com um teste-isca que falha se o scanner parar de morder
Nenhuma lista de mapas redeclarada	<code>test_known_maps_ssot.py</code>	Proíbe o trio mirage/inferno/nuke nos consumidores convertidos e verifica que cada um ainda importa a SSOT
Uma unica configuracao pytest	<code>test_single_pytest_config.py</code>	Verifica a ausencia do arquivo sombra e confirma com um subprocesso qual configuracao e resolvida
Toda ferramenta mutante e protegida	<code>test_verify_all_safe_gate.py</code>	Censo: nenhuma ferramenta destrutiva pode ser invocada nua
Todo token citado existe de fato	<code>test_design_token_references.py</code>	Confronta cada <code>tokens.<nome></code> no codigo Qt com os campos reais da dataclass
Os chips se recolorem na troca de tema	<code>test_theme_live_restyle.py</code>	Passa pelo relay de modulo
O timeout de parsing e real	<code>test_parse_timeout_real.py</code>	Com um worker travado por 30s, o chamador deve retornar em menos de 5
Um unico runner reclama cada demo	<code>test_ingestion_atomic_claim.py</code>	Claim exclusivo e tarefa ja reclamada sendo pulada
O classificador recusa dicionarios sem vocabulario	<code>test_role_vocabulary_guard.py</code>	Veja a Parte 2, motores de analise
Zero imports de QtCharts	<code>test_charts.py</code>	Gate de licenca

Na mesma campanha o piso minimo de cobertura subiu de 33% para **50%** (`pyproject.toml`), entrou o `ruff` com um conjunto de regras inicial, e foi acrescentado um gate `pip check` sobre as dependencias.

As cifras do portao de testes relatadas pela campanha — 2.574 testes verdes, zero falhos, zero erros — sao as registradas em `docs/audit/FINAL_REPORT.md` em 14 de agosto de 2026. O que e verificavel lendo o repositorio, sem executar nada, e o seu tamanho: **167 arquivos de teste** e **2.470 funcoes `test_` estaticas**.

12.20 Pre-commit Hooks e Quality Gates

Arquivo: `.pre-commit-config.yaml`

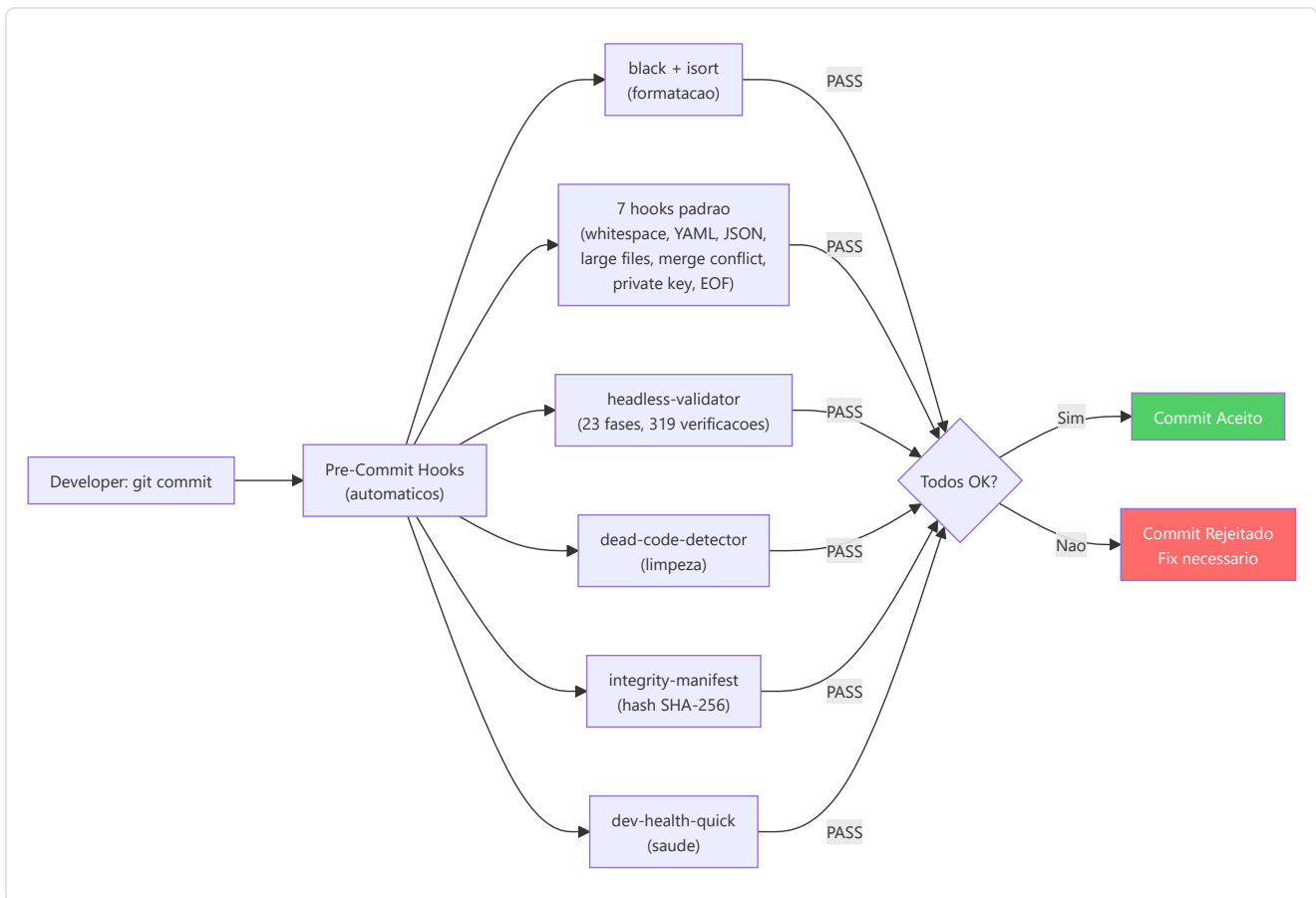
O projeto utiliza um sistema de **pre-commit hooks** que se ativam automaticamente antes de cada commit, impedindo que código não conforme chegue ao repositório.

4 Hooks Locais (custom do projeto):

Hook	Script	Timeout	Descricao
<code>headless-validator</code>	<code>tools/headless_validator.py</code>	20s	23 fases, 319 verificacoes regression gate — o mais importante
<code>dead-code-detector</code>	<code>tools/dead_code_detector.py</code>	15s	Identifica imports e funcoes nao referenciados
<code>integrity-manifest-check</code>	<code>tools/sync_integrity_manifest.py</code>	10s	Verifica coerencia hash SHA-256 do manifesto
<code>dev-health-quick</code>	<code>tools/dev_health.py</code>	10s	Quick check saude projeto

Hooks Padrao (7 de `pre-commit/pre-commit-hooks` + 2 Python):

Hook	Fonte	Descricao
<code>trailing-whitespace</code>	<code>pre-commit/pre-commit-hooks</code>	Remocao espacos em branco finais
<code>end-of-file-fixer</code>	<code>pre-commit/pre-commit-hooks</code>	Newline final obrigatorio
<code>check-yaml</code>	<code>pre-commit/pre-commit-hooks</code>	Validacao sintaxe YAML
<code>check-json</code>	<code>pre-commit/pre-commit-hooks</code>	Validacao sintaxe JSON
<code>check-added-large-files</code>	<code>pre-commit/pre-commit-hooks</code>	Bloqueia arquivos >1MB (previne <code>.pt</code> acidentais)
<code>check-merge-conflict</code>	<code>pre-commit/pre-commit-hooks</code>	Detecta marcadores de merge conflict nao resolvidos
<code>detect-private-key</code>	<code>pre-commit/pre-commit-hooks</code>	Bloqueia commit de chaves privadas
<code>black</code>	<code>psf/black</code>	Formatacao automatica Python (line length 100, target py3.12)
<code>isort</code>	<code>pycqa/isort</code>	Ordenacao automatica imports (profile black, line length 100)



12.21 Build, Packaging e Deployment

Arquivos chave: `packaging/windows_installer.iss`, `setup_new_pc.bat`, `export_env.bat`

O projeto inclui um sistema de build e packaging para a distribuicao da aplicacao desktop em Windows.

3 Arquivos Requirements:

Arquivo	Proposito	Dependencias
<code>requirements.txt</code>	Base — dependencias principais para execucao	28 pacotes (torch, pyside6, sqlmodel, demoparser2, playwright, httpx, etc.)
<code>requirements-ci.txt</code>	CI/CD — adiciona ferramentas de teste e analise	pytest, coverage, mypy, black, isort, pre-commit
<code>requirements-lock.txt</code>	Lock — versoes exatas para builds reproduciveis	Pin exato de cada dependencia e sub-dependencia

Packaging Windows (`packaging/windows_installer.iss`):

- Installer Inno Setup com wizard grafico
- Icone personalizado CS2 Analyzer
- Atalho no desktop e no menu Start
- Tamanho estimado: ~500MB (inclui PyTorch e modelos)
- Target: Windows 10/11 64-bit

Scripts de Setup:

Script	Proposito
<code>setup_new_pc.bat</code>	Configuracao de ambiente do zero: Python, pip, venv, dependencias
<code>export_env.bat</code>	Exportacao de variaveis de ambiente necessarias
<code>run_full_training_cycle.py</code>	Orquestracao ciclo completo de treinamento ML

12.22 Sistema de Migracoes Alembic

Diretorio: `alembic/` (root) + `Programma_CS2_RENAN/backend/storage/migrations/`

O projeto utiliza **dois setups Alembic separados** para gerenciar as migracoes do schema do banco de forma organizada.

Alembic Root (`alembic/` , 13 versoes):

Contem as migracoes principais do schema, desde a criacao inicial das tabelas ate as modificacoes mais recentes. Cada migracao:

- Tem um hash unico como identificador
- Contem `upgrade()` (aplica modificacao) e `downgrade()` (desfaz modificacao)
- E idempotente — pode ser reaplicada sem erros
- E testada em schema production-like antes do deploy

Backend Migrations (`backend/storage/migrations/` , 2 versoes):

Migracoes especificas para a storage layer, separadas por modularidade. Gerenciam modificacoes em tabelas que sao responsabilidade exclusiva do modulo `backend/storage/`.

Execucao automatica: As migracoes sao executadas automaticamente durante a fase 3 da sequencia de inicializacao (`main.py`), garantindo que o schema esteja sempre atualizado antes de qualquer operacao DB.

12.23 Orquestrador de Ingestao Principal (`run_ingestion.py`)

Arquivo: `Programma_CS2_RENAN/run_ingestion.py`

O `run_ingestion.py` e o **coracao orquestrador** da pipeline de ingestao inteira. E o arquivo maior dedicado a ingestao e coordena todas as fases da descoberta das demos a persistencia dos resultados no banco.

12+ funcoes principais:

Funcao	Papel
<code>_check_duplicate_demo()</code>	Deduplicacao SHA-256 em 3 arquivos: IngestionTask (caminho exato), PlayerMatchStats (por stem) e Registry
<code>_ingest_single_demo()</code>	Pipeline completa para uma demo: parse agregado → <code>persist_round_stats_and_enrichment()</code> → salvamento do shard por-match
<code>_save_player_stats()</code>	Persiste <code>PlayerMatchStats</code> com sanitizacao de NaN/Inf (<code>_sanitize_value</code>)
<code>_save_sequential_data()</code>	Extracao de ticks em chunks: <code>BATCH_SIZE = 10000</code> se <code>HP_MODE=1</code> , senao <code>2000</code> ; dual-write no DB por-match
<code>enrich_tick_data()</code>	(de <code>backend/processing/tick_enrichment.py</code>) calcula as features cross-player 20-24 por tick
<code>_EventExtractor</code> / <code>_extract_and_store_events()</code>	Extrai weapon_fire/hurt/death/granadas/bomba → <code>MatchEventState</code> no DB por-match
<code>_build_match_tick_dataframe()</code>	DataFrame de ticks para todos os jogadores (POV completo)
<code>_finalize_match_record()</code>	Fechamento do registro da partida e metadados
<code>run_ml_pipeline()</code> / <code>_save_insights()</code>	ML de follow-on e persistencia de <code>CoachingInsight</code>

ResourceManager (`ingestion/resource_manager.py`):

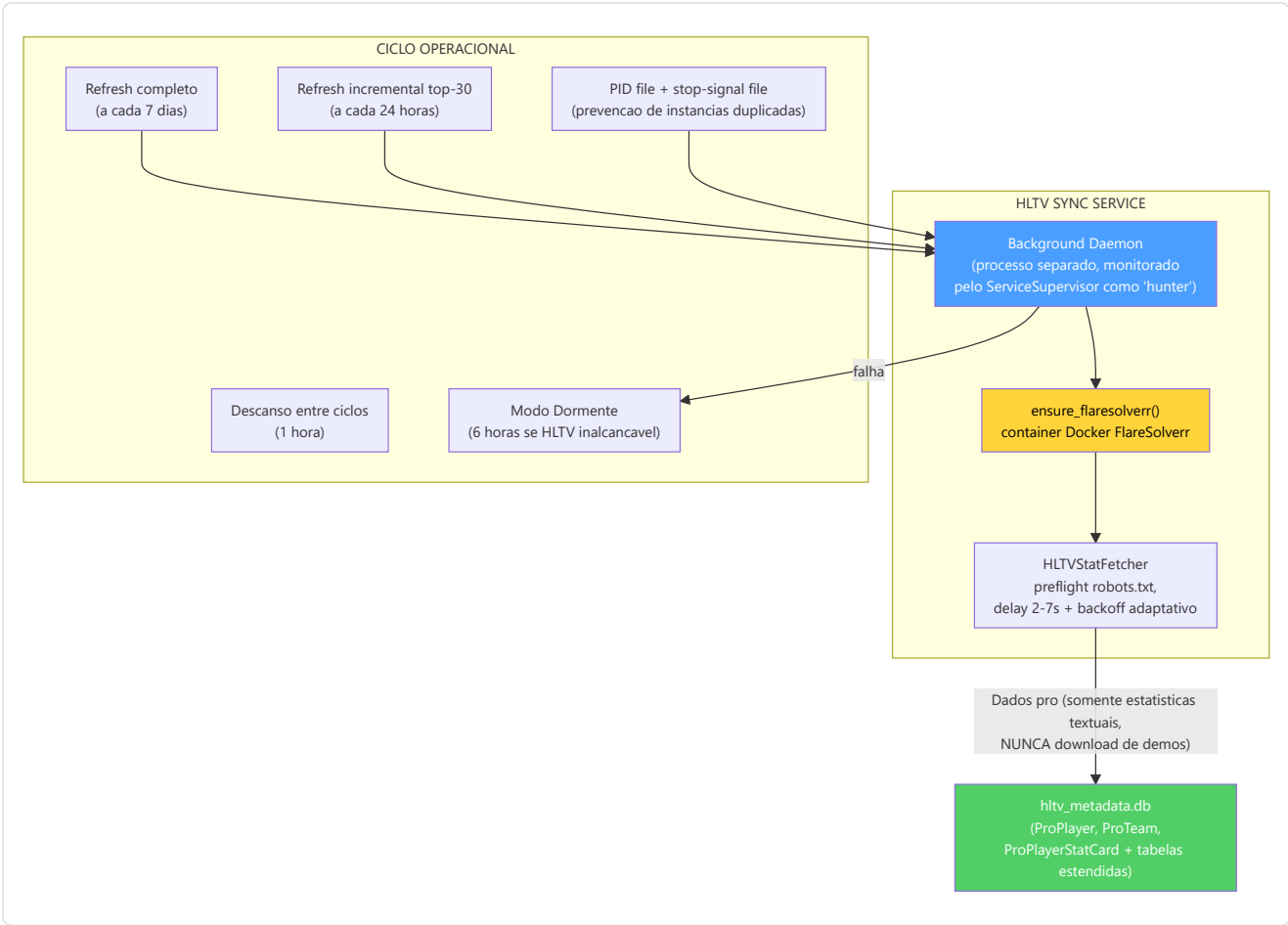
O ResourceManager gerencia os **recursos de hardware** durante a ingestao para evitar a sobrecarga do sistema:

Parametro	Valor Default	Proposito
CPU threshold	80%	Pausa ingestao se CPU supera esse limite
RAM threshold	85%	Pausa ingestao se RAM supera esse limite
Disk check	Sim	Verifica espaco disco suficiente antes do processing
Throttle delay	2s	Delay entre tasks consecutivos para resfriamento

12.24 HLTV Sync Service e Background Daemon

Arquivo: `Programma_CS2_RENAN/hltv_sync_service.py` **Arquivos relacionados:** `backend/data_sources/hltv/`, `backend/services/telemetry_client.py`

O HLTV Sync Service e um **daemon em background** que sincroniza automaticamente os dados dos jogadores profissionais da HLTV.org. Opera como um servico monitorado pelo `ServiceSupervisor` da Console.



Politica de refresh: sincronizacao **completa a cada 7 dias**, **incremental dos top-30 a cada 24 horas**, descanso de 1 hora entre os ciclos, e **modo dormente de 6 horas** quando a HLTV nao esta alcançavel. O servico escreve exclusivamente **estatísticas textuais** (Rating 2.0, K/D, ADR, KAST, HS%) — nunca baixa arquivos de demo. Roda como processo detached (`subprocess.Popen`) com **PID file** e **stop-signal file** (`start_detached` / `stop_service`), separado do session engine para evitar contencao WAL no monolite.

12.25 RASP Guard — Integridade Runtime do Codigo

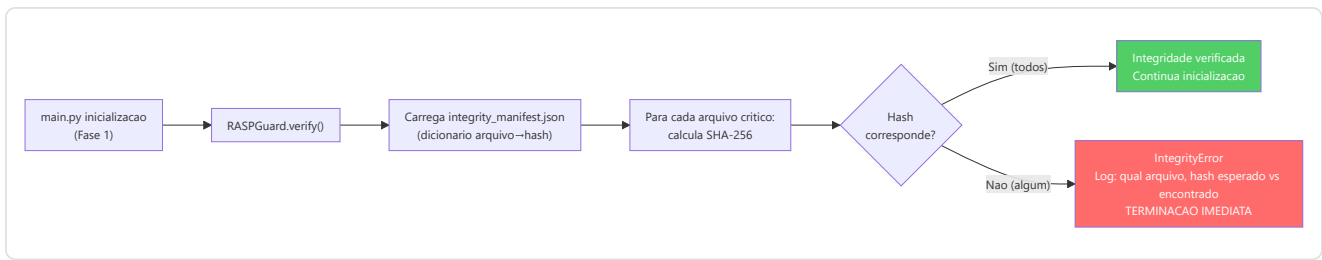
Arquivo: `Programma_CS2_RENAN/observability/rasp.py` **Arquivo de dados:** `data/integrity_manifest.json`

O RASP (Runtime Application Self-Protection) Guard e a **primeira verificacao** executada na inicializacao da aplicacao (Fase 1 da sequencia de boot). Verifica que nenhum arquivo fonte tenha sido modificado em relacao ao manifesto de integridade.

Componentes do RASP Guard:

Componente	Descricao
<code>RASPGuard</code>	Classe principal — carrega o manifesto, verifica hash, levanta execucao
<code>integrity_manifest.json</code>	Arquivo JSON com hash SHA-256 de cada arquivo fonte critico
<code>IntegrityError</code>	Execucao custom — terminacao imediata com log detalhado
<code>sync_integrity_manifest.py</code>	Ferramenta para atualizar o manifesto apos modificacoes legitimas

Fluxo de verificacao:



12.26 MatchVisualizer — Rendering Avancado

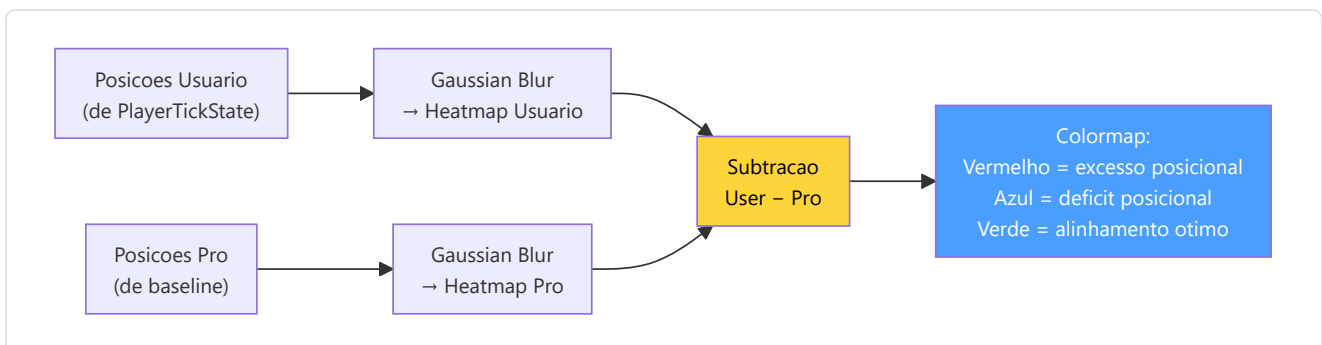
Arquivo: `Programma_CS2_RENAN/reporting/visualizer.py`

O `MatchVisualizer` estende o sistema de reporting (secao 12.13) com **6 metodos de rendering** especializados para a geracao de graficos e visualizacoes.

6 Metodos de rendering:

Metodo	Output	Descricao
<code>render_skill_radar()</code>	Radar chart PNG	Pentagono de 5 eixos (Mecanica, Posicionamento, Utility, Timing, Decisao) com overlay pro
<code>render_trend_chart()</code>	Line chart PNG	Tendencia historica de metricas-chave (KPR, ADR, KAST) em N partidas
<code>render_heatmap()</code>	Heatmap PNG	Ocupacao gaussiana 2D no mapa tatico
<code>render_differential_heatmap()</code>	Heatmap diff PNG	Subtracao user-pro: vermelho=demais, azul=pouco, verde=otimo
<code>render_critical_moments()</code>	Annotated timeline PNG	Timeline com marcadores coloridos para momentos-chave (kills, deaths, plantagens)
<code>render_comparison_table()</code>	Table PNG/HTML	Tabela comparacao user vs pro com delta percentual para cada metrica

Heatmap diferencial — algoritmo:



12.27 Arquivos de Dados e Configuracao Runtime

O projeto inclui varios arquivos de dados que sao lidos ou escritos durante a execucao. Esses arquivos nao sao codigo Python mas sao essenciais para o funcionamento do sistema.

`data/map_config.json` — Configuracao dos mapas CS2:

Contem os parametros de transformacao de coordenadas para cada mapa suportado (Dust2, Mirage, Inferno, Nuke, Vertigo, Overpass, Ancient, Anubis):


```
{
  "de_dust2": {
    "pos_x": -2476, "pos_y": 3239, "scale": 4.4,
    "z_cutoff": null, "level": "single"
  },
  "de_nuke": {
    "pos_x": -3453, "pos_y": 2887, "scale": 7.0,
    "z_cutoff": -495, "level": "multi"
  }
}
```

data/integrity_manifest.json — Manifesto de integridade RASP:

Dicionario `{caminho_arquivo: hash_sha256}` para todos os arquivos Python criticos. Gerado/atualizado por `tools/sync_integrity_manifest.py`. Verificado na inicializacao por **RASPGuard** (secao 12.25).

data/training_progress.json — Progresso de treinamento:

Persiste o estado do training entre reinicios: epoca atual, melhor loss, learning rate, ultima demo processada. Consultado pelo Teacher daemon para retomar o training do ponto exato em que parou.

user_settings.json — Configuracoes do usuario:

Arquivo de configuracao nivel 2 (cf. secao 12.3). Salvo na diretorio do projeto, contem preferencias personalizaveis: paths demo, tema UI, idioma, flags funcionalidade.

12.28 Entry Point Root-Level

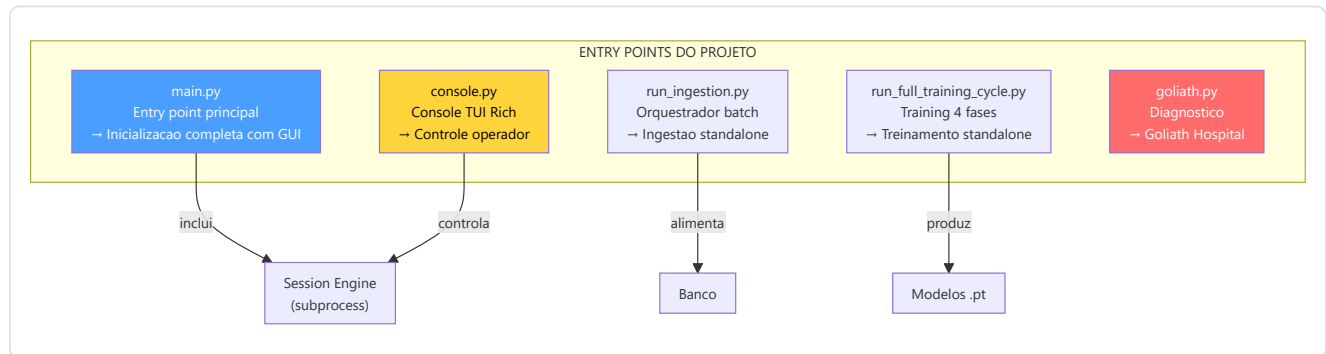
Alem de `main.py`, o projeto inclui varios **scripts executaveis** em nivel root que servem como pontos de entrada alternativos para operacoes especificas:

Script	Posicao	Proposito	Invocacao
<code>console.py</code>	raiz	"MACENA UNIFIED CONSOLE v3.0": TUI Rich + CLI argparse, CommandRegistry com 10 categorias (ml, ingest, build, db, ...)	<code>python console.py</code>
<code>run_ingestion.py</code>	<code>Programma_CS2_RENAN/</code>	Orquestrador de ingestao standalone (cf. 12.23)	<code>python -m Programma_CS2_RENAN.run_ingestion</code>
<code>batch_ingest.py</code>	raiz	Driver batch paralelo de ingestao (delega a <code>run_ingestion</code>)	<code>python batch_ingest.py / ./ingest.sh</code>
<code>goliath.py</code>	raiz	"MACENA GOLIATH": orquestrador Rich com subcomandos <code>build</code> , <code>sanitize</code> , <code>integrity</code> , <code>audit</code> , <code>db</code> , <code>doctor</code>	<code>python goliath.py <subcomando></code>
<code>schema.py</code>	raiz	SchemaSuite : CLI sqlite3 raw (nao ORM) de inspecao/migracao do DB — subcomandos <code>inspect</code> , <code>migrate</code> , <code>verify</code>	<code>python schema.py <subcomando></code>
<code>run_full_training_cycle.py</code>	raiz	Treinamento completo standalone (JEPA→Pro→User→RAP→RoleHead) com flags CLI (cf. Parte 1A)	<code>python run_full_training_cycle.py</code>
<code>run_worker.py</code>	<code>Programma_CS2_RENAN/</code>	Worker de ingestao com claim atomico das tarefas e recuperacao de stale	<code>python -m Programma_CS2_RENAN.run_worker</code>

Console interativo (`console.py` , ~61KB):

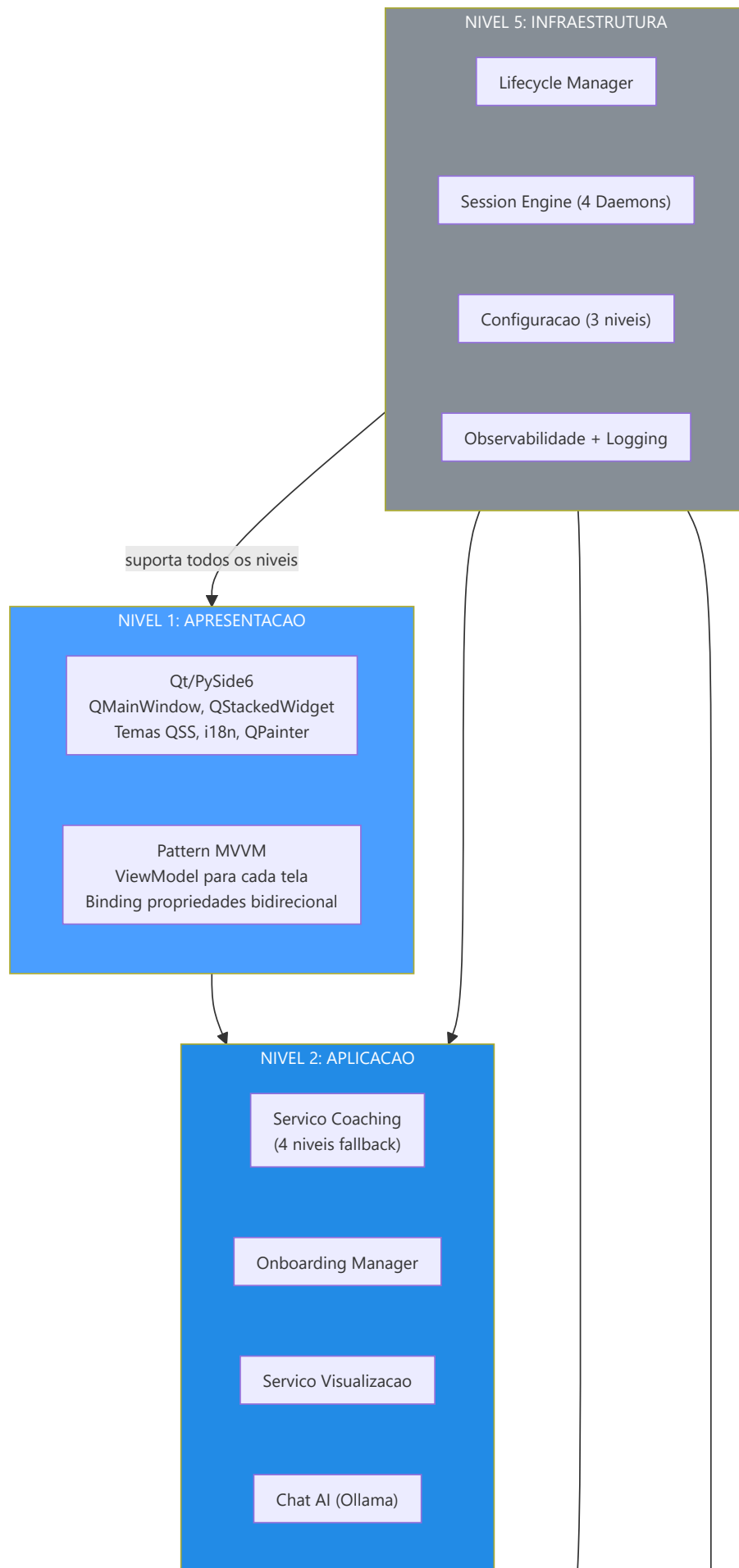
O console e o **ponto de entrada mais potente** para operadores experientes. Oferece uma TUI (Terminal User Interface) baseada em Rich com:

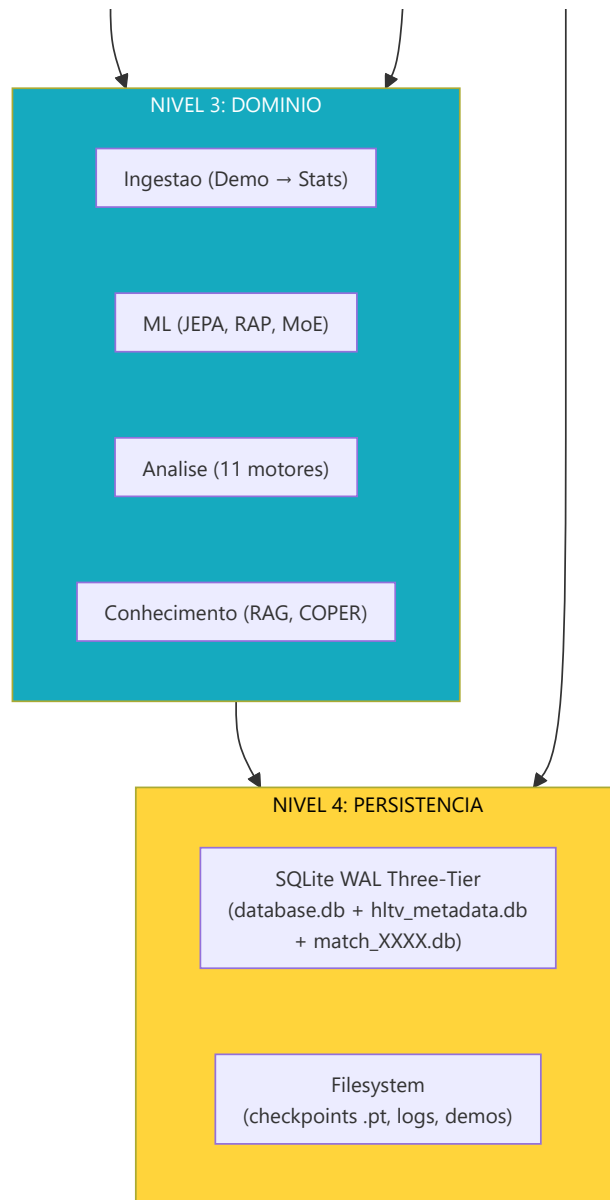
- **Registro de comandos:** Sistema registrado de comandos com autocompletamento
- **Modo CLI:** Execucao single-shot de comandos (`console.py status` , `console.py train`)
- **Modo Interativo:** Shell REPL com prompt personalizado e historico de comandos
- **Paineis Rich:** Dashboard com tabelas coloridas, progress bar, tree view do estado



Resumo Arquitetural

Macena CS2 Analyzer e uma aplicacao **estratificada e modular** organizada em 5 niveis arquiteturais com 3 processos separados.





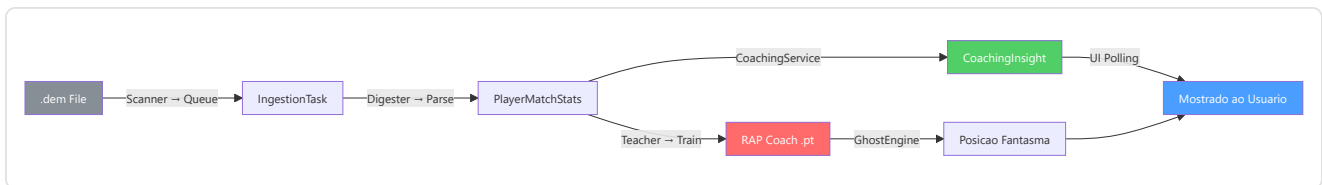
Stack tecnologico completo (28 dependencias):

Categoria	Biblioteca	Versao	Papel no projeto
ML Core	PyTorch	2.x	Redes neurais (JEPA, RAP, MoE, NeuralRoleHead, WinProb)
ML Ext	ncps	latest	Liquid Time-Constant Networks (LTC) para memoria RAP
ML Ext	hflayers	latest	Hopfield layers para atencao associativa
UI	PySide6	6.x	Bindings Qt 6 para Python — framework UI desktop primario
DB	SQLModel	latest	ORM (Pydantic + SQLAlchemy)
DB	SQLAlchemy	2.x	Engine banco subjacente
DB	Alembic	1.x	Migracoes schema
HTTP	requests	2.x	API Steam/FACEIT (sincrono)
HTTP	httpx	latest	Telemetria (assincrono)
Scraping	playwright	latest	Browser headless para HLTV
Parsing	demoparser2	latest	Parser demo CS2 (Rust-based, rapido)
Data	pandas	2.x	Manipulacao dados tabelares
Data	numpy	1.x	Calculos numericos
Viz	matplotlib	3.x	Graficos e visualizacoes
NLP	sentence-transformers	latest	Embedding 384-dim para RAG
TUI	rich	latest	Console colorida, tabelas, progress bar
Logging	logging (stdlib)	—	Logging estruturado
Security	keyring	latest	Gestao segura de API keys
Monitoring	sentry-sdk	latest	Error tracking remoto (opcional)
Testing	pytest	latest	Framework test
Formatting	black	latest	Formatacao codigo
Import	isort	latest	Ordenacao imports
QA	pre-commit	latest	Hooks pre-commit
Crypto	hashlib (stdlib)	—	SHA-256 para RASP manifesto
Concurrency	threading (stdlib)	—	Daemon thread, MLControlContext
IPC	subprocess (stdlib)	—	Session Engine como subprocess
Config	json (stdlib)	—	user_settings.json, map_config.json

Os 3 processos da aplicacao:

Processo	Tipo	Responsabilidades	Comunicacao
Main	Qt GUI	Interface usuario, rendering, interacao	Polling DB a cada 10-15s
Daemon	Subprocess (Session Engine)	Scanner, Digester, Teacher, Pulse	stdin pipe (IPC) + DB compartilhado
Servicos Opcionais	Processos externos	HLTV sync, Ollama LLM local	HTTP/API + supervisao Console

Fluxos de dados principais:



Nota sobre Remediacao — Codigo Eliminado (G-06)

Durante o processo de remediacao em 12 fases (370+ problemas resolvidos no total — cf. secao 12.19), a diretoria `backend/nn/advanced/` foi **completamente eliminada** pois continha codigo morto nao referenciado:

- `superposition_net.py` — Uma rede de superposicao experimental nunca integrada ao fluxo de treinamento ou inferencia. A funcionalidade de superposicao ativa e implementada em `layers/superposition.py` (usada pela camada Estrategia RAP).
- `brain_bridge.py` — Uma ponte entre modelos experimental nunca chamada por nenhum modulo.
- `feature_engineering.py` — Feature engineering duplicada; a versao canonica reside em `backend/processing/feature_engineering/`.

Motivacao (G-06): Manter codigo morto cria riscos de confusao (qual `superposition` e o verdadeiro?), aumenta a superficie de manutencao e pode introduzir imports acidentais. A eliminacao foi verificada via analise estatica das dependencias: nenhum arquivo do projeto importava de `backend/nn/advanced/`.

Utilidades Compartilhadas — `round_utils.py`

Arquivo: `Programma_CS2_RENAN/backend/knowledge/round_utils.py`

A funcao `infer_round_phase(equipment_value)` e uma **utilidade compartilhada** usada tanto pelo servico de coaching quanto pelo sistema de conhecimento para classificar a fase economica de um round. Reside em `backend/knowledge/` e e importada de modulos em `services/` e `processing/`.

Valor equipamento	Fase retornada
< \$1.500	"pistol"
\$1.500 – \$2.999	"eco"
\$3.000 – \$3.999	"force"
>= \$4.000	"full_buy"

Utilidades Core — Lock Files e Plataforma

Lock Files (`core/lock_files.py`):

Sistema de lock file para concorrencia entre processos D-track / HLTV-track. Utiliza um diretorio `.locks/` local ao repositorio (sobrevive as sessoes, nao aos reinicios).

Componente	Descricao
Formato lock file	<code><pid> <iso_timestamp></code> — identifica o processo proprietario
Lock stale	Recuperados automaticamente se o PID proprietario esta morto (<code>os.kill(pid, 0)</code>)
<code>LockConflict(RuntimeError)</code>	Levantada quando o lock e detido por um processo vivo
<code>acquire(name) → Path</code>	Cria lock, verifica conflitos, recupera stale, escreve PID + timestamp
<code>release(name)</code>	Remove lock file. Idempotente
<code>lock(name)</code> (context manager)	Acquire na entrada, release na saida
Signal handlers	<code>install_signal_handlers()</code> registra handlers SIGTERM/SIGINT que liberam todos os locks antes da terminacao
<code>_held_locks: Set[str]</code>	Estado module-level que rastreia os locks atualmente detidos

Pontos Fortes da Arquitetura

- Contrato de funcionalidade unificado a 25 dimensoes** — `METADATA_DIM = 25` impoe a paridade de treinamento/inferencia ao nivel de sistema.
- Gating de maturidade em 3 niveis** — Previne a implementacao prematura do modelo com dados insuficientes.
- Fallback de coaching em 4 niveis** — COPER → Hibrido → RAG → Base garante sempre a entrega de insights.
- Diversidade multi-modelo** — JEPA, VL-JEPA, LSTM+MoE, RAP e NeuralRoleHead contribuem com biases indutivos complementares.
- Subdivisao temporal** — Previne a perda de dados garantindo a ordenacao cronologica.
- Ciclo de feedback COPER** — Monitoramento da eficacia baseado em EMA com decaimento da experiencia obsoleta.
- Suite de analise de Fase 6** — 11 motores de analise (papel, probabilidade de vitoria, arvore de jogo, conviccao, engano, momentum, entropia, pontos cegos, utility e economia, distancia de engajamento, qualidade de movimento).
- Persistencia do limite** — Os limites de papel sobrevivem aos reinicios atraves da tabela DB `RoleThresholdRecord`.
- Heuristica configuravel** — `HeuristicConfig` externaliza os limites de normalizacao em JSON.
- Polishing LLM** — Integracao opcional com Ollama para narrativas de coaching em linguagem natural.
- Training Observatory** — Introspeccao em 4 niveis (Callback, TensorBoard, Maturity State Machine, Embedding Projector) com impacto zero quando desabilitada e callbacks isoladas dos erros.
- Neural Role Consensus** — Dupla classificacao heuristica + NeuralRoleHead MLP com protecao cold-start, que garante uma atribuicao de papeis confiavel mesmo com dados parciais.
- Per-Round Statistical Isolation** — O modelo `RoundStats` impede a contaminacao entre rounds, permitindo um coaching granular a nivel de round e avaliacoes HLTV 2.0 por round.
- Arquitetura Quad-Daemon** — Separacao completa entre GUI e trabalho pesado, com shutdown coordenado e zombie task cleanup automatico.
- Degradacao gradual pervasiva** — Cada componente tem um plano de fallback: o sistema nunca crasha, sempre degrada de forma controlada.
- Arquitetura Three-Tier Storage** — Separacao de `database.db` (core + conhecimento, 18 tabelas), `hltv_metadata.db` (dados pro, 3 tabelas) e `match_data/{id}.db` (telemetria por-match) para eliminar a contencao WAL e prevenir o crescimento descontrolado do monolite.
- Calibracao Bayesiana Live (G-07)** — O estimador de death se auto-calibra com `extract_death_events_from_db()` → `auto_calibrate()`, transformando-se de modelo estatico em sistema adaptativo.
- Controle Live Treinamento (MLControlContext)** — Pause/resume/stop/throttle em tempo real do training via `threading.Event`, com execucao custom `TrainingStopRequested` no lugar de `StopIteration`.

19. **Circuit Breaker Resiliente** — `_CircuitBreaker` para APIs externas (HLTV) com MAX_FAILURES=10, RESET_WINDOW_S=3600, previne cascade failure com pattern CLOSED→OPEN→HALF_OPEN.
20. **Piramide de Validacao em 5 Niveis** — Headless Validator → pytest → Backend Validator → Goliath Hospital → Brain Verify: cada nivel progressivamente mais profundo, do quick smoke test (10s) ao audit completo de 118 regras.
21. **RASP Guard** — Runtime Application Self-Protection com manifesto SHA-256: verifica integridade do codigo fonte na inicializacao, previne manipulacoes acidentais e intencionais.
22. **Pre-commit Gate de 13 Hooks** — 4 hooks custom + 7 padroes + 2 Python impedem que codigo nao conforme chegue ao repositório. Formatacao, validacao, dead code, integridade, merge conflict e private key verificados automaticamente.
23. **ResourceManager Hardware-Aware** — Throttling automatico CPU/RAM durante ingestao: o sistema desacelera autonomamente se os recursos de hardware estao sob pressao, sem intervencao humana.
24. **Test Forensics** — 10 scripts de investigacao post-mortem para diagnosticar training failure, weight anomalies, coaching path traces e DB consistency — a "caixa preta" do sistema.

PONTOS FORTES ARQUITETURAIS - OS 24 PILARES

1. Contrato unificado 25-dim - Todos falam a mesma lingua

2. Gate maturidade 3 niveis - Nenhum release prematuro

3. Fallback coaching 4 niveis - Nunca de maos vazias

4. Diversidade multi-modelo - 5 cerebros > 1 cerebro

5. Divisao temporal - Nenhum truque viagens no tempo

6. Loop feedback COPER - Aprende com seus proprios conselhos

7. Analise Fase 6 (10 mot.) - 10 detetives especializados

8. Persistencia limites - Sobrevive aos reinicios

9. Heurísticas configuraveis - Override via JSON

10. Refinamento LLM (Ollama) - Conselhos soam naturais

11. Observatorio Treinamento - Boletim para o cerebro

12. Consenso Neural Papeis - Dois professores comparam notas

13. Isolamento Per-Round - Avalia cada questao, nao apenas o teste

14. Arquitetura Quad-Daemon - GUI responsiva, trabalho pesado em background

15. Degradacao Gradual - O sistema nunca crasha

16. Three-Tier Storage - Nenhuma contencao entre escrita e leitura

17. Calibracao Bayesiana Live - Auto-calibra com os dados

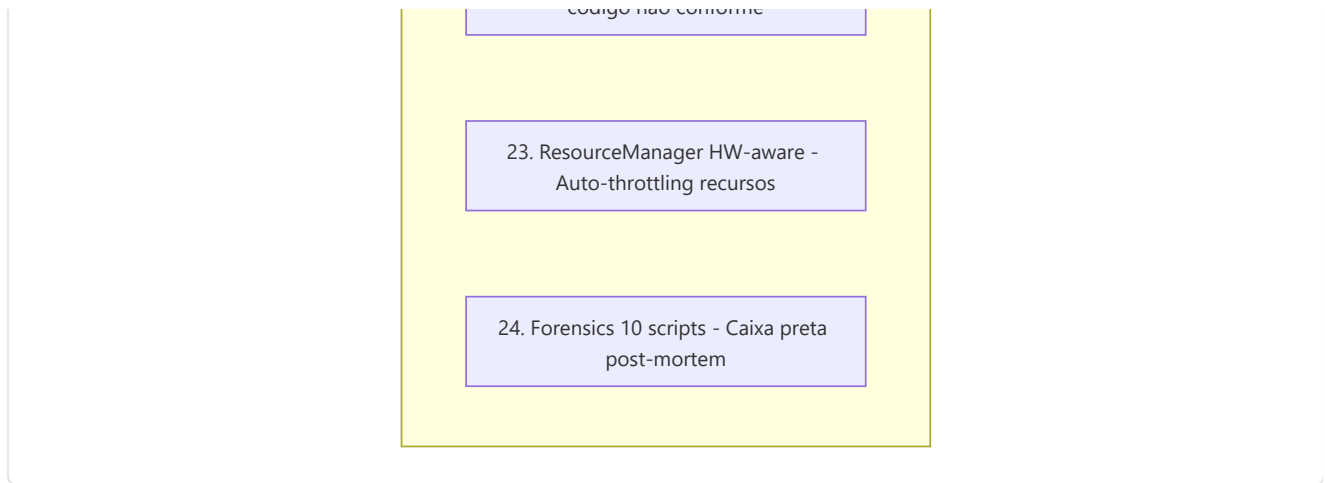
18. Controle Live Training - Pausa/Stop sem perda

19. Circuit Breaker - Resiliencia APIs externas

20. Piramide Validacao 5 niveis - Do quick test ao deep audit

21. RASP Guard SHA-256 - Integridade runtime garantida

22. Pre-commit 13 hooks - Zero codigo nao conforme

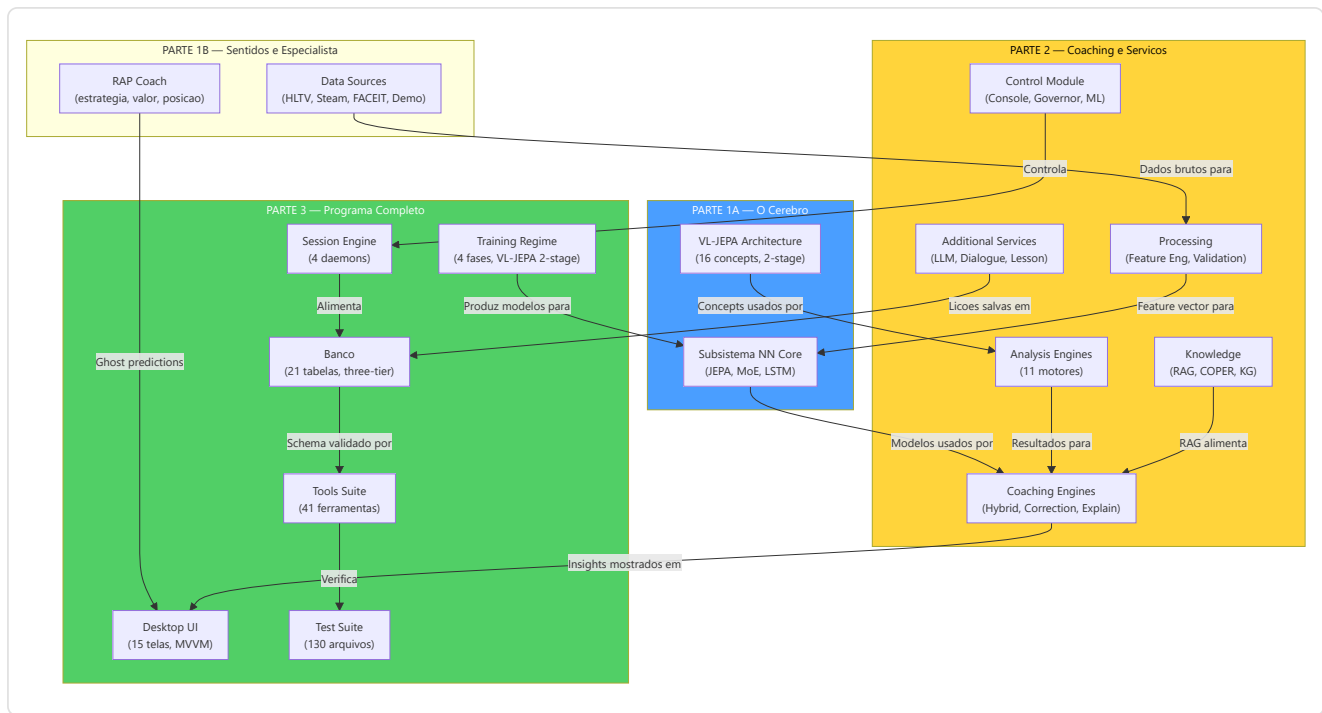


Fim do documento — Guia completo do Macena CS2 Analyzer

Total arquivos `.py` no projeto: **445** (em `Programma_CS2_RENAN/`) Total linhas de código Python verificadas: ~ **102.000+**
 Subsistemas AI cobertos: **8** (NN Core, VL-JEPA, RAP Coach, Servicos de Coaching, Motores de Coaching, Conhecimento, Analise, Processamento + Observatorio Treinamento) Subsistemas programa cobertos: **18** (Inicializacao, Lifecycle, Configuracao, Session Engine, UI Desktop, Ingestao, Storage, Observabilidade, Console de Controle, RASP Guard, HLTV Sync, Orquestrador Ingestao, ResourceManager, Tools Suite, Test Suite, Pre-commit, Build/Packaging, Migracoes Alembic)
 Modelos documentados: **6** (AdvancedCoachNN/TeacherRefinementNN, JEPA, VL-JEPA, RAPCoachModel, NeuralRoleHead, WinProbabilityNN) Motores de analise documentados: **10** (Papel, WinProb, GameTree, Crenca, Engano, Momentum, Entropia, Pontos Cegos, Utility e Economia, Distancia de Engajamento) Motores de coaching documentados: **7** (HybridEngine, CorrectionEngine, ExplainabilityGenerator, NNRefinement, ProBridge, TokenResolver, LongitudinalEngine)
 Servicos adicionais documentados: **7** (CoachingDialogue, LessonGenerator, LLMSERVICE, VisualizationService, ProfileService, AnalysisService, TelemetryClient) Tabelas do banco documentadas: **21** (distribuidas em arquitetura three-tier storage: `database.db`, `hlvtv_metadata.db`, `match_data/{id}.db`) Telas UI documentadas: **15** (Wizard, Home, Coach, Tactical Viewer, Settings, Help, Match History, Match Detail, Performance, User Profile, Profile, Pro Comparison, Steam Config, FACEIT Config, Pro Player Detail) — Qt/PySide6 Daemons documentados: **4** (Scanner, Digester, Teacher, Pulse) Ferramentas de validacao documentadas: **41** (Headless Validator, Brain Verify, Goliath Hospital, DB Inspector, Demo Inspector, ML Coach Debugger, Backend Validator, Dead Code Detector, Dev Health, Feature Audit, etc.) Arquivos de teste documentados: **130** (+ `confTest.py`, 10 forensics, 15 verification scripts) Fases headless validator: **23** (319 verificacoes automatizadas) Pre-commit hooks documentados: **13** (4 locais custom + 7 padrao + 2 Python) Pilares arquiteturais: **24** (incluidos Three-Tier Storage, Calibracao Bayesiana Live, Controle Live Training, Circuit Breaker, Piramide Validacao, RASP Guard, Pre-commit Gate, ResourceManager HW-aware, Forensics) Problemas resolvidos via remediacao: **368** (em 12 fases sistematicas) Fases de remediacao documentadas: **12** (com codigos G-XX e F-XX)

Mapa das Interconexoes entre as 3 Partes

As tres partes da documentacao formam um sistema interconectado. Este mapa mostra as principais dependencias entre as secoes:



Referencias cruzadas chave:

De (Parte)	Para (Parte)	Ligacao
Parte 1 Secao VL-JEPA	Parte 3 Secao 10 (Training)	O protocolo two-stage descrito na Parte 1 e implementado pelo training regime na Parte 3
Parte 1 Secao Data Sources	Parte 3 Secao 12.6 (Ingestion)	As fontes de dados alimentam a pipeline de ingestao
Parte 2 Secao Coaching Engines	Parte 3 Secao 12.16 Fluxo 1	Os motores de coaching geram os insights mostrados na UI
Parte 2 Secao Control Module	Parte 3 Secao 12.4 (Session Engine)	O modulo de controle gerencia os 4 daemons
Parte 2 Secao Processing	Parte 1 Secao NN Core	O vectorizer produz o METADATA_DIM=25 consumido pelos modelos
Parte 3 Secao 9 (Banco)	Parte 2 Secao Knowledge	As tabelas RAG/COPER sao usadas pelo sistema de conhecimento
Parte 3 Secao 11 (Loss)	Parte 1 Secao VL-JEPA	As loss functions documentadas implementam o treinamento VL-JEPA
Parte 3 Secao 12.17 (Tools)	Parte 3 Secao 12.18 (Tests)	A piramide de validacao inclui tanto tools quanto test suite

Glossario Tecnico

Termo	Definicao
ADR	Average Damage per Round — dano medio causado por round
BCE	Binary Cross-Entropy — loss para classificacao binaria
COPER	Coaching through Past Experiences and References — coaching baseado em experiencias passadas
EMA	Exponential Moving Average — media movel exponencial usada para target encoder JEPA
HLTV 2.0	Sistema de rating do HLTV.org baseado em KPR, survival, impact, damage
InfoNCE	Noise Contrastive Estimation — loss contrastiva para alinhar representacoes
JEPA	Joint-Embedding Predictive Architecture — arquitetura que preve embeddings target de contexto
KAST	Kill/Assist/Survive/Trade — percentual de rounds com contribuicao positiva
KG	Knowledge Graph — grafo de conhecimento com entidades e relacoes
KPR	Kills Per Round — kills medios por round
LTC	Liquid Time-Constant — redes neurais com constantes temporais aprendidas
METADATA_DIM	Dimensao do feature vector unificado (25 no projeto)
MoE	Mixture of Experts — rede com especialistas especializados e gating
MVVM	Model-View-ViewModel — pattern arquitetural para separacao UI/logica
RAG	Retrieval-Augmented Generation — geracao enriquecida por retrieval
RAP	Reasoning Action Planning — modelo para coaching estrategico
RASP	Runtime Application Self-Protection — protecao runtime da aplicacao
TUI	Terminal User Interface — interface de usuario no terminal
VICReg	Variance-Invariance-Covariance Regularization — regularizacao para diversidade de embedding
VL-JEPA	Vision-Language JEPA — extensao com conceitos linguisticos
WAL	Write-Ahead Logging — modo SQLite para leituras/escritas concorrentes
Z-score	Numero de desvios-padrao da media — mede o quanto um valor esta longe da norma

Autor: Renan Augusto Macena